EasyLink Manual Guide

V1.16.04
Date: May 23, 2024

Revision history

Version	Date	Author	Comments
V1.00.00	Oct 26, 2016	ZhangYuan	Initial version
V1.00.01	Nov 03, 2016	ZhangYuan	1. Update description for UI;
V1.00.02	Dec 29, 2016	ZhangYuan	1. Update §Appendix3 – Transaction parameter list
V1.00.03	April 18, 2017	ZhangYuan	1. Add diagram for Master Device-POS-Cardholder to fulfill a transaction; 2. Add sample code for each interface;
V1.00.04	April 26, 2017	ZhangYuan	1. Update sample code;
V1.00.05	Aug. 28, 2017	ZhangYuan	1. Update diagram in Chapter 3.
V1.00.06	Oct. 23, 2017	ZhangYuan	1. Update description for supported terminal. 2. Add ExpressPay parameter configuration.
V1.00.07	March 22, 2018	ZhangYuan	1. Update Appendix 2;
v1.01.01	March 28, 2018	ZhangYuan	1. Update Appendix 3
v1.01.02	July 27, 2018	ZhangYuan	1. Update §1.2 Supported devices; 2. Update §2.2 Working pattern; 3. Update §2.3 System components; 4. Update §Appendix 2, add TAG 0214, 0215;
v1.01.03	Sep. 10, 2018	ZhangYuan	1. Update §Appendix 2: add TAG 0216, 0217;
v1.01.04	Dec. 4, 2018	ZhangYuan	1. Update description for §Appendix 2;
v1.01.05	Dec. 13, 2018	ZhangYuan	1. Add §Appendix 4 – Data to be set before StartTransaction; 2. Add §Appendix 5 – Data to be set before CompleteTransaction;
v1.02.00	Jan. 4, 2019	ZhangYuan	1. Add TAG 0219, 021A to §Appendix 2; 2. Add §Appendix 6;
v1.03.00	Feb. 21, 2019	ZhangYuan	1. Add TAG 031F to §Appendix 3;
V1.04.00	June 13, 2019	ZhangYuan	1. Add §4.5.3 Dynamic update transaction parameter for multi-host; 2. Add §Appendix 7 – callBackCfg.report configuration;
V1.05.00	Sept.16, 2019	ZhangYuan	1. Add TAG 021B, 021C, 021D to §Appendix 2; 2. Add §4.5.4;
V1.06.00	Oct.18, 2019	ZhangYuan	1. Add description for §4.6 FileDownload;
V1.07.00	Feb.13, 2020	ZhangYuan	1. Remove duplicate items of TAG 031A and TAG 031B in Appendix 3;
V1.08.00	May 09, 2020	ZhangYuan	1. Add description for §4.5.1 StartTransaction; 2. Add description for §4.6 FileDownload; 3. Add Appendix 8 – Track2 data format;
v1.09.00	June 04, 2020	ZhangYuan	1. Update §Appendix 2-Application parameter list, add TAG 021E, 021F;

v1.10.00	Sept.17, 2020	ZhangYuan	1. Update §Appendix 2-Application parameter list, add TAG 0220;
v1.11.00	Dec. 15, 2020	ZhangYuan	1. Update description for §4.3.2 EncryptData;
v1.12.00	April 8, 2021	ZhangYuan	1. Add <connectionWithEncryption> command to §4.1 Communication chapter; 2. Add TAG 012B to §Appendix 3 Terminal Information List.
V1.13.00	July 7, 2021	ZhangYuan	1. Add §5.2 M30&M50;
v1.14.00	December 23, 2021	ZhangYuan	1. Add §4.3.4 increaseKsn; 2. Update §Appendix 1 – Terminal information list; 3. Update §Appendix 2 – Application parameter list; 4. Update §Appendix 3 – Transaction parameter list;
V1.15.00	March 3, 2022	Jolie Yang	1. Add US feature, add §5 RKI Feature, §Appendix 9; 2. Update §Appendix 1 – Terminal information list; 3. Update §Appendix 2 – Application parameter list; 4. Update §Appendix 3 – Transaction parameter list;
V1.16.00	May 5, 2022	Jolie Yang	1. Add Sensitive Tag List §Appendix 8 2. Add Return code §Appendix 9 3. Update number of "Track 2 Data format" to §Appendix 10 4. Update nember of "Reported data format" to §Appendix 11
V1.16.01	Aug 17, 2022	Jolie Yang	1. Add §4.10 "ClearTransData" 2. Update Application parameter list §Appendix2
V1.16.02	Nov 09, 2022	Chenglin Zhou, Ziqi Liu Jolie Yang	1. Add §6.3 "Sync And Process Task" 2. Add "SyncAndProcessTask Return Code" 3. Add §4.3.6 "CalcMAC", §4.3.3 "ReplaceData" 4. Add §4.6 "Reader module" 5. Update content for §4.10.1 §4.10.2, §4.10.4 6. Add §4.10.5-§4.10.7 7. Update §Appendix2, §Appendix3, §Appendix10,§Appendix12. 8. Add §Appendix6
V1.16.02	Dec 13, 2022	Jolie Yang	Update §Appendix2-Application parameter list: Add Tag 0241
V1.16.03	May 17, 2023	Jolie Yang	Add §4.5.4-Security Instruction Add §4.4.4 GetCardBinIndex Add Tag 0330 for §Appendix3

			Add §4.4.5 CheckCardLuhn Update error code 1004 and 1007 in §Appendix 10
	Jan 25, 2024	Jolie Yang	Add language selection report in in §Appendix 12 Add error code “EL_TRANS_RET_LUHN_NO_CARD_ERR”, “ EL_PARAM_RET_UNSUPPORTED_CARD”
V1.16.04	May 23, 2024	Teng Zhang	Introduce updated APIs, "rkiRemoteDownload" and "rkiUnBindDevice", in the iOS SDK, each including an additional parameter that specifies the path to the "rkiCert.bundle".

Contents

1 Introduction

1.1 Purpose

This document is used to describe the architecture, interfaces, features and the processing flow to make users get familiar with the EasyLink solution.

1.2 Supported devices

Client device (PAX POS terminal):

Monitor platform: D180, D135, SP20V4, S200, M30(M30M), M50(R20);

Prolin platform: D200, D220, Q20, D190, QR55, Exxx(Q20), SKxxx(IMxxx);

PayDroid platform: A920, A930 payment terminal;

Host device:

Android device;

IOS device;

1.3 Audience

All Android or IOS application programmers and customers who need to develop or maintain the application in Android or IOS platform.

1.4 Abbreviation

Name	Description
POS	Point of sale
PDK	PAX Platform Development Kit
SDK	Software development kit
RKI	Remote key injection
PAN	Primary account number
TM	Terminal management
TMS	Terminal management system
AID	Application identifier
CAPK	Certification authority public key

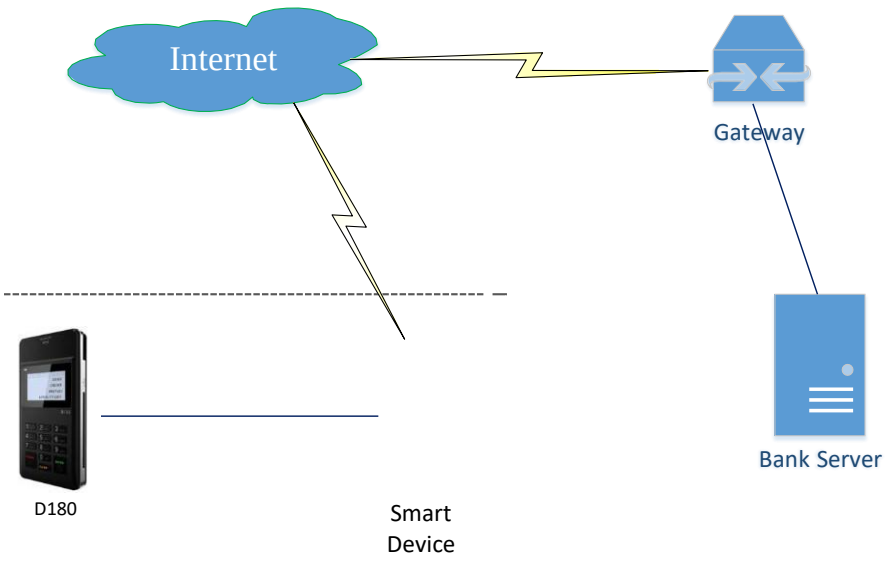
2 System introduction

2.1 Outline

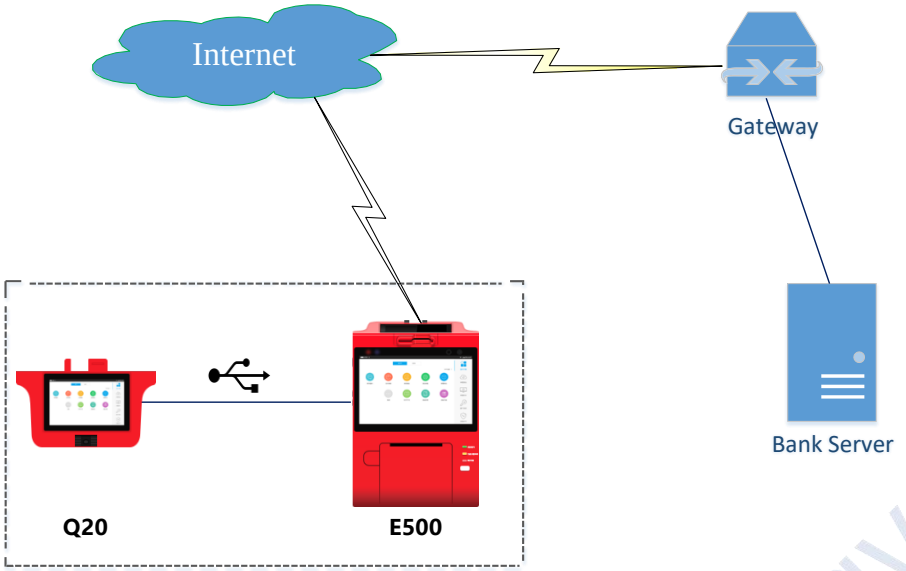
As mobile devices are increasingly being used in conjunction with POS by merchants. In order to ensure safe, simple and smart transaction, PAX launched EasyLink solution.
The EasyLink solution is PAX MPOS solution that the PAX D180, D200, D220, Q20 terminal work in conjunction with Android/IOS phone to perform the functions of a cash register or electronic of point of sale terminal. The EasyLink application (on PAX D180, D200, D220, etc.) provides a set of commands with larger granularity to improve the interaction efficiency and allow quick and easy integration with customized applications in Android/IOS application devices.

2.2 Working pattern

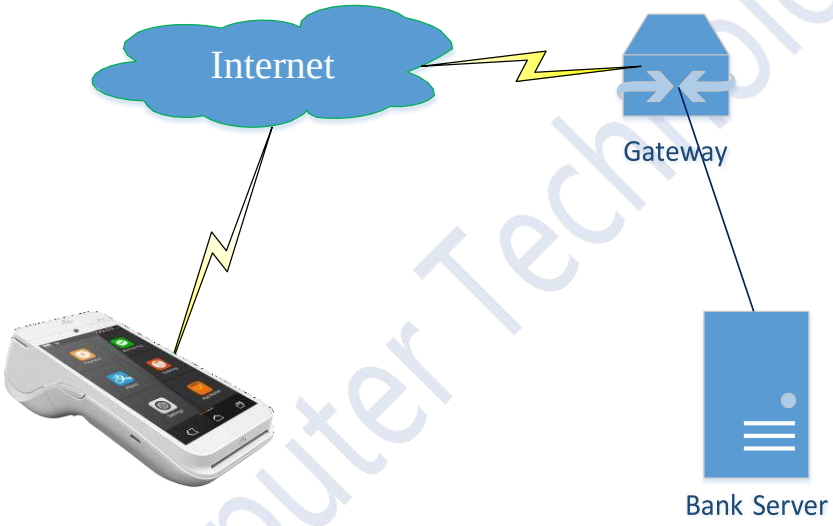
PAX Device mainly refers to PAX D180, D200, D220 which works as slave unit, it provides a set of commands to smart phones;
Smart device (such as Android, iOS devices) works as host unit which conduct the transactions and communicate to the server;
MPOS mode:



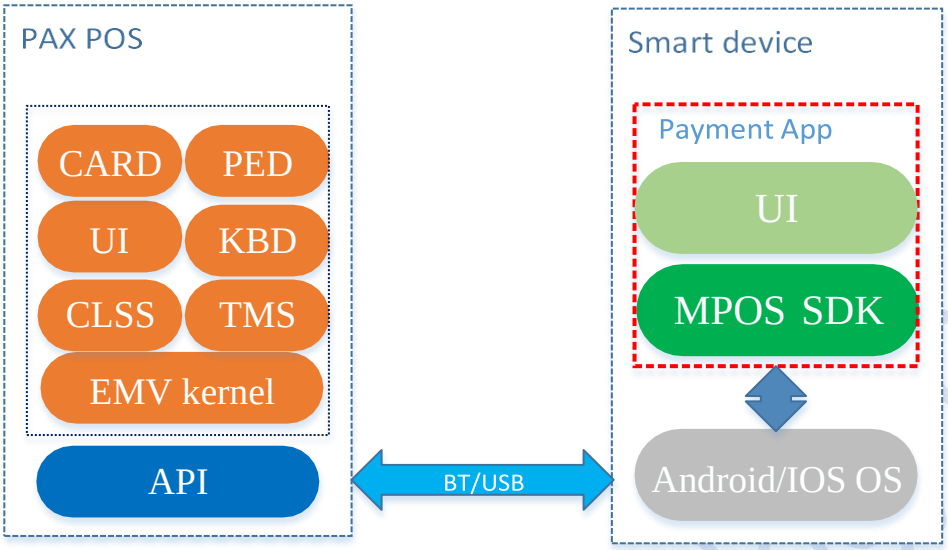
ECR mode:



Payment SDK mode:



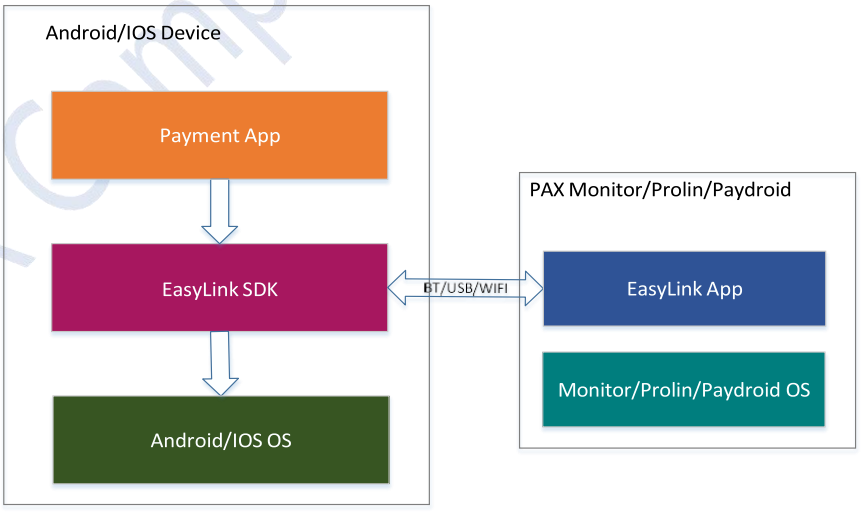
2.3 System components



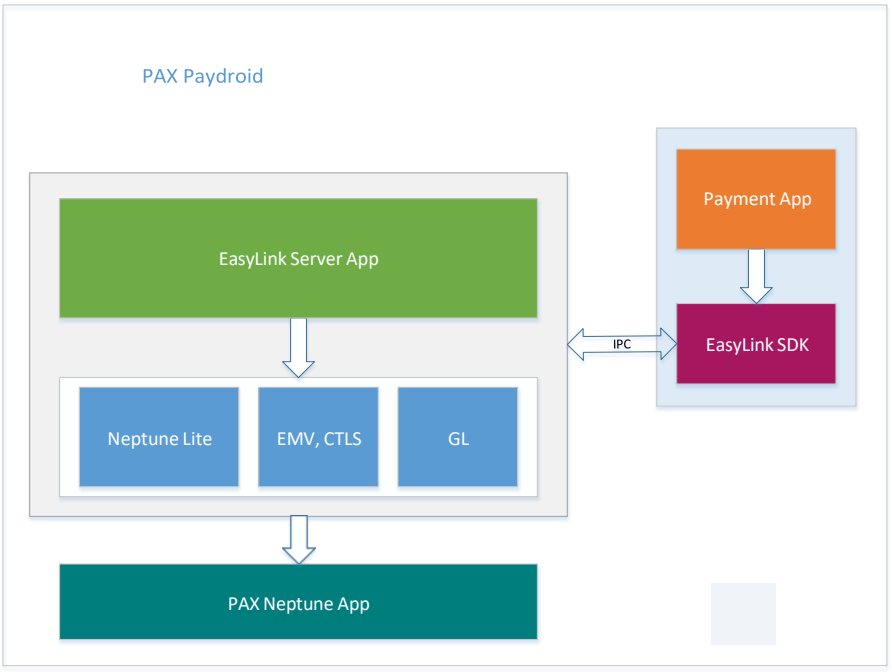
PAX EasyLink solution contains the following parts: EasyLink App (EasyLink Lite version for D180 specially) in PAX device, Android SDK, IOS SDK;

- EasyLink App: EasyLink app is a collection of modules to provide a set of command to smart phone;
- Android SDK: the Android SDK implements the communication and encapsulates the commands to ensure the request and response data been transmitted transparently between PAX device (D180, D200, D220) and Android phone;
- IOS SDK: the same to Android SDK;
- Android/IOS application: developed and maintained by the users to fulfill the payment business requirements.

The architecture of MPOS, ECR mode:

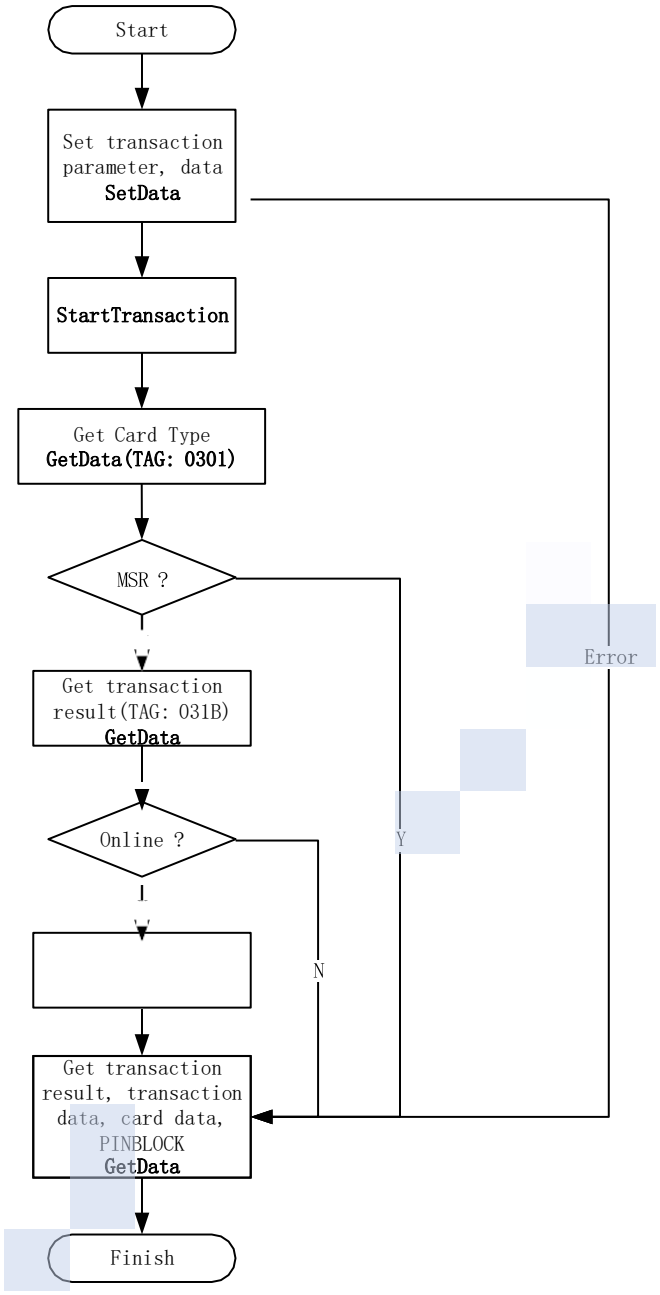


The architecture of Payment SDK mode:

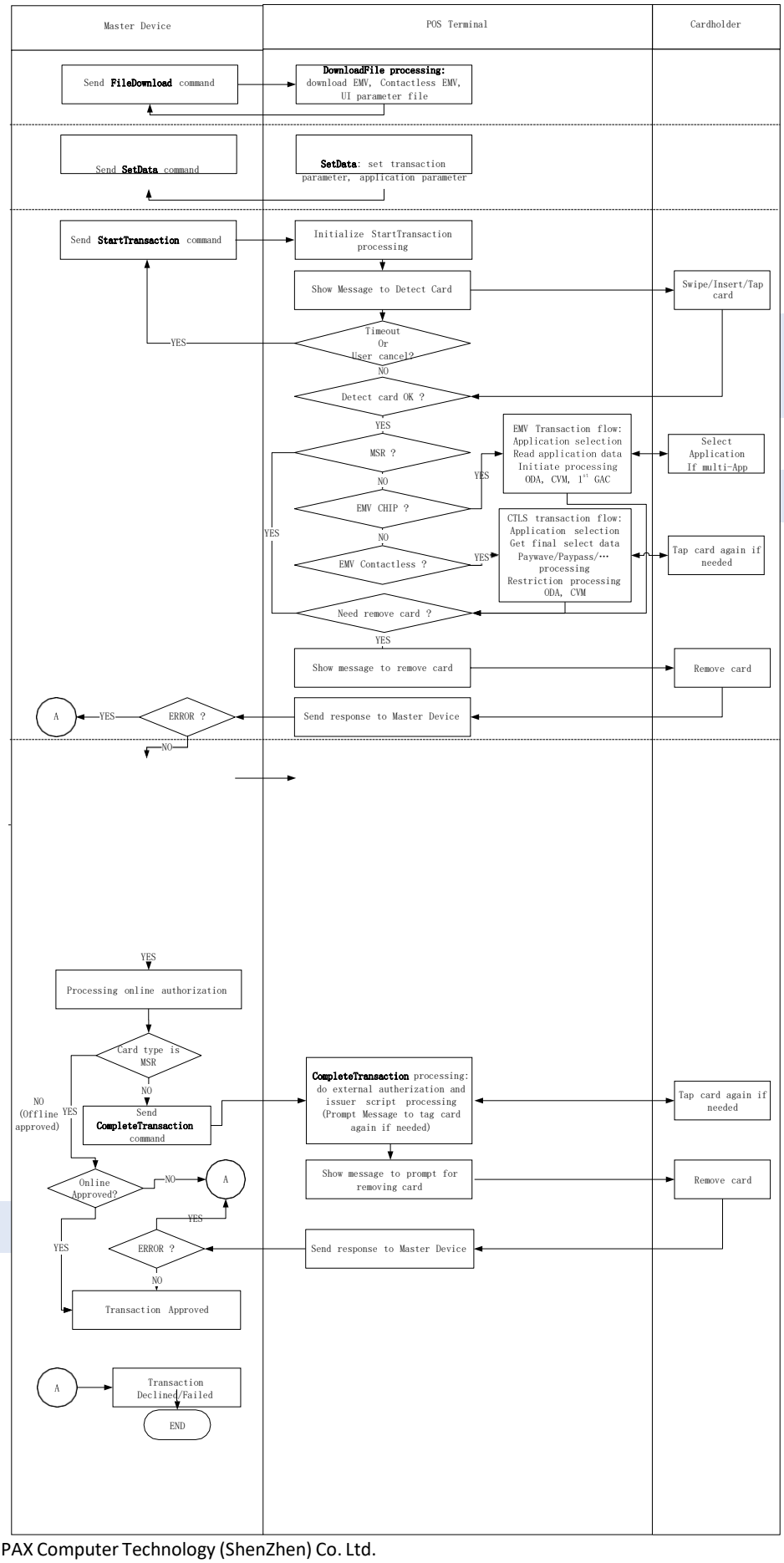


3 Example of application programming in smart phone

The diagram below simply show the transaction flow of payment application on EASYLINK Host Device:



The diagram below shows how do Master Device, POS terminal and Cardholder work together to perform a transaction (MSR, EMV contact, EMV contactless):


Master Device
POS Terminal
Cardholder
Send FileDownload command
DownloadFile processing:
download EMV, Contactless EMV,
UI parameter file
Send SetData command
SetData: set transaction
parameter, application parameter
Send StartTransaction command
Initialize StartTransaction
processing
Show Message to Detect Card
Swipe/Insert/Tap
card
Timeout
Or
User cancel?
YES
NO
Detect card OK ?
YES
MSR ?
YES
EMV Transaction flow:
Application selection
Read application data
Initiate processing
ODA, CVM, 1st GAC
Select
Application
If multi-App
NO
EMV CHIP ?
YES
NO
EMV Contactless ?
YES
CTLS transaction flow:
Application selection
Get final select data
Paywave/Paypass/…
processing
Restriction processing
ODA, CVM
Tap card again if
needed
YES
Need remove card ?
YES
Show message to remove card
Remove card
A
YES
ERROR ?
Send response to Master Device
NO
YES
Processing online authorization
Card type is
MSR
NO
NO
(Offline
approved)
YES
Send
CompleteTransaction
command
CompleteTransaction processing:
do external autherization and
issuer script processing
(Prompt Message to tag card
again if needed)
Tap card again if
needed
Online
Approved?
NO
A
Show message to prompt for
removing card
Remove card
YES
YES
ERROR ?
Send response to Master Device
NO
Transaction Approved
A
Transaction
Declined/Failed
END

4 Functions

4.1 Communication

The command <**Connect**> is used to build physical and logical connection between the PAX terminal and Android or iOS device.

The command < **ConnectWithEncryption**> is used to create connection between the POS terminal and Android/IOS/Windows device with data encryption. Note: only the communication data of writeKey, writeTIK will be encrypted by the session key created in < **ConnectWithEncryption**> command.

The command <**Disconnect**> is used to drop physical and logical connection between the PAX terminal and Android or iOS device.

Note: PAX D180 only support Bluetooth, USB.

PAX D135 only support Bluetooth, USB;

PAX SP20V4 support USB, RS232;

PAX S200 support RS232;

PAX M50, M30 support COM19;

For D200, D220, Q20 may support BT, USB, WIFI, depends on the hardware;

For A920, A930 work with EasyLink Server, support IPC;

4.2 UI

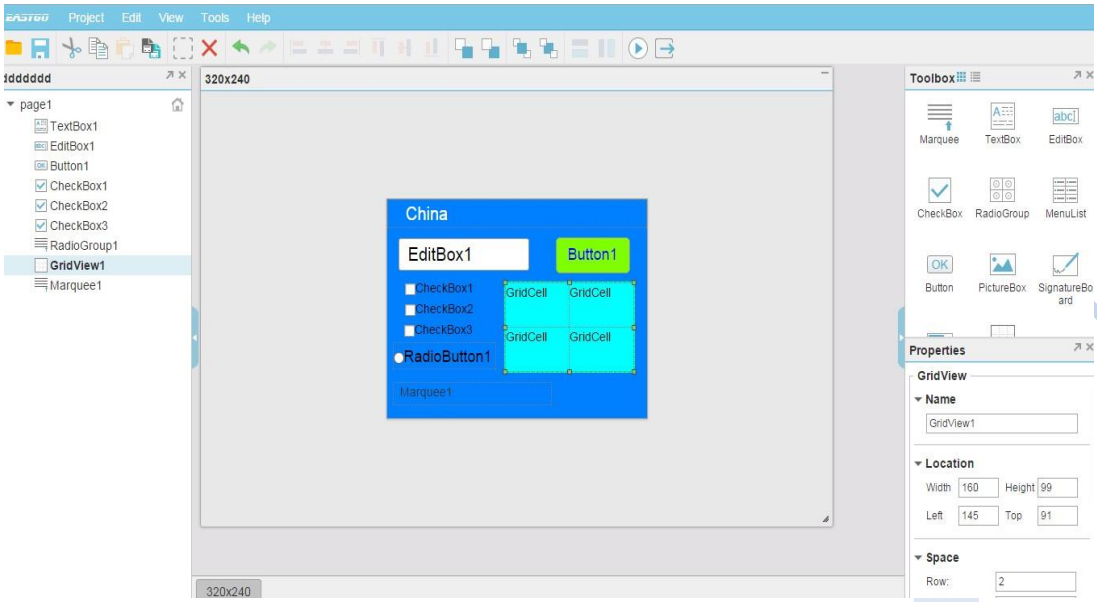
The UI module of EasyLink adopts PAX EUI design:

For users, if the change and update of the user interface is not associated with the code, UI will be more convenient to use. And UI designers needn't to care about the code process.

In order to achieve the design interface, as far as possible without modifying the code, code and UI design are independent of each other. EUI presents a visible, interface to XML script to describe, more easy to operate a UI design. Really do to modify any interface, do not move to a line of code. Nonprofessional staff after a simple training can be done on the UI to modify. In the use of the process, only need to use the interface provided by the UI library to display the page can be described in the XML script.

4.2.1 UI tool

UI tools need to be paired with the UI library to use， this can achieve the purpose of visual interface design. The edit page of the UI tool is as follows：



UI tools can same as MFC, in order to drag and drop the form of the design of the page, and can set the attributes of each control. A project has and only one XML file, each page for the XML file in a <page></page> defined by the XML script.

Tools can also be edited to simulate the page out. as follows：

When the editor is complete, the tool can generate a XML file for all the pages of the project. UI library to rely on the XML file, load the page, showing in the terminal.

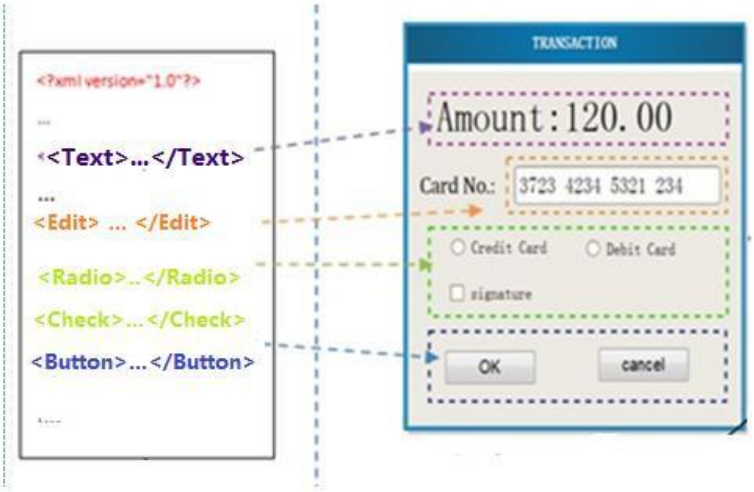
Note: This EUI Tool does not support to design UI XML for D180 currently.

4.2.2 UI XML file

With the page, usually see in the browser page is very similar, painted with text, pictures, buttons, but it is a lightweight page, the page can be displayed on the contents of the constraints in the tool making page, page description by XML script.

Considering the hardware configuration of UI machine to play the best performance and UI compatibility issues in different machines in the future, we put forward “a XML script

corresponding to a page "mentioned here refers to a page visible to the user and the related set of information processing of all consumers, but the page includes the concise content, the corresponding sample page description and control interface:



A XML corresponding to a page, a page at the same time can only present a XML script description content, a page load a new XML script, will inevitably lead to the replacement of all resources within the page. A XML corresponding to a page more want to emphasize is that the application of the page is more convenient and easy to move, to a specific page is generally only need to modify a specific XML script to do the replacement. In this way, the code and interface design can be independent of each other.

Note: PAX D180/D135 uses the lite version of EUI XML. It only contains below widgets:

<Page name="home_screen.xml" >

<TextBox name="title" x="" y="" textAlign="" value="" mode ="" fontSize=""></TextBox>

<InputBox name=" " x="" y="" textAlign="" fontSize="" mode ="" ></InputBox>

< PictureBox name ="logo" x="" y="" value =""></PictureBox>

<Menu name=" " x="" y="" textAlign="" fontSize="" ></InputBox>

</Page>

- Page

Property name	Attribute
name	Page Name

- TextBox

Property name	Attribute
name	Widget Name
x	X coordinate (row)
y	Y coordinate (column)
textAlign	Text Align 1: ALIGN_LEFT 2: ALIGN_CENTER 3: ALIGN_RIGHT

mode	Display mode: 0-normal, 1-reverse
fontSize	Font size 0-small 1-medium 2-big
value	Message to be displayed
keyAccept	Key Accept 1-BUTTON_NONE, 2-BUTTON_OK, 3-BUTTON_CANCEL, 4-BUTTON_OK_CANCEL

- InputBox

Property name	Attribute
name	Widget Name
x	X coordinate (row)
y	Y coordicate (column)
textAlign	Text Align 1: ALIGN_LEFT 2: ALIGN_CENTER 3: ALIGN_RIGHT
Mode	Mode, 0-nornal display, 1-masked with “*”
fontSize	Font size 0-small 1-medium 2-big
value	Message to be displayed
minlen	Minimum length
maxlen	Maximum length

- PictureBox

Property name	Attribute
name	Widget Name
x	X coordinate (row)
y	Y coordicate (column)
value	Picture to be displayed

- Menu

Property name	Attribute
name	Widget Name

x	X coordinate (row)
y	Y coordinate (column)
fontSize	Font size 0-small 1-medium 2-big
textAlign	Text Align 1: ALIGN_LEFT 2: ALIGN_CENTER 3: ALIGN_RIGHT

4.2.3 UI mandatory page

Below UI page are mandatory for the application in POS terminal:
"Idle.xml"
"Processing.xml"
"AppSelect.xml"
"AppSelectAgain.xml"
"PinVerifyOK.xml"
"LastEnterPin.xml"
"PinError.xml"
"Timeout.xml"
"PedErr.xml"
"EnterPIN.xml"
"InvalidCard.xml"
"NeedOnline.xml"
"TxnApproved.xml"
"TxnDeclined.xml"
"GetCard.xml"
"Fallback.xml"
"TapCardAgain.xml"
"RemoveCard.xml"
"SeePhone.xml"
"AmountConfirm.xml"
"InsertCard.xml"
"ReswipeCard.xml"
"ReadCardErr.xml"
"OpenPiccErr.xml"
"TooManyCards.xml"
"TryAnotherInter.xml"

4.2.4 ShowPage

The command <**ShowPage**> is used to show some message or picture (for PAX D180, only support .bmp format) at the specific position on the screen as the UI parameter file described. At the first time of the EasyLink Application run, the UI XML file should be downloaded into the PAX terminal with command <**FileDownload**>.

For example:

UI XML file dinition

```
<Page name="input_pin.xml">
    <TextBox name="prompt1" x="0" y="0" value="PAX" fontSize="0" textAlign="1" mode="0" keyAccept="1"></TextBox>
    <TextBox name="prompt2" x="0" y="8" value="PAX" fontSize="0" textAlign="1" mode="0" keyAccept="1"></TextBox>
    <InputBox name="input" x="0" y="16" value="PAX Input" fontSize="0" textAlign="2" mode="0" keyAccept="4" minlen="4" maxlen="8"></InputBox>
</Page>
```

Call **ShowPage** API on smart phone to show some message in POS terminal.

- **Sample code in Android:**

```
private void showMsgBoxFun() {
    short timeout = 600; // 600 * 100ms
    ArrayList<ShowMsgInfo> showMsgInfo = new ArrayList<ShowMsgInfo>();

    easyLink.showPage ("input_pin.xml", (short) 60, showMsgInfo);
}
```

Or

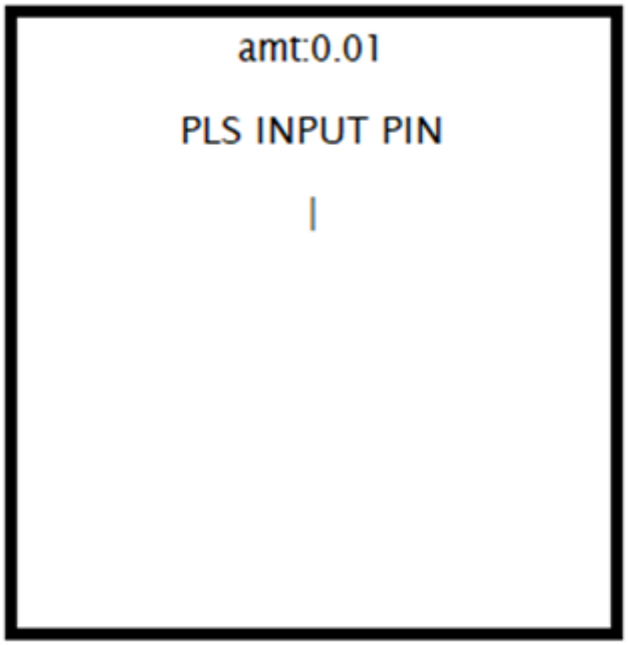
```
private void showMsgBoxFun2() {
    short timeout = 600; // 600 * 100ms
// set text of Widget "prompt1" to "amt:0.01"
    ShowMsgInfo msgInfo1 = new ShowMsgInfo("prompt1", "amt:0.01");
// set text of Widget "prompt2" to "PLS INPUT PIN"
    ShowMsgInfo msgInfo2 = new ShowMsgInfo("prompt2", "PLS INPUT PIN");
    ArrayList<ShowMsgInfo> showMsgInfo = new ArrayList<ShowMsgInfo>();
    showMsgInfo.add(msgInfo1);
    showMsgInfo.add(msgInfo2);

    easyLink.showMsgBox("input_pin.xml", (short) 60, showMsgInfo);
}
```

- **Sample code in IOS:**

```
PaxEasyLinkRetCode ret;
```

```
    ret = [[PaxEasyLinkController getInstance]showPage:pageName timeOut:timeOut
pageContentInfo:pageContentInfo];
    if(EL_RET_OK ==
        ret){ dispatch_async(dispatch_get_main_queue
        (), ^{
            [self.textInfo setText:@"showMsgBox success"];
        });
    }
    else{
        dispatch_async(dispatch_get_main_queue(), ^{
            [self.textInfo setText:[NSString stringWithFormat:@"showMsgBox
error ret= %d",ret]];
        });
    }
```

Then the terminal will show:



4.3 Security

4.3.1 Key injection (Optional)

EasyLink solution support to inject keys by Master POS and PAX RKI. User can select the key injection method by themselves.

Note: Key injection by Master POS is optional from EasyLink_Lite_v1.00.00;

4.3.2 EncryptData

PAX EasyLink supports TDES, DUKPT, RSA encryption for some sensitive data like track1/2/3 data, EMV tag 5A, EMV tag 56, EMV tag 57, and so on.

To set the encryption type and key index, user need to call <**SetData**> command to set DataEncryptionType(TAG: 0205, hex format) and DataEncryptionType(TAG: 0206, hex format) first.

- **Sample code in Android:**

```
byte data[] = {0x5F, 0x20, 0x04, 0x4A, 0x41, 0x43, 0x4B};
ByteArrayOutputStream respData = new ByteArrayOutputStream();
easyLink.encryptData(data, respData);
```

- **Sample code in IOS:**

```
PaxEasyLinkRetCode ret = 0;
    NSData *dataEncrypted = [[NSData alloc]init];
    ret = [[PaxEasyLinkController getInstance] encryptData:data
encrypyedData:&dataEncrypted];
    if(EL_RET_OK ==
        ret){ dispatch_async(dispatch_get_main_queue
        (), ^{
            [self.textInfo setText:[NSString stringWithFormat:@"Len = %ld,
value = %@", (long)dataEncrypted.length, dataEncrypted]];
        });
    }
    else{
        dispatch_async(dispatch_get_main_queue(), ^{
            [self.textInfo setText:[NSString stringWithFormat:@"ret = %d",
ret]];
        });
    }
```

4.3.3 GetPinBlock

For PIN encryption, PAX EasyLink supports DUKPT, TDES encryption to getting the PINBLOCK. To set the encryption type and key index, user need to call <**SetData**> command to set corresponding parameter.

- **Sample code in Android**

```
private void getPinBlockFun() {
   ByteArrayOutputStream pinBlock = new ByteArrayOutputStream();
   ByteArrayOutputStream ksn = new ByteArrayOutputStream();
   easyLink.getPinBlock("6225776765463826", pinBlock, ksn);
}
```

- **Sample code in IOS**

```
PaxEasyLinkRetCode ret = 0;
    NSData *dataPin = [[NSData alloc]init];
    NSData *dataKsn = [[NSData alloc]init];
    ret = [[PaxEasyLinkController getInstance] inputPin:data pinBlock:&dataPin
ksnBlock:&dataKsn];
    if(EL_RET_OK == ret){ dispatch_async(dispatch_get_main_queu(), ^{
```

```
            [self.textInfo setText:[NSString stringWithFormat:@"Len = %ld,
value = %@", (long)dataPin.length, dataPin]];
            if(nil != dataKsn){
                [self.textInfo setText:[NSString stringWithFormat:@"Len = %ld,
value = %@,ksn%@", (long)dataPin.length, dataPin,dataKsn]];
            }
        });
    }
    else{
        dispatch_async(dispatch_get_main_queue(), ^{
            [self.textInfo setText:[NSString stringWithFormat:@"ret = %d",
ret]];
        });
    }
```

4.3.4 IncreaseKsn

For DUKPT, please call increaseKsn to increase the KSN for each transaction.

- **Sample code in Android:**

```
private void testIncreaseKsn(String groupIndexStr) {
    int groupIndex = Integer.parseInt(groupIndexStr);
    int ret = easyLink.increaseKSN((byte) groupIndex);
    updateResult("IncreaseKSN ret:" + ret);
}
```

- **Sample code in IOS**

```
- (IBAction)testClicked:(id)sender {
  if(_inputField.text.length < 2) return;
  NSData *data = [self stringToBcd:_inputField.text paddingRight:NO];

  dispatch_async(dispatch_get_global_queue(DISPATCH_QUEUE_PRIORITY_DEFAULT,
0), ^{
      PaxEasyLinkRetCode ret = [easyLinkController increaseKsn:data];
     [self appendMessage:@"increaseKSN" code:ret];
  });
}
```

4.3.5 GetKeyInfo

Pax Easylink support get special key information in terminal according to the key type and key index.
Support key type as below:

```
int PED_TLK = 1;
int PED_TMK = 2;
int PED_TPK = 3;
int PED_TAK = 4;
int PED_TDK = 5;
int PED_TIK = 16;
```

Key index:

TLK: 1
TMK/TPK/TAK/TDK: 1-100
TIK: 1-40

- **Sample code in Android:**

```
private void testGetKeyInfo(String keyType) {
    if (keyType.length() != 2) {
        return;
    }

    byte[] bKeyType = StringUtils.hexString2Bytes(keyType, 1);
    int maxGroupIndex;
    if (KeyType.PED_TLK == bKeyType[0]){
        maxGroupIndex = 1;
    }else if (KeyType.PED_TIK == bKeyType[0]){
        maxGroupIndex = 3;
    }else
        maxGroupIndex = 99;

    new Thread(new Runnable() {
        @Override
        public void run() {
            StringBuffer result = new StringBuffer();
```

```
                result.append("KeyIndex   Kcv         KSN:\n");
                for(int i=1; i<(maxGroupIndex+1); i++) {
                    ByteArrayOutputStream keyInfo = new
ByteArrayOutputStream();
                    int ret = easyLink.getKeyInfo(bKeyType[0], (byte) i,
keyInfo);
                    if ((ret == ResponseCode.EL_RET_OK) && (keyInfo != null) &&
(keyInfo.size()>0)) {
                        byte[] keyInfoData = keyInfo.toByteArray();
                        result.append(" "+ i + "      " + bcd2Str(keyInfoData,
1, 4) + "  " + bcd2Str(keyInfoData, 5, 10) + "\n");
                    }
                }
                updateResult(result.toString());
            }
        }).start();

    }
```

- **Sample code in IOS**

```
-(IBAction) buttonClicked:(id)sender {
    [_output setText:@""];
    NSString *keyType = _input.text;
    if ([keyType length] != 2) {
        return;
    }
    KeyInfoParam *keyInforParam = [[KeyInfoParam alloc] init];
    unsigned iKeyType = 0;
    NSScanner *scanner = [NSScanner scannerWithString:keyType];
    [scanner scanHexInt:&iKeyType];
    keyInforParam.keyType = iKeyType;
    int maxGroupIndex = 0;
    if (keyInforParam.keyType == PED_TLK) {
        maxGroupIndex = 1;
    } else if (keyInforParam.keyType == PED_TIK){
        maxGroupIndex = 3;
    } else {
        maxGroupIndex = 99;
    }
    [_output insertText:@"KeyIndex          Kcv      Ksn\n"];
    for(int i = 1; i < (maxGroupIndex + 1); i++){
        keyInforParam.groupIndex = (Byte) i;
        NSData * keyinfo = [[NSData alloc] init];
        PaxEasyLinkRetCode ret = [_easyLinkController GetKeyInfo:keyInforParam
KeyInfo:&keyinfo];
        if (ret == EL_RET_OK  && nil!=keyinfo){
            NSString *keyIndex = [NSString stringWithFormat:@"%d",i];
            NSData *kcvData =[[NSData alloc]init];
            kcvData = [keyinfo subdataWithRange:NSMakeRange(1, 4)];
            NSString *kcv = [EasyUtils hexStringForData:kcvData];

            NSData *ksnData =[[NSData alloc]init];
            ksnData = [keyinfo subdataWithRange:NSMakeRange(5, 10)];
```

```
            NSString *ksn = [EasyUtils hexStringForData:ksnData];
            NSMutableString *info= [[NSMutableString alloc]init];
            [info appendString:@"    "];
            [info appendString:keyIndex];
            [info appendString:@"       "];
            [info appendString:kcv];
            [info appendString:@"       "];
            [info appendString:ksn];
            [info appendString:@"\n"];
            [_output insertText:[info stringByAppendingString:@"\n"]];
        }
    }
}
```

4.3.6 CalcMAC

PAX EasyLink supports TDES, DUKPT encryption to calculate the MAC data.

To set the MAC key type, MAC calculation mode and key index, user need to call <**SetData**> command to set

MAC calculation key type: tag0210
MAC calculation mode: tag0213
MAC calculation key index: tag0210

- **Sample code in Android:**

```
private void testCalcMac(String data) {
    int length = data.length() / 2;
    if (length == 0) {
        return;
    }

    byte[] databy = StringUtils.hexString2Bytes(data, length);

    ByteArrayOutputStream mac = new ByteArrayOutputStream();
    int ret = easyLink.calcMAC(databy, mac);
    updateResult("calcMac ret:" + ret + "\ncalcMac:" +
bcd2Str(mac.toByteArray()));
}
```

- **Sample code in IOS**

```
- (IBAction)caclCustomBytesClicked:(id)sender {
  NSData *data = [self stringToBcd:textView.text paddingRight:NO];
  dispatch_async(dispatch_get_global_queue(DISPATCH_QUEUE_PRIORITY_DEFAULT,
0), ^{
    NSData *output = [[NSData alloc] init];
    PaxEasyLinkRetCode ret = [easyLinkController CalcMAC:data MAC:&output];
    [self appendMessage:[self bcdToString:output] code:ret];
  });
}
```

4.4 Parameter and data management

4.4.1 SetData

The EasyLink provides <**SetData**> command to set parameters or transaction data into the PAX terminal.

All the parameters and data are set in TLV format: TAG LENGTH VALUE format.

Parameters and data can be set into the terminal:

Appendix 2 – Application parameter list, Appendix 3 – Transaction parameter list, contact EMV tag, contactless EMV tag;

- **Sample code in Android**

```
private  void  setDataFun()
   {byte data[] =
{(byte)0x9c, 0x01, 0x01, 0x5f, 0x2a, 0x02, 0x01, 0x24, 0x5f, 0x36, 0x01, 0x02, (byte)0x9a, 0
x03, 0x16, 0x10, 0x02, (byte)0x9f, 0x21, 0x03, 0x05, 0x50, 0x59, (byte)0x9f, 0x03, 0x06, 0x0
0, 0x00, 0x00, 0x00, 0x00, 0x00, (byte)0x9f, 0x02, 0x06, 0x00, 0x00, 0x00, 0x00, 0x10, 0x00};
   ByteArrayOutputStream  failedTags = new ByteArrayOutputStream();
   easyLink.setData(DataType.TRANSACTION_DATA, data, failedTags);
}

private void setDataFUN2() {
      ByteArrayOutputStream  failedTags = new ByteArrayOutputStream();
      byte data[] =       {0x02,0x02,0x01,0x01}
      int ret = easyLink.setData(DataType.CONFIGURATION_DATA, data,
failedTags);
}
```

- **Sample code in IOS**

```
PaxEasyLinkRetCode ret;
NSData *dataList = [[NSData alloc]init];
ret = [[PaxEasyLinkController getInstance] setData:dataType dataList:data
tagList:&dataList];
    if(EL_RET_OK == ret){

        dispatch_async(dispatch_get_main_queue(), ^{
            [self.textInfo setText:[NSString stringWithFormat:@"Len = %ld,
value = %@", (long)dataList.length, dataList]];
                    });
    }
    else{
        dispatch_async(dispatch_get_main_queue(),
            ^{if(dataList != nil){
                [self.textInfo setText:[NSString stringWithFormat:@"Len = %ld,
value = %@ ret %d", (long)dataList.length, dataList,ret]];
            }
            else{
                [self.textInfo setText:[NSString
stringWithFormat:@"ret %d",ret]];
            }
```

```
        });
    }
```

4.4.2 GetData

Easylink provides <**GetData**> command to get parameters or transaction data from PAX terminal.
All the parameters and data are get in TLV format: TAG LENGTH VALUE format.
Parameters and data can be got from the terminal:

Appendix 1 – Terminal information list, Appendix 2 – Application parameter list, Appendix 3 – Transaction parameter list, contact EMV tag, contactless EMV tag.

Note: the Appendix 3 – Transaction parameter list is collection of customer tags, like track1, 2 and 3 of magnetic card are defined specific tags. So user can get the value with the corresponding tag.

- **Sample code in Android:**

```
private void  getDataFun1() {
    byte data[] = {(byte)0x9F,0x27};
    easyLink.getData(1, data, new myGetDataListener());
}

private void getDataFun2()
    { byte data[] = {0x02,0x01};
    easyLink.getData(2, data, new myGetDataListener());
}
private void getDataFUN1() {
    byte data[] = {0x9c,0x01};
    ByteArrayOutputStream dataList = new ByteArrayOutputStream();
    int ret = easyLink.getData(DataType.TRANSACTION_DATA, data, dataList);
}

private void getDataFUN2() {
    byte data[] = {0x02,0x01};
    ByteArrayOutputStream dataList = new ByteArrayOutputStream();
    int ret = easyLink.getData(DataType.CONFIGURATION_DATA, data, dataList);
}
```

- **Sample code in IOS:**

```
PaxEasyLinkRetCode ret = 0;
    NSData *dataOut = [[NSData alloc]init];
    NSData* dataTag = [[NSData alloc]initWithData:data];
    ret = [[PaxEasyLinkController getInstance] getData:dataType tagList:dataTag
dataList:&dataOut];
    if(0 == ret){
        sucData = [[NSData alloc]initWithData:dataOut];
        dispatch_async(dispatch_get_main_queue(), ^{
            if(dataOut != nil){
```

```
                [self.textInfo setText:[NSString stringWithFormat:@"Len = %ld,
value = %@ ret %d", (long)dataOut.length, dataOut,ret]];
            }
            else{
                [self.textInfo setText:[NSString
stringWithFormat:@"ret %d",ret]];
            }
        });
                }
    else{
         sucData = [[NSData alloc]initWithData:dataOut];
        dispatch_async(dispatch_get_main_queue(), ^{
            if(dataOut != nil){
            [self.textInfo setText:[NSString stringWithFormat:@"Len = %ld,
value = %@ ret %d", (long)dataOut.length, dataOut,ret]];
            }
            else{
                [self.textInfo setText:[NSString
stringWithFormat:@"ret %d",ret]];
            }
        });
    }
```

4.4.3 ReplaceData

If payment terminal need support local and bank host with 2 different encryption method to encrypt sensitive data on the same terminal. For example, the payment terminal keep encrypted sensitive data with AES method in local data base, and the sensitive data which will be sent to bank host those need to be encrypted by DUKPT Des.

At that situation, the EasyLink provides < replaceData > command to support replace local encrypted data by host encrypted data.

Prerequisite:

Before calling this function, User should use <setData> command to set parameter tag0205, tag 0206 to encrypt the local sensitive data, use <getData> to get encrypted data and save into local database.

To set the replacement encryption type tag0237 and replacement encryption key index tag0238, user need to call <**setData**> command to setcorresponding parameter.

- **Sample code in Android:**

```
public void testReplaceData(DataType type, String data){
    new Thread(new Runnable() {
        @Override
        public void run() {
            int length = data.length() / 2;
            if (length == 0) {
                return;
            }
```

```
                byte[] databy = StringUtils.hexString2Bytes(data, length);
                ByteArrayOutputStream dataList = new ByteArrayOutputStream();
                int ret = easyLink.getData(type, databy, dataList);
                Log.d("testReplaceData", " getData ret : " + ret);
                if (ret == 0) {
                    byte[] data = {
                            0x02, 0x37, 0x01, 0x04,
                            0x02, 0x38, 0x01, 0x02};    //dukpt key index 02
                    ByteArrayOutputStream failedTags = new
ByteArrayOutputStream();
                    easyLink.setData(DataType.CONFIGURATION_DATA, data,
failedTags);

                    ByteArrayOutputStream reDataList = new
ByteArrayOutputStream();
                    byte[] buffer = new byte[dataList.size()-2];
                    System.arraycopy(dataList.toByteArray(), 2, buffer, 0,
dataList.size()-2);
                    ret = easyLink.replaceData(type, buffer, reDataList);
                    updateResult("replaceData:" + ret + "\ndataList:" +
bcd2Str(reDataList.toByteArray()));
                }else{
                    updateResult("getData:" + ret + "\ndataList:" +
bcd2Str(dataList.toByteArray()));
                }
            }
        }).start();
    }
```

- **Sample code in IOS**

```
- (void)replaceData:(NSInteger)dataType data:(NSData *)data
{
    //02370104  02380102
    [self.textInfo setText:@""];

    dispatch_async(dispatch_get_global_queue(0, 0),^{
        NSData *getDataRes = [[NSData alloc] init];
        PaxEasyLinkRetCode getRet = [self->_easyController getData:dataType
tagList:data dataList:&getDataRes];
        if (getRet == EL_RET_OK) {
            NSData *output = [[NSData alloc] init];
            NSData *buffer = [getDataRes subdataWithRange:NSMakeRange(2,
[getDataRes length] - 2)];;
            PaxEasyLinkRetCode ret = [self->_easyController
replaceData:dataType tlvData:buffer resultData:&output];
            if (ret == EL_RET_OK) {
                [self appendMessage:[NSString stringWithFormat:@"replaceData
result code: %@", [[PaxEasyLinkStatusCode sharedInstance]
statusCode2String:ret]]];
                [self appendMessage:[NSString stringWithFormat:@"replaceData
result data: %@", output]];
            } else {
                [self appendMessage:@"replaceData error:"
code:[[PaxEasyLinkStatusCode sharedInstance] statusCode2String:ret]];
            }
```

```
        } else {
            [self appendMessage:@"getData error:" code:[[PaxEasyLinkStatusCode
sharedInstance] statusCode2String:getRet]];
        }
    });
}
```

4.4.4 GetCardBinIndex

Easylink provides <**GetCardBinIndex**> command to get the index of current card bin in card bin range list.

Card bin range content should be:

panLow: 6 bytes (BCD)
panHigh: 6 bytes (BCD)
panMinLength: 1 byte (BCD)
panMaxLength: 1 byte (BCD)
szCountryCode: 2 bytes (BCD)
cardType: 1 byte (BCD)
productid: 3 bytes (BCD)
options: 2 bytes (BCD)
total count not more than 186.

Prerequisite:

Before calling this function, User should use <startTransaction> command to finished read a card, and before call <clearTransaction> command.

- **Sample code in Android:**

```
private void getCardBinIndex() {
    new Thread(new Runnable() {
        @Override
        public void run() {
            byte[] binRange = new byte[44];

System.arraycopy(Tools.str2Bcd("4934200000004934200009991616084001462020
500049342000100049342192999916160840322020201000"), 0, binRange, 0,
binRange.length);
            ByteArrayOutputStream idx = new ByteArrayOutputStream();
            int ret = easyLink.getCardBinIndex(binRange, idx);
            updateResult("getCardBinIndex :" + ret + "\n idx:" +
bcd2Str(idx.toByteArray()));
        }
    }).start();
}
```

- Sample code in IOS
  Easylink doesn't support this command in IOS version.

4.4.5 CheckCardLuhn

Easylink provides <**CheckCardLuhn**> command to verify the Luhn of current pan.

Prerequisite:

Before calling this function, User should use <startTransaction> command to finished read a card, and before call <clearTransaction> command.

➢ **Sample code in Android:**

```
private void checkCardLuhn(){
    new Thread(new Runnable() {
        @Override
        public void run() {
            int ret = easyLink.checkCardLuhn();
            updateResult("checkCardLuhn :" + ret);
        }
    }).start();
}
```

➢ Sample code in IOS
Easylink doesn't support this command in IOS version.

4.5 Transaction processing

EasyLink provides two commands: <**StartTransaction**>, <**CompleteTransaction**> to handle MSR, contact EMV, contactless EMV processing.

4.5.1 StartTransaction

<StartTransaction> is used to detect card (including magnetic card, EMV chip card, contactless EMV card).

Note: before invoke StartTransaction API, please invoke setData API to set transaction data in Appendix 4 – Data to be set before StartTransaction

- If magnetic card is swiped, then PAX terminal will only read the track1, track2, track3 from magnetic card, if read success, then user can call <GetData> command to get track1, track2, track3. After read track data, the terminal will prompt to request PIN.
- If contact EMV chip card is inserted, then PAX terminal will process according to EMV specification. If processing return success, then user can call <GetData> command to get transaction result (TAG: 9F 27). If transaction need to go online, then after online processing, user need to call <CompleteTransaction> to complete EMV transaction.
- If contactless EMV chip card is tapped, then PAX terminal will processing according to the corresponding contactless EMV specification (Paypass, Paywave, AMEX, and so on). If processing return success, then user can call <GetData> command to get transaction result (TAG: 9F 27). If transaction need to go online, then after online processing, user need to call <CompleteTransaction> to complete contactless EMV transaction.

Note: for track 2 data format, please refer to appendix Appendix 8 – Track 2 data format

➢ **Sample code in Android:**

```
private void startTransactionFun() {
    int ret = easyLink.startTransaction();
}
```

➢ **Sample code in IOS:**

```
PaxEasyLinkRetCode ret = 0;
    ret = [[PaxEasyLinkController getInstance] startTransaction];
```

```
    if(EL_RET_OK == ret){
        dispatch_async(dispatch_get_main_queue(),
            ^{ [self.textInfo setText:@"startTransaction
            success"];
        });
    }
    else{
        dispatch_async(dispatch_get_main_queue(), ^{
           [self.textInfo setText:[NSString stringWithFormat:@"startTransaction
error ret= %d",ret]];        });
    }
```

4.5.2 CompleteTransaction

After online processing, user need to call <**SetData**> to set the host authorization response code and script if returned into the PAX terminal, then call <**CompleteTransaction**> to finish the transaction.

Note: before invoke completeTransaction API, please invoke setData API to set host response data to the POS terminal in Appendix 5 – Data to be set before CompleteTransaction

- **Sample code in Android:**

```
private void completeTransactionFun() {
   int ret = easyLink.completeTransation();
}
```

- **Sample code in IOS:**

```
PaxEasyLinkRetCode ret;
    ret = [[PaxEasyLinkController getInstance] completeTransaction];
    if(EL_RET_OK  == ret){
        dispatch_async(dispatch_get_main_queue(),
            ^{ [self.textInfo setText:@"completeTransaction
            success"];
        });
    }
    else{
        dispatch_async(dispatch_get_main_queue(),
             ^{[self.textInfo setText:[NSString
stringWithFormat:@"completeTransaction error ret= %d",ret]];
        });
    }
```

4.5.3 ClearTransData

After finish transaction, user need to call <**clearTransData**> to purge all transactions data.

- **Sample code in Android:**

```
private void clearTransData(){
    new Thread(new Runnable() {
        @Override
        public void run() {
            int ret = easyLink.clearTransData();
            updateResult("clearTransData:" + ret);
        }
    }).start();
}
```

- **Sample code in IOS:**

```
PaxEasyLinkRetCode  ret;
ret = [_easyLinkController clearTransaction];
NSString *result = [[NSString alloc] init];
if (ret == EL_RET_OK) {
    result = @"success";
} else {
    result = [@(ret).stringValue stringByAppendingString:@"\n"];
}
dispatch_async(dispatch_get_main_queue(), ^{
    UIAlertView *view = [[UIAlertView alloc] initWithTitle:currTitle message:result
delegate:nil cancelButtonTitle:@"cancel" otherButtonTitles:@"OK", nil];
    [view show];
});
```

4.5.4 Security Instruction

If user develop their application with P2PE feature, at end of transaction, user must call <**clearTransData**> to purge all transactions data, otherwise reader will return P2PE data invalided to application.

4.6 Reader module

Pax Easylink provide a group interface for payment terminal to control each reader module, include contactless/contact/magnetic card reader, and control reader open/close/detect actions.

4.6.1 Contactless card reader module

4.6.1.1 piccOpen

Power on the contactless card reader module and prepare for detect card.

- **Sample code in Android:**

```
private void testPiccOpen() {
    new Thread(new Runnable() {
        @Override
        public void run() {
            int ret = easyLink.piccOpen();
            updateResult("iccOpen ret:" + ret);
        }
```

```
        }).start();
    }
```

- **Sample code in IOS:**

```
PaxEasyLinkRetCode ret = [[PaxEasyLinkStatusCode alloc] init];
ret = [_easyLinkController clearTransaction];
NSString *result = [[NSString alloc] init];
if (ret == EL_RET_OK) {
    result = @"success";
} else {
    result = [@(ret).stringValue stringByAppendingString:@"\n"];
}
```

- *piccDetect*

Check the contactless card in the detection area.

If terminal need to detect card continues and a contactless card has already in the detection area, then it needs call piccRemove before call piccDetect, otherwise, piccDetect will return an error result code.

Support detect card's mode:

```
ISO14443_AB((byte)0),
EMV_AB((byte)1),
ONLY_A((byte)65),
ONLY_B((byte)66),
ONLY_M((byte)77),
ONLY_C;
```

- **Sample code in Android:**
  -
```
private void testPiccDetect() {
    new Thread(new Runnable() {
        @Override
        public void run() {
            PiccCardInfo outPiccCardInfo = new PiccCardInfo();
            int ret = easyLink.piccDetect(ISO14443_AB, outPiccCardInfo);
            Log.d(TAG, "piccDetect getSerialInfo:" +
Utils.bcd2Str(outPiccCardInfo.getSerialInfo()));
            Log.d(TAG, "piccDetect getCardType:" +
outPiccCardInfo.getCardType());
            Log.d(TAG, "piccDetect getOther:" +
Utils.bcd2Str(outPiccCardInfo.getOther()));
            updateResult("piccDetect ret:" + ret);
        }
    }).start();
}
```

- **Sample code in IOS:**

```
PaxEasyLinkRetCode ret = [[PaxEasyLinkStatusCode alloc] init];
PiccCardInfo *picc = [[PiccCardInfo alloc] init];
ret = [_easyLinkController piccDetect:EMV_AB piccCardInfo:picc];
cid = picc.cid;
NSString *result = [[NSString alloc] init];
if (ret == EL_RET_OK) {
    result = @"success";
} else {
    result = [@(ret).stringValue stringByAppendingString:@"\n"];
}
```

- *piccRemove*

  Remove the detected contactless card information from reader.

➢ **Sample code in Android:**

```
private void testPiccRemove() {
    new Thread(new Runnable() {
        @Override
        public void run() {
            int ret =
easyLink.piccRemove(EPiccRemoveMode.REMOVE,(byte)0);
            updateResult("piccRemove ret:" + ret);
        }
    }).start();
}
```

➢ **Sample code in IOS:**

```
PaxEasyLinkRetCode ret = [[PaxEasyLinkStatusCode alloc] init];
ret = [_easyLinkController piccRemove:EMV cid:cid];
NSString *result = [[NSString alloc] init];
if (ret == EL_RET_OK) {
    result = @"success";
} else {
    result = [@(ret).stringValue stringByAppendingString:@"\n"];
}
```

- *piccClose*

  Power off for the contactless card reader module

➢ **Sample code in Android:**

```
private void testPiccClose() {
    new Thread(new Runnable() {
        @Override
        public void run() {
            int ret = easyLink.piccClose();
            updateResult("iccClose ret:" + ret);
        }
    }).start();
}
```

➢ **Sample code in IOS:**

```
PaxEasyLinkRetCode ret = [[PaxEasyLinkStatusCode alloc] init];
ret = [_easyLinkController piccClose];
NSString *result = [[NSString alloc] init];
if (ret == EL_RET_OK) {
    result = @"success";
} else {
    result = [@(ret).stringValue stringByAppendingString:@"\n"];
}
```

4.6.2 Contact card reader module

- *iccOpen*

Power on the contact card reader module and prepare for detect card.

➢ **Sample code in Android:**

```
private void testIccOpen() {
    new Thread(new Runnable() {
        @Override
        public void run() {
            int ret = easyLink.iccOpen((byte) 0);
            updateResult("iccOpen ret:" + ret);
        }
    }).start();
}
```

➢ **Sample code in IOS:**

```
PaxEasyLinkRetCode ret = [[PaxEasyLinkStatusCode alloc] init];
ret = [_easyLinkController iccOpen:0x00];
NSString *result = [[NSString alloc] init];
if (ret == EL_RET_OK) {
    result = @"success";
} else {
    result = [@(ret).stringValue stringByAppendingString:@"\n"];
}
```

- *iccDetect*

Check whether a card be inserted into the contact card slot.

➢ **Sample code in Android:**

```
private void testIccDetect() {
    new Thread(new Runnable() {
        @Override
        public void run() {
            int ret = easyLink.iccDetect((byte) 0);
            updateResult("iccDetect ret:" + ret);
        }
    }).start();
}
```

➢ **Sample code in IOS:**

```
PaxEasyLinkRetCode ret = [[PaxEasyLinkStatusCode alloc] init];
ret = [_easyLinkController iccDetect:0x00];
NSString *result = [[NSString alloc] init];
if (ret == EL_RET_OK) {
    result = @"success";
} else {
    result = [@(ret).stringValue stringByAppendingString:@"\n"];
}
```

- *iccInit*

Power on to the inserted card, and read ATR data from the chip card

➢

➢ **Sample code in Android:**

```
private void testIccInit() {
    new Thread(new Runnable() {
        @Override
        public void run() {
            byte[] atr = new byte[34];
            int ret = easyLink.iccInit((byte) 0x00, (byte) 0, (byte) 0,
(byte) 0, (byte) 1, atr);
            updateResult("iccDetect ret:" + ret + "\n Utils.bcd2Str(atr)
= " + Utils.bcd2Str(atr));
        }
    }).start();
}
```

➢ **Sample code in IOS:**

```
PaxEasyLinkRetCode ret = [[PaxEasyLinkStatusCode alloc] init];
NSData *atr = [[NSData alloc] init];
ret = [_easyLinkController iccInit:0x00 cardVoltage:0 supportPPS:0 rate:0
standard:0 atr:&atr];
NSString *result = [[NSString alloc] init];
if (ret == EL_RET_OK) {
    result = @"success";
} else {
    result = [@(ret).stringValue stringByAppendingString:@"\n"];
}
```

- *iccClose*

Power off for the contact card reader module.

➢ **Sample code in Android:**

```
private void testIccClose() {
    new Thread(new Runnable() {
        @Override
        public void run() {
            int ret = easyLink.iccClose((byte) 0);
            updateResult("iccClose ret:" + ret);
        }
    }).start();
}
```

➢ **Sample code in IOS:**

```
PaxEasyLinkRetCode ret = [[PaxEasyLinkStatusCode alloc] init];
ret = [_easyLinkController iccClose:0x00];
NSString *result = [[NSString alloc] init];
if (ret == EL_RET_OK) {
    result = @"success";
} else {
    result = [@(ret).stringValue stringByAppendingString:@"\n"];
}
```

4.6.3 Magnetic card module

- *msrOpen*

Power on the magnetic card reader module and prepare for detect card.

➢ **Sample code in Android:**

```
private void testMagOpen() {
    new Thread(new Runnable() {
        @Override
        public void run() {
            int ret = easyLink.msrOpen();
            updateResult("msrOpen ret:" + ret);
        }
    }).start();
}
```

Sample code in IOS:

```
PaxEasyLinkRetCode ret = [[PaxEasyLinkStatusCode alloc] init];
ret = [_easyLinkController msrOpen];
NSString *result = [[NSString alloc] init];
if (ret == EL_RET_OK) {
    result = @"success";
} else {
    result = [@(ret).stringValue stringByAppendingString:@"\n"];
}
```

- *msrSwiped*

Detects whether a magnetic card is swiped from the card slot.

➢ **Sample code in Android:**

```
private void testMagSwiped() {
    new Thread(new Runnable() {
        @Override
        public void run() {
            int ret = easyLink.msrSwiped();
            updateResult("testMagSwiped ret:" + ret);
        }
    }).start();
}
```

Sample code in IOS:

```
PaxEasyLinkRetCode ret = [[PaxEasyLinkStatusCode alloc] init];
ret = [_easyLinkController msrSwiped];
NSString *result = [[NSString alloc] init];
if (ret == EL_RET_OK) {
    result = @"success";
} else {
    result = [@(ret).stringValue stringByAppendingString:@"\n"];
}
```

- *msrReset*

Clear data catch in the magnetic card reader.

➢ **Sample code in Android:**

```
private void testMagRest() {
    new Thread(new Runnable() {
        @Override
        public void run() {
            int ret = easyLink.msrReset();
```

```
                updateResult("msrReset ret:" + ret);
            }
        }).start();
    }
```

Sample code in IOS:

```
PaxEasyLinkRetCode ret = [[PaxEasyLinkStatusCode alloc] init];
ret = [_easyLinkController msrReset];
NSString *result = [[NSString alloc] init];
if (ret == EL_RET_OK) {
    result = @"success";
} else {
    result = [@(ret).stringValue stringByAppendingString:@"\n"];
}
```

- *msrClose*

  Power off for the magnetic card reader module.

➢ **Sample code in Android:**

```
private void testMagClose() {
    new Thread(new Runnable() {
        @Override
        public void run() {
            int ret = easyLink.msrClose();
            updateResult("msrClose ret:" + ret);
        }
    }).start();
}
```

Sample code in IOS:

```
PaxEasyLinkRetCode ret = [[PaxEasyLinkStatusCode alloc] init];
ret = [_easyLinkController msrClose];
NSString *result = [[NSString alloc] init];
if (ret == EL_RET_OK) {
    result = @"success";
} else {
    result = [@(ret).stringValue stringByAppendingString:@"\n"];
}
```

4.7 Dynamic update transaction parameter for multi-host

To support multi-host, such as:

- ➢ Update or set different PIN key type (TAG 0202) and PIN key index (TAG 0203) according to card brand, PAN, or transaction type, etc.
- ➢ Update or set different floor limit, or TAC denial/online/default, etc, for selected AID according to the transaction type, PAN, etc.
- ➢ Update or set different ICS parameter (see ICS configuration in emv_param.emv), like terminal capability, pinByPass, forceOnline, etc, according to the transaction type, PAN, etc.
- ➢ To write working key to the PED in EasyLink Slave Side for the corresponding host according to the transaction type, PAN, etc.

To support dynamic update transaction parameter for multi-host, for example to update the ICS and PinKeyType (TAG 0202), PinKeyIndex (TAG 0203) during startTransaction for current transaction, the Payment APP in master device need to:

1) Update the EasyLink Android SDK to the version v2.01.00 or above;
2) Invoke setData API to set value of TAG 0216 to 1;
3) Update the callBackCfg.report file (the callBackCfg.report file is in released package),

```
<Root version="000001" Name="CallBackConfig">
    <Item   CallBackName="SetParamByClient">false</Item>
    <Item   CallBackName="SetEmvAidParam">false</Item>
    <Item   CallBackName="SetICSParam">true</Item>
    <Item   CallBackName="ReportReadCardOkStatus"> true</Item>
</Root>
```

4) Invoke fileDownload API to download the updated callBackCfg.report file into EasyLink APP;
5) Override the onRunCmd function according to the business requirement;(Please see the Sample Code Below)

Note:

- Update the Selected AID parameter, ICS parameter for current transaction, not for every transaction, and the emv_param.emv parameter file will not be updated
- EasyLink Lite: not support;
- EasyLink Full: support from v2.01.00_20190612
- EasyLink Server: not support;
- EasyLink Android SDK: support from v2.01.00_20190612
- EasyLink IOS SDK: not support

Sample code:

```
@Override
public ArrayList<BaseRunCmdModel> onRunCmd(ArrayList<? extends BaseReportedData> reportedParamList, int timeout)
    {ArrayList<BaseRunCmdModel> rspList = new ArrayList<>();
    for (BaseReportedData baseReportedParam : reportedParamList) {
        if (baseReportedParam.getStatus().equals(ReportConstant.STATUS_SET_EMV_AID_PARAM))
            { ReportedData<EmvAidParam> emvAidParamReportedParam = (ReportedData<EmvAidParam>)
    baseReportedParam;

            // TODO: Update EmvAidParam OBJECT according to the business requirement
            // For example, update the floorLimit to 0 if transaction type is 0x20
            // 1. Get EmvAidParam OBJECT
```

```
            EmvAidParam emvAidParamOBJ = emvAidParamReportedParam.getParamBean();

            // 2. update the floorLimit to 0 according to the business requirement
            emvAidParamOBJ.setFloorLimit(0);

           // 3. Set the updated EmvAidParam OBJECT to set SetEmvAidParamModel
           SetEmvAidParamModel emvAidParamModel = new SetEmvAidParamModel();
           emvAidParamModel.setEmvAidParam(emvAidParamOBJ);

            // 4. Apply the changes to the response result
            rspList.add(emvAidParamModel);

}
else if (baseReportedParam.getStatus().equals(ReportConstant.STATUS_SET_ICS_PARAM))
        { ReportedData<ICSParam> icsParamReportedParam = (ReportedData<ICSParam>) baseReportedParam;

            // TODO: Update ICSParam OBJECT according to the business requirement
            // For example, update the pinByPass to 0 and forcedOnline to 1 if transaction type is 0x20
            // 1. Get ICSParam OBJECT
            ICSParam iCSParamOBJ = icsParamReportedParam.getParamBean();

            // 2. update the pinByPass to 0 and forcedOnline to 1 according to the business requirement.
            iCSParamOBJ.setBypassPinEntry(0);
            iCSParamOBJ.setForcedOnlineCapability(1);

            // 3. Set the updated ICSParam OBJECT to the SetICSParamModel
            SetICSParamModel icsParamModel = new SetICSParamModel();
            icsParamModel.setICSParam(iCSParamOBJ);

            // 4. Apply the changes to the response result
            rspList.add(icsParamModel);

 } else if (baseReportedParam.getStatus().equals(ReportConstant.STATUS_READ_CARD_OK))
        { BaseReportedData readCardOKReportedData = baseReportedParam;

            // TODO: update TAG by setData COMMAND according to the business requirement
            // TODO: write key for specific host according to the business requirement if writeKey/writeTIK supported
            // Eg. set pinKeyType(TAG 0202) and pinKeyIndex(TAG 0203) according to PAN range and transaction type

            // 1. Update TAG by setData COMMAND according to the business requirement
            // set PinKeyType (TAG 0202)
            TLVDataObject pinKeyTypeTlv = new TLVDataObject(new byte[]{0x02,0x02}, new byte[]{0x01});
            // set PinKeyInidex (TAG 0203)
            TLVDataObject pinKeyIndexTlv = new TLVDataObject(new byte[]{0x02,0x03}, new byte[]{0x09});

             ArrayList<TLVDataObject> setDataTlvList = new ArrayList<>();
            setDataTlvList.add(pinKeyTypeTlv);
            setDataTlvList.add(pinKeyIndexTlv);

            // 2. Add the list of TLVDataObject to SetDataModel
            SetDataModel setDataModel = new SetDataModel();
            setDataModel.setConfigTlv(setDataTlvList);

            // 3. Apply the changes to the response result
            rspList.add(setDataModel);
```

```
                // optional if writeKey/writeTIK supported
                // 1. writeKey/writeTIK for corresponding host according to the business requirement if writeKey/writeTIK
        supported
                //       For example, write tpk
                byte[] tpkValue = new byte[]{0x33, 0x33, 0x33, 0x33, 0x33, 0x33, 0x33, 0x33, 0x33, 0x33, 0x33, 0x33, 0x33, 0x33,
        0x33, 0x33};
                KeyParam tpkParam = new KeyParam();
                tpkParam.setSrcKeyType(KeyType.PED_TMK);
                tpkParam.setSrcKeyIndex(0x30);
                tpkParam.setDstKeyType(KeyType.PED_TPK);
                tpkParam.setDstKeyIndex(0x09);
                tpkParam.setDstKeyValue(tpkValue);
                tpkParam.setCheckMode(KcvMode.KCV_NONE);

                // 2. Add the KeyParam OBJECT to WriteKeyModel
                WriteKeyModel writeKeyModel = new WriteKeyModel();
                writeKeyModel.setKeyParam(tpkParam);

                // 3. Apply the changes to the response result
                rspList.add(writeKeyModel);

                // optional if writeKey/writeTIK supported
                // 1. writeTIK for corresponding host according to the business requirement if writeTIK supported
                // For example, write TIK
                // byte[] ipek = Convert.strToBcd("6AC292FAA1315B4D858AB3A3D7D5933A",
        Convert.EPaddingPosition.PADDING_RIGHT);
                // byte[] ksn = Convert.strToBcd("FFFF9876543210E00000", Convert.EPaddingPosition.PADDING_RIGHT);
                // TikKeyParam tikParam = new TikKeyParam();
                // tikParam.setSrcKeyIndex(0x30);
                // tikParam.setGroupIndex(5);
                // tikParam.setKeyValue(ipek);
                // tikParam.setKsn(ksn);
                // tikParam.setCheckMode(KcvMode.KCV_NONE);

                // 2. Add the TikKeyParam OBJECT to WriteTikKeyModel
                WriteTikKeyModel writeTikKeyModel = new WriteTikKeyModel();
                writeTikKeyModel.setTikKeyParam(tikParam);

                // 3. Apply the changes to the response result
                rspList.add(writeTikKeyModel);
            }
        }
        return rspList;
    }
```

4.8 Handle the notification/report message from EasyLink

The Easylink support to send notification/report message to the Master device, the notification or report message are all the prompt message or processing status during startTransaction/completeTransaction/getPinBlock API, eg:

- Search card mode notifications (during startTransaction/completeTransaction):
  - Please Insert/Swipe/Tap Card;
  - Fallback, Please Swipe Card;

- Please Use Contact;
- Tap Card Again;
- Chip Card, Please Insert;
- etc. (details: please see the onReportSearchMode listener in EasyLink Android SDK);

- Read Card Status notifications (during startTransaction/completeTransaction):
  - Ready to read;
  - Processing;
  - Read card successfully;
  - Please remove card;
  - etc. (details: please see the onReadCard listener in EasyLink Android SDK)
- Enter PIN notifications (during startTransaction/getPinBlock):
  - Please Enter PIN; (initialize the UI)
  - PIN error, Retry; (for offline PIN)
  - Last enter PIN; (for offline PIN)
  - The number of the PIN digit entered;
  - The function key of PIN entry: cancel, clear, OK/enter;
  - etc. (details: please see the onEnterPin listener in EasyLink Android SDK);
- EMV Application Selection notifications (during startTransaction):
  - Prompt: Please Select App;
  - Candidate Application List;

    Based on different models of POS terminals, the Candidate Application List would be shown in two formats. If the POS terminal does not have screen, then the number of "appList" would be 1. And if the POS terminal has screen, then the number of "appList" could be larger than 1, and then terminal would provide a different format of response for customers to select.

Sample code in Android:

```
@Override
public AppSelectResponse onSelectApp(String prompts, int timeout,
List<String> appList, List<String> aidList) {
    Log.w(TAG, "onSelectApp:" + prompts + ", timeout:" + timeout + ",
appList:" + appList);
    updateResult("onSelectApp:" + prompts + ", timeout:" + timeout + ",
appList:" + appList + ", aidList:" + aidList);
    ArrayList<String> arrayList = new ArrayList<>();
    ArrayList<String> arrayAidList = new ArrayList<>();

    AppSelectResponse appSelectResponse = new AppSelectResponse();
    if (appList != null && appList.size() > 0) {
        arrayList.addAll(appList);
        arrayAidList.addAll(aidList);
        updateResultWithWhat(arrayList, arrayAidList, timeout, prompts,
SELECT_APP);
        appSelectCv.block();

        if (selectAppLabel==null || selectAppLabel.size()==0) {
            if (userCancel) {

appSelectResponse.setStatus(AppSelectResponse.CANCEL);//cancel
            } else {

appSelectResponse.setStatus(AppSelectResponse.TIMEOUT);//timeout
            }
        } else {//normal
```

```
                appSelectResponse.setStatus(AppSelectResponse.SUCCESS);
                List<AppList> appLists = new ArrayList<>();
                for(int idx=0; idx< selectAppLabel.size(); idx++) {
                    AppList selApp = new AppList();
                    selApp.setAppLabel(selectAppLabel.get(idx));
                    selApp.setAppIndex(selectAppIndex.get(idx));
                    appLists.add(selApp);
                }
                appSelectResponse.setAppList(appLists);
            }
            return appSelectResponse;

        } else {
            appSelectResponse.setStatus(AppSelectResponse.NO_APP);
            return appSelectResponse;

        }
    }
```

Sample code in IOS:

```
- (PaxReportResponse *)onSelectAppWithPrompts:(NSString *)prompts
timeout:(int)timeout list:(NSArray *)appList list:(NSArray *)aidList{

    PaxReportResponse *report = [[PaxReportResponse alloc] init];

    NSMutableArray *appArray = [[NSMutableArray alloc]init];
    PaxAppList *apps;
    NSString *output = @"";
    for(int i=0; i<[appList count]; i++){

        NSLog(@"onSelectApp appList=%@ aidList=%@", [appList
objectAtIndex:i], [aidList objectAtIndex:i]);
        apps = [[PaxAppList alloc] init];
        apps.appLabel = [appList objectAtIndex:i];
        apps.appIndex = i;
        [appArray addObject:apps];
        output = [output stringByAppendingString:[appList objectAtIndex:i]];
        output = [output stringByAppendingString:@"\n"];
        output = [output stringByAppendingString:[aidList objectAtIndex:i]];
        output = [output stringByAppendingString:@"\n"];
        //report.
    }
    dispatch_async(dispatch_get_main_queue(), ^{
        UIAlertView *view = [[UIAlertView alloc] initWithTitle:@"report"
message:output delegate:self cancelButtonTitle:@"cancel"
otherButtonTitles:@"OK", nil];
        [view show];
    });
    while (!makeSelection) {}
    report.status = @"SUCCESS";
    report.appList= appArray;
    return report;
```

```
}

- (PaxReportResponse *)onSelectAppWithPrompts:(NSString *)prompts
timeout:(int)timeout list:(NSArray *)appInfoList {
    PaxReportResponse *report = [[PaxReportResponse alloc] init];
    NSMutableArray *appArray = [[NSMutableArray alloc]init];
    PaxAppList *apps;
    int i = 0;
    NSLog(@"received report: ");
    for (AppSelectRequest *appInfo in appInfoList) {
        apps = [[PaxAppList alloc] init];
        apps.appLabel = appInfo.appLabel;
        apps.appIndex = i;
        [appArray addObject:apps];
        i++;
    }
    report.status = @"SUCCESS";
    report.appList= appArray;
    NSString *output = [NSMutableString stringWithString:@""];
    NSMutableString *error;
    for (AppSelectRequest *appInfo in appInfoList) {
        NSString *s = [AppSelectRequest toString:appInfo];
        s = [s stringByAppendingString:@"\n"];
        output = [output stringByAppendingString:s];
    }

    dispatch_sync(dispatch_get_main_queue(), ^{
        UIAlertView *view = [[UIAlertView alloc] initWithTitle:@"report"
message:output delegate:self cancelButtonTitle:@"cancel"
otherButtonTitles:@"OK", nil];
        [view show];

    });
    while (!makeSelection) {}
    return report;
}
```

To support receiving notification/report from the EasyLink APP, the Payment APP in master deviceneed to:

1) Update the EasyLink Android SDK to the version v2.00.00 or above;
2) Call setData API to set the value of TAG 0216 to 1;

Then the Easylink Android SDK will trigger the listeners during the startTransaction/completeTransaction/getPinBlock processing.

Note: for the number of PIN digit entered notification, please kindly note that the it needs to be supported by the OS of the slave device as well, the version starts from:

- Prolin v2.4 platform: 2.4.131
- Prolin v2.6 platform: 2.6.45
- Prolin v2.7 platform: 2.7.27

4.9 EMV parameter XML file configuration

The contact EMV parameters which should not be modified by merchant will be configured into a XML file. The XML file format is described in the subsequent chapters. The XML file should be downloaded into terminal through <**FileDownload**> command from smart phone.

4.9.1 EMV AID configuration

No.	Field Name	Required	Attribute	Description
1	PartialAIDSelection	M	B 1	Partial AID Selection Flag The value range is shown as below: 0: Partial Match 1: Full Match
2	ApplicationID	M	Hex…34	Application Identifier
3	IfUseLocalAIDName	M	B 1	The value is as below: 0: Use Application Name From Card 1: Use Application Local Name
4	LocalAIDName	M	Char…16	Local Application Name
5	TerminalAIDVersion	M	Hex 4	Application Version
6	TACDenial	M	Hex 10	TAC Denial
7	TACOnline	M	Hex 10	TAC Online
8	TACDefault	M	Hex 10	TAC Default
9	IfUseTerminalDefaultDDOL	M	N 1	The value range is shown as below: 0: Don't Use Terminal Default DDOL 1: Use Terminal Default DDOL
10	TerminalDefaultDDOL	M	Hex…256	Terminal Default DDOL
11	FloorLimit	M	N…12	Terminal Floor Limit
12	Threshold	M	N…12	Threshold
13	TargetPercentage	M	N…2	Target Percentage
14	MaxTargetPercentage	M	N…2	Maximum Target Percentage
15	TerminalDefaultTDOL	M	Hex…512	
16	TerminalDefaultDDOL	M	Hex…512	
17	TerminalRiskManagementData	M	Hex…10	Provided by issuer. No need to set this until issuer requested.

Note: the AID parameter shall be configured into XML file.

4.9.2 EMV CAPK

No.	Field Name	Required	Attribute	Description
1	RID	M	Hex 10	Application Service Provider ID

No.	Field Name	Required	Attribute	Description
2	KeyID	M	Hex 2	Key Index
3	HashArithmetic Index	M	N 1	HASH Flag
4	RSAArithmeticI ndex	M	N 1	RSA Flag
5	ModuleLength	M	N...4	Module Length
6	Module	M	Hex...496	Module
7	ExponentLengt h	M	N1	Exponent Length
8	Exponent	M	Hex...6	Exponent
9	ExpireDate	M	N 6	Expiration Date (YYMMDD)
10	CheckSum	M	Hex 40	Key Check Sum

Note: the CAPK parameter shall be configured into XML file.

4.9.3 EMV Revoked CAPK

No.	Field Name	Required	Attribute	Description
1	RID	M	Hex 10	Application Service Provider ID
2	KeyID	M	Hex 2	Key Index
3	CertificateSN	M	Hex...6	Issuer Certificate Serial No.

Note: the revoked CAPK parameter shall be configured into XML file.

4.9.4 ICS (Implementation Conformance Statement) Configuration

No.	Field Name	Required	Attribute	Description
1	Type	M	ans...128	ICS configuration type. Currently there are 12 different configurations available for PX terminals. All the different ICS configurations can be pre-configured in the XML parameter file. This field "Type" defines an specific type of ICS configuration. And ECR-POS can designate any one of them to be applied for the current transaction according to different scenarios.
2	TerminalType	M	Hex 2	Terminal type. Below is the supported terminal type

No.	Field Name	Required	Attribute	Description
				according to PX EMV L2 certificates: 22: attended, online with offline capability terminal 25: unattended, online with offline capability terminal
3	CardDataInputCapability	M	Hex 2	Card Data Input Capability (2 characters, i.e. 1 byte HEX data). Each bit represents a flag of a specific capability describing the capability of the terminal to capture card data: bit set to 1 means that the capability is supported while 0 means not supported. Below is the representation of each bit : (bit 8 indicates the most significant bit - i.e. the leftmost bit) b1~b5: Reserved b6: IC with contacts b7: Magnetic stripe b8: Manual key entry
4	CVMCapability	M	Hex 2	CVM Capability (2 characters, i.e. 1 byte HEX data). Each bit represents a flag of a specific capability describing the capability of the terminal to perform CVM (Cardholder Verification Method): bit set to 1 means that the capability is supported while 0 means not supported. Below is the representation of each bit: (bit 8 indicates the most significant bit - i.e. the leftmost bit) b1~b3: Reserved b4: No CVM b5: Enciphered PIN for offline ICC verification

No.	Field Name	Required	Attribute	Description
				b6: Signature (paper) b7: Enciphered PIN for online verification b8: Plaintext PIN for offline ICC verification
5	SecurityCapability	M	Hex 2	Security Capability (2 characters, i.e. 1 byte HEX data). Each bit represents a flag of a specific capability describing the security capability of the terminal: bit set to 1 means that the capability is supported while 0 means not supported. Below is the representation of each bit: (bit 8 indicates the most significant bit - i.e. the leftmost bit) b1~b3: Reserved b4: CDA (Combined DDA/Application Cryptogram Generation) b5: Reserved b6: Card Capture (Always be 0 for POS terminal) b7: DDA (Dynamic Data Authentication) b8: SDA (Static Data Authentication)
6	AdditionalTerminalCapabilities	M	Hex 10	Additional Terminal Capabilities (10 characters, i.e. 5 byte HEX data). Each bit (totally 40 bits) represents a flag of a specific additional terminal capability: bit set to 1 means that the capability is supported while 0 means not supported. Below is the representation of each bit: (byte 1 indicates the leftmost byte)

No.	Field Name	Required	Attribute	Description
				(bit 8 indicates the most significant bit - i.e. the leftmost bit) Byte 1 - Transaction Type Capability b1: Administrative b2: Payment b3: Transfer b4: Inquiry b5: Cashback b6: Services b7: Goods b8: Cash Byte 2 - Transaction Type Capability b1 ~ bit7: Reserved bit 8: Cash Deposit Byte 3 - Terminal Data Input Capability b1 ~ b4: Reserved b5: Function keys b6: Command keys b7:Alphabetic and special characters keys b8: Numeric keys Byte 4 - Terminal Data Output Capability b1: Code table 9 b2: Code table 10 b3~b4: Reserved b5: Display, cardholder b6: Display, attendant b7: Print, cardholder b8: Print, attendant Byte 5 - Terminal Data Output Capability b1: Code table 1 b2: Code table 2 b3: Code table 3 b4: Code table 4 b5: Code table 5 b6: Code table 6

No.	Field Name	Required	Attribute	Description
				b7: Code table 7 b8: Code table 8 (Note: The code table number refers to the corresponding part of ISO/IEC 8859.)
7	GetDataForPINTryCounter	M	N 1	Indicates if supported get the PIN retry counter before let cardholder enter offline PIN. 0 - NOT supported 1 - supported
8	BypassPINEntry	M	N 1	Indicates if bypass PIN entry is allowed 0 - NOT supported 1 - supported
9	SubsequentBypassPINEntry	M	N 1	Indicates if subsequent bypass PIN entry is allowed 0 - NOT supported 1 - supported
10	ExceptionFileSupported	M	N 1	indicates if check exception file is supported 0 - NOT supported 1 - supported
11	ForcedOnlineCapability	M	N 1	indicates if merchant force transaction online is allowed 0 - NOT supported 1 - supported
12	IssuerReferralsSupported	M	N 1	Indicates if referral supported 0 - NOT supported 1 - supported
13	ConfigurationCheckSum	M	Hex 8	Configuration checksum. This checksum is used to make sure that the configuration that is being used is configured according to the EMV L2 certificates. If the checksum is wrong, the transaction will be terminated.

4.9.5 Card Scheme Configuration

No.	Field Name	Required	Attribute	Description
1	AID	O	Hex...34	Application identifier
2	RID	O	Hex...10	Application Service Provider ID
3	ICSTYPE	M	ans...128	ICS configuration type. Currently there are 12 different configurations available for PX terminals. All the different ICS configurations can be pre-configured in the XML parameter file. This field "Type" defines an specific type of ICS configuration. And ECR-POS can designate any one of them to be applied for the current transaction according to different scenarios.

Note: AID, RID is optional, but ether AID, or RID is set at least. If AID is set, then the ICS configuration which ICSTYPE indicates shall be loaded when transaction AID is the same with the set AID; if RID is set, then the ICS configuration which ICSTYPE indicates shall be loaded when card's RID is the same with the set RID.

4.9.6 Terminal common EMV configuration

No.	Field Name	Required	Attribute	Description
1	MerchantID	M	ans...15	Merchant Identification
2	TerminalID	M	ans 8	Terminal Identification
3	TerminalCountryCode	M	Hex 4	Terminal Country Code
4	TerminalCurrencyCode	M	Hex 4	Terminal Currency Code
5	ReferenceCurrencyCode	M	Hex 4	Reference Currency Code
6	TerminalCurrencyExponent	M	Hex 2	Terminal Currency Exponent
7	ReferenceCurrencyExponent	M	Hex 2	Reference Currency Exponent
8	ConversionRatio	M	n...6	the conversion quotients between transaction currency and reference currency (default : 1000) (the exchange rate of transaction currency to reference currency *1000)

No.	Field Name	Required	Attribute	Description
1	MerchantID	M	ans...15	Merchant Identification
2	TerminalID	M	ans 8	Terminal Identification
3	TerminalCountryCode	M	Hex 4	Terminal Country Code
4	TerminalCurrencyCode	M	Hex 4	Terminal Currency Code
5	ReferenceCurrencyCode	M	Hex 4	Reference Currency Code
6	TerminalCurrencyExponent	M	Hex 2	Terminal Currency Exponent
7	ReferenceCurrencyExponent	M	Hex 2	Reference Currency Exponent
8	ConversionRatio	M	n...6	the conversion quotients between transaction currency and reference currency (default : 1000) (the exchange rate of transaction currency to reference currency *1000)

Note: the EMV common configuration parameter shall be configured into XML file.

4.10 Contactless EMV parameter XML file configuration

The contactless EMV parameters which should not be modified by merchant will be configured into a XML file. The XML file format is described in the subsequent chapters. The XML file should be downloaded into terminal through <**FileDownload**> command from smart phone

4.10.1 Paywave parameter configuration

4.10.1.1 Paywave AID configuration

No.	Field Name	Required	Attribute	Description
1	PartialAIDSelection	M	N 1	Select Flag The value is as below: 0: Partial Match 1: Full Match
2	ApplicationID	M	Hex…34	Application Identifier
3	IfUseLocalAIDName	M	N 1	The value is as below: 0: Use Application Name From Card 1: Use Application Local Name
4	LocalAIDName	M	Char…16	Local Application Name
5	TerminalAIDVersion	M	Hex 4	Application Version
6	CryptogramVersion17Supported	M	N…1	MSD CVN17 support flag, 0- not support 1- support
7	ZeroAmountNoAllowed	M	N…1	Amount, Authorized of Zero Check] flag, 0- [Amount, Authorized of Zero Check] activated, online required

No.	Field Name	Required	Attribute	Description
				1- [Amount, Authorized of Zero Check] activated, amount zero not allowed 2- [Amount, Authorized of Zero Check] deactivated
8	StatusCheckSupported	M	N…1	status check support flag, 0-not support， 1- support
9	ReaderTTQ	M	Hex…8	Indicates reader capabilities, requirements, and preferences to the card. Byte 1 bit 8: 1 = MSD supported bit 7: RFU (0) bit 6: 1 = qVSDC supported bit 5: 1 = EMV contact chip supported bit 4: 1 = Offline-only reader bit 3: 1 = Online PIN supported bit 2: 1 = Signature supported bit 1: 1 = Offline Data Authentication (ODA) for Online Authorizations supported. Note: Readers compliant to this specification set TTQ byte 1 bit 1 to 0b Byte 2 bit 8: 1 = Online cryptogram required bit 7: 1 = CVM required bit 6: 1 = (Contact Chip) Offline PIN supported bits 5-1: RFU (00000) Byte 3 bit 8: 1 = Issuer Update Processing supported bit 7: 1 = Mobile functionality supported (Consumer Device CVM) bits 6-1: RFU (000000) Byte 4

No.	Field Name	Required	Attribute	Description
				RF‘ (’00')
10	Inter_WareFloorlimitByTransactionType	M	Node	Sub Node, refer to “2.1.2.2.1 Sub Node Inter_WareFloorlimitByTransactionType description”
11	AdditionalTerminalCapability	M	Hex10	Tag 9F40 Additional Terminal Capbilities Byte 1 Transaction Type Capability bit 8 : 1 = Cash bit 7 : 1 = Goods bit 6 : 1 = Services bit 5 : 1 = Casackhb bit 4 : 1 = Inquiry bit 3 : 1 = Transfer bit 2 : 1 = Payment bit 1 : 1 = Administrative Byte 2 Transaction Type Capability bit 8 : 1 = Cash deposit bit 7 - 1 : RFU (0) Byte 3 (Terminal Data Input Capability) bit 8 : 1 = Numeric keys bit 7 : 1 = Allphabetic and special characters keys bit 6 : 1 = Command keys bit 5 : 1 = Function keys bit 4 – 1 : RFU (0) Byte–4 - Terminal Data Output Capability bit 8: 1 = Print, attendant bit7: 1 = Print, cardholder bit6: 1 = Display, attendant bit5: 1 = Display, cardholder bit4~3: Reserved bit2: 1 = Code table 10 bit1: 1 = Code table 9 Byte–5 - Terminal Data Output Capability bit8: 1 = Code table 1 bit7: 1 = Code table 2 bit6: 1 = Code table 3

No.	Field Name	Required	Attribute	Description
				bit5: 1 = Code table 4 bit4: 1 = Code table 5 bit3: 1 = Code table 6 bit2: 1 = Code table 7 bit1: 1 = Code table 8 (Note: The code table number refers to the corresponding part of ISO/IEC 8859.)
12	TermType	M	Hex…2	*Terminal Type: Below is the supported terminal type according to PX EMV L2 certificates:* *22: attended, online with offline capability terminal* *25: unattended, online with offline capability terminal*
13	TerminalCapability	M	Hex6	*Tag 9F33 Terminal Capability* Byte 1 : Card Data Input Capability b8: Manual key entry b7: Magnetic stripe b6: IC with contacts *b5-1 RFU* Byte 2 : CVM Capability b8:Plaintext PIN for ICC verification b7:Enciphered PIN for online verification b6:Signature (paper) b5:Enciphered PIN for offline verification b4:No CVM required b3-1:RFU Byte 3 ： Security capability b8:SDA b7:DDA b6:Card capture b5:RFU b4:CDA

4.10.1.1.1 Sub Node "Inter_WareFlmtByTransType" description

No.	Field Name	Required	Attribute	Description
1	ContactlessCVMLimit	M	N…12	Contactless CVM Floor Limit
2	ContactlessTransactionLimit	M	N…12	Contactless Transaction Floor Limit
3	ContactlessFloorLimit	M	N…12	Contactless Floor Limit
4	TerminalFloorLimit	M	N…12	Terminal Floor Limit

No.	Field Name	Required	Attribute	Description
5	TerminalFloorLimitSupported	M	N…1	Terminal Offline limit check flag 0-Deactivated, 1-Active and exist, 2-Active but not exist
6	ContactlessTransactionLimitSupported	M	N…1	Card reader contactless transaction limit check flag, 0-Deactivated, 1-Active and exist, 2-Active but not exist
7	CVMLimitSupported	M	N…1	Card reader CVM limit check flag, 0-Deactivated, 1-Active and exist, 2-Active but not exist
8	ContactlessFloorLimitSupported	M	N…1	Card reader contactless Offline limit check flag, 0-Deactivated, 1-Active and exist, 2-Active but not exist
9	TransactionType	M	N2	Transaction type same as Tag“0x9C”

4.10.1.2 Program ID configuration

No.	Field Name	Required	Attribute	Description
1	ProgramId	M	N…17	Program ID
2	ContactlessCVMLimit	M	N…12	Contactless CVM Floor Limit
3	ContactlessTransactionLimit	M	N…12	Contactless Transaction Floor Limit
4	ContactlessFloorLimit	M	N…12	Contactless Floor Limit
5	TerminalFloorLimit	M	N…12	Terminal Floor Limit
6	TerminalFloorLimitSupported	M	N…1	Terminal Offline limit check flag 0-Deactivated, 1-Active and exist, 2-Active but not exist
7	ContactlessTransactionLimitSupported	M	N…1	Card reader contactless transaction limit check flag, 0-Deactivated, 1-Active and exist, 2-Active but not exist
8	CVMLimitSupported	M	N…1	Card reader CVM limit check flag, 0-Deactivated, 1-Active and exist, 2-Active but not exist
9	ContactlessFloorLimitSupported	M	N…1	Card reader contactless Offline limit check flag, 0-Deactivated, 1-Active and exist, 2-Active but not exist
10	CryptogramVersion17Supported	M	N…1	*MSD CVN17 support flag,–0 - not support*–1 - support
11	ZeroAmountNoAllowed	M	N…1	*Amount, Authorized of Zero Check] flag, –0 - [Amount, Authorized of Zero Check] activated, online required–1 - [Amount, Authorized of Zero Check] activated, amount zero not allowed*–2 - [Amount, Authorized of Zero Check] deactivated
12	StatusCheckSupported	M	N…1	status check support flag, 0-not support， 1- support

No.	Field Name	Required	Attribute	Description
13	ReaderTTQ	M	Hex…8	Indicates reader capabilities, requirements, and preferences to the card. Byte 1 bit 8: 1 = MSD supported bit 7: RFU (0) bit 6: 1 = qVSDC supported bit 5: 1 = EMV contact chip supported bit 4: 1 = Offline-only reader bit 3: 1 = Online PIN supported bit 2: 1 = Signature supported bit 1: 1 = Offline Data Authentication (ODA) for Online Authorizations supported. Note: Readers compliant to this specification set TTQ byte 1 bit 1 to 0b Byte 2 bit 8: 1 = Online cryptogram required bit 7: 1 = CVM required bit 6: 1 = (Contact Chip) Offline PIN supported bits 5-1: RFU (00000) Byte 3 bit 8: 1 = Issuer Update Processing supported bit 7: 1 = Mobile functionality supported (Consumer Device CVM) bits 6-1: RFU (000000) Byte 4 RF‘ (’00')

4.10.2 Paypass parameter configuration

4.10.2.1 Paypass AID configuration

No.	Field Name	Required	Attribute	Description
1	KernelConfiguration	M	Hex…2	DF811B Indicates the Kernel configuration options

No.	Field Name	Required	Attribute	Description
				b8:Mag-stripe mode contactless transactions not supported b7:EMV mode contactless transactions not supported b6:On device cardholder verification supported b5:Relay resistance protocol supported b4-1:Each bit RFU decide by AID
2	TornLeftTime	M	N…4	DF811C Maximum time, in seconds, that a record can remain in the Torn Transaction Log. (only for chip card)
3	MaximumTornNumber	M	Hex…2	DF811D Max Number of Torn Transaction Log Records (only for chip card)
4	MagneticCVM	M	Hex…2	DF811E Mag-stripe CVM Capability-CVM required(PayPass) b8-5 0000: NO CVM 0001: OBTAIN SIGNATURE 0010: ONLINE PIN 1111: N/A b4-1 NFU
5	MageticNoCVM	M	Hex…2	DF812C Mag-stripe CVM Capability-No CVM required b8-5 0000: NO CVM 0001: OBTAIN SIGNATURE 0010: ONLINE PIN 1111: N/A b4-1 NFU
6	MobileSupport	M	Hex…2	Tag 9F7E Mobile Support Indicator b8-3： RFU b2 :Offline PIN Required

No.	Field Name	Required	Attribute	Description
				b1：Mobile supported
7	CardDataInput	M	Hex…2	Tag DF8117 Card Data Input Capability
8	CVMCapability_CVMRequired	M	Hex…2	Tag DF8118 CVM Capability - CVM Required
9	CVMCapability_NoCVMRequired	M	Hex…2	Tag DF8119 CVM Capability - No CVM Required (0x08)
10	TerminalType	M	Hex…2	Tag 9F35 Terminal Type
11	AccoutType	M	Hex2	Tag 5F57 Account type
12	AdditionalTerminalCapability	M	Hex10	Tag 9F40 Additional Terminal Capbilities
13	KernelID	M	Hex 1	Tag DF810C Kernel ID
14	SecurityCapability	M	Hex 1	Tag DF811F security capability b1~b3: reserved b4: CDA b5: reserved b6: Card Capture (Always be 0) b7: DDA b8: SDA
15	PartialAIDSelection	M	N 1	Select Flag The value is as below: 0: Partial Match 1: Full Match
16	ApplicationID	M	Hex…34	Application Identifier
17	IfUseLocalAIDName	M	N 1	The value is as below: 0: Use Application Name From Card 1: Use Application Local Name
18	LocalAIDName	M	Char…16	Local Application Name
19	TerminalAIDVersion	M	Hex 4	Tag 9F09 : Application Version
20	MagneticApplicationVersionNumber	M	Hex 4	9F6D Mag-stripe Application Version Number (Reader)
21	TACDenial	M	Hex 10	TAG DF8121 : TAC Denial (only for chip card)

No.	Field Name	Required	Attribute	Description
22	TACOnline	M	Hex 10	TAG DF8122:TAC Online (only for chip card)
23	TACDefault	M	Hex 10	TAG DF8120: TAC Default (only for chip card)
24	TerminalRisk	M	Hex 16	(Tag 9F1D) B7:Plaintext PIN for ICC verification (Contactless) B6:Enciphered PIN for online verification (Contactless) B5:Signature (paper) (Contactless) B4:Enciphered PIN for offline verification (Contactless) b3:No CVM required (Contactless) b2:On device cardholder verification (Contactless) *(only for chip card)*
25	ContactlessCVMLimit	M	N…12	*Tag DF8126 Contactless CVM Floor Limit*
26	ContactlessTransactionLimit_NoOnDevice	M	N…12	*Tag DF8124 Indicates the transaction amount above which the transaction is not allowed, when on-device cardholder verification is not supported.*
27	ContactlessTransactionLimit_OnDevice	M	N…12	*Tag DF8125 Indicates the transaction amount above which the transaction is not allowed, when on-device cardholder verification is supported.*
28	ContactlessFloorLimit	M	N…12	*Tag DF8123 Contactless Floor Limit*

4.10.3 Other contactless EMV parameter configuration

No.	Field Name	Required	Attribute	Description
1	CountryCode	M	Hex 4	Terminal Country Code
2	CurrencyCode	M	Hex 4	Terminal Currency Code
3	RefCurcyCode	M	Hex 4	Reference Currency Code
4	CurrencyExponent	M	Hex 1	Terminal Currency Exponent
5	ReferenceCurrencyExponent	M	Hex 2	Reference Currency Exponent

No.	Field Name	Required	Attribute	Description
6	MerchantCategoryCode	M	Hex 4	Classifies the type of business being done by the merchant, represented in accordance with [ISO 8583:1993] for Card Acceptor Business Code.
7	MerchantId	M	Asc…15	When concatenated with the Acquirer Identifier, uniquely identifies a given merchant.
8	TerminalID	M	N 8	Designates the unique location of the Terminal.
9	MerchantName	M	ASC…128	Indicates the name of the merchant.
10	MerchantLocalAddress	M	ASC…128	Indicates the location of the merchant.
11	ReferenceCurrenceConverRate	M	N6	The conversion quotients between transaction currency and reference currency. (the exchange rate of transaction currency to reference currency *1000) default : 1000

4.10.4 ExpressPay parameter configuration

4.10.4.1 ExpressPay AID configuration

No.	Field Name	Required	Attribute	Description
1	PartialAIDSelection	M	N 1	Select Flag The value is as below: 0: Partial Match 1: Full Match
2	ApplicationID	M	Hex…34	Application Identifier
3	IfUseLocalAIDName	M	N 1	The value is as below: 0: Use Application Name From Card 1: Use Application Local Name
4	LocalAIDName	M	Char…16	Local Application Name
5	TerminalAIDVersion	M	Hex 4	Tag 9F09 : Application Version
6	TACDenial	M	Hex 10	TAG DF8121 : TAC Denial (only for chip card)
7	TACOnline	M	Hex 10	TAG DF8122:TAC Online (only for chip card)
8	TACDefault	M	Hex 10	TAG DF8120: TAC Default (only for chip card)
9	ExpresspayTerminalCapabilities	M	Hex2	Tag 9F6D: Expresspay Terminal Capability Bit(8-7)

No.	Field Name	Required	Attribute	Description
				00: Expresspay 1.0 01: Expresspay 2.0 MagStrip Only 10: Expresspay 2.0 EMV and MagStrip 11: Expresspay Mobile *EMV Supported*
10	DDOL	M	Hex…256	Terminal Default Dynamic Data. Authentication Data Object List
11	TDOL	M	Hex…256	Terminal Default Data Object List
12	ContactlessCVMLimit	M	N…12	Contactless CVM Limit
13	ContacltessTransactionLimit	M	N…12	Indicates the tranasaction amount above which the transaction is not allowed
14	ContactlessFloorLimit	M	N…12	Contactless Floor Limit
15	ContactlessFloorLimitCheck	M	N	Check Contactless floor limit: 0- Not check 1- Check No used, Reserved
16	TerminalSupportOptimizationModeTransaction	M	N1	The terminal whether to support the optimization mode transaction 0: Not supported *1: Supported*
17	UnpredictableNumberRange	M	N4	*The Range of Magnetic Stripe unpredictable Number(00-99)*
18	TerminalTransactionCapability	M	Hex8	9F6E TerminalTransactionCapability Byte 1: Bit8: 1=AEIPS contact mode supported Bit7: 1=Expresspay Magstripe Mode supported Bit6: 1 = Expresspay EMV full online mode supported Bit5: 1=Expresspay EMV partial online mode supported Bit4: 1=Expresspay Mobile Supported b3-b1: RFU Byte 2: Bit8: 1 = Mobile CVM supported

No.	Field Name	Required	Attribute	Description
				Bit7: 1 = Online PIN supported Bit6: 1 = Signature Bit5: 1 = Plaintext Offline PIN B4-b1: RFU Byte 3: Bit8: 1 = Terminal is offline only Bit7: 1 = CVM Required B6-b1: RFU Byte 4: RFU
19	DelayAuthorizationSupport	M	N1	1: support Delayed Authorization; 0: not support Delayed Authorization
20	TerminalCapability	M	Hex6	Tag 9F33 Terminal Capability
21	TerminalAdditionalCapability	M	Hex10	Tag 9F40 Terminal Additional Capability
22	TerminalType	M	Hex 2	Terminal type. Below is the supported terminal type according to PX EMV L2 certificates: 22: attended, online with offline capability terminal *25: unattended, online with offline capability terminal*

4.10.4.2 ExpressPay DRL configuration

No.	Field Name	Required	Attribute	Description
1	RdClssTxnLmt	M	N…12	Indicates the tranasaction amount above which the transaction is not allowed
2	RdCVMLmt	M	N…12	Contactless CVM Limit
3	RdClssFloorLmt	M	N…12	Contactless Floor Limit
4	TermFloorLmt	M	N…12	Terminal Floor Limit
5	ProgramId	M	N…17	Program ID
6	PrgramIdLen	M	N2	Program ID Length
7	RdClssFloorLmtFlg	M	N…1	Card reader contactless Offline limit check flag, 0-Deactivated, 1-Active and exist, 2-Active but not exist
8	RdClssTxnLmtFlg	M	N…1	Card reader contactless transaction limit check flag, 0-Deactivated, 1-Active and exist, 2-Active but not exist

No.	Field Name	Required	Attribute	Description
9	RdCVMLmtFlg	M	N…1	Card reader CVM limit check flag, 0-Deactivated, 1-Active and exist, 2-Active but not exist
10	TermFloorLmtFlg	M	N…1	Terminal Offline limit check flag *0-Deactivated, 1-Active and exist, 2-Active but not exist*
11	StatusCheckFlg	M	N1	status check support flag, 0-not support， 1- support
12	AmtZeroNoAllowed	M	N…1	Amount, Authorized of Zero Check] flag, 0- [Amount, Authorized of Zero Check] activated, online required 1- [Amount, Authorized of Zero Check] activated, amount zero not allowed 2- [Amount, Authorized of Zero Check] deactivated
13	DynaLmicLimitSet	M	Hex…2	Dynamic Limit Set Id

4.10.5 DPAS parameter configuration

4.10.5.1 DPAS AID configuration

No.	Field Name	Required	Attribute	Description
1	PartialAIDSelection	M	N 1	Select Flag The value is as below: 0: Partial Match 1: Full Match
2	ApplicationID	M	Hex…34	Application Identifier
3	IfUseLocalAIDName	M	N 1	The value is as below: 0: Use Application Name From Card 1: Use Application Local Name
4	LocalAIDName	M	Char…16	Local Application Name
5	TerminalAIDVersion	M	Hex 4	Tag 9F09 : Application Version
6	ZeroAmountNoAllowed	M	N…1	Amount, Authorized of Zero Check] flag, 0- [Amount, Authorized of Zero Check] activated, online required 1- [Amount, Authorized of Zero Check] activated, amount zero not allowed

No.	Field Name	Required	Attribute	Description
				2- [Amount, Authorized of Zero Check] deactivated
7	StatusCheckSupported	M	N1	status check support flag, 0-not support， 1- support
8	TACDenial	M	Hex 10	TAG DF8121 : TAC Denial (only for chip card)
9	TACOnline	M	Hex 10	TAG DF8122:TAC Online (only for chip card)
10	TACDefault	M	Hex 10	TAG DF8120: TAC Default (only for chip card)
11	TerminalFloorLimit	M	N…12	Terminal Floor Limit
12	ContactlessCVMLimit	M	N…12	TagDF8126 : Contactless CVM Limit
13	ContactlessTransactionLimit	M	N…12	TagDF8125 : Indicates the tranasaction amount above which the transaction is not allowed
14	ContactlessFloorLimit	M	N…12	TagDF8124 : Contactless Floor Limit
15	ReaderTTQ	M	Hex…8	Tag9F66 : reader capabilities. Byte 1 bit 8: 1 = Magnetic stripe mode supported bit 7: RFU (0) bit 6: 1 = EMV Mode supported bit 5: 1 = EMV contact chip supported bit 4: 1 = Offline-only reader bit 3: 1 = Online PIN supported bit 2: 1 = Signature supported bit 1: RFU Byte 2 bit 8: 1 = Online cryptogram required bit 7: 1 = CVM required bit 6: 1 = (Contact Chip) Offline PIN supported bits 5-1: RFU (00000) Byte 3 bit 8: 1 = Issuer Update Processing supported

No.	Field Name	Required	Attribute	Description
				bit 7: 1 = Mobile functionality supported (Consumer Device CVM) bits 6-1: RFU (000000) Byte 4 RF‘ (’00')
16	TerminalCapability	M	Hex6	Tag 9F33 Terminal Capability
17	TerminalFloorLimitSupported	M	N…1	Terminal Offline limit check flag 0-Deactivated, 1-Active and exist, 2-Active but not exist
18	ContactlessTransactionLimitSupported	M	N…1	Card reader contactless transaction limit check flag, 0-Deactivated, 1-Active and exist, 2-Active but not exist
19	CVMLimitSupported	M	N…1	Card reader CVM limit check flag, 0-Deactivated, 1-Active and exist, 2-Active but not exist
20	ContactlessFloorLimitSupported	M	N…1	Card reader contactless Offline limit check flag, 0-Deactivated, 1-Active and exist, 2-Active but not exist
21	TerminalType	M	Hex…2	Tag9F35 : Terminal Type
22	AcquirerID	M	Hex…6	Tag9F01 : Acquirer ID

4.10.6 JCB parameter configuration

4.10.6.1 JCB AID configuration

No.	Field Name	Required	Attribute	Description
1	PartialAIDSelection	M	N 1	Select Flag The value is as below: 0: Partial Match 1: Full Match
2	ApplicationID	M	Hex…34	Application Identifier
3	IfUseLocalAIDName	M	N 1	The value is as below: 0: Use Application Name From Card 1: Use Application Local Name
4	LocalAIDName	M	Char…16	Local Application Name
5	TerminalAIDVersion	M	Hex 4	Application Version
6	ContactlessCVMLimit	M	N…12	TagDF8126 Contactless CVM Floor Limit
7	ContactlessFloorLimit	M	N…12	TagDF8123Contactless Floor Limit
8	ContactlessTransactionLimit	M	N…12	TagDF8124 Contactless Transaction Floor Limit

No.	Field Name	Required	Attribute	Description
9	CombinationOptions	M	Hex 4	TagFF8130 : Combination Options Byte 1 bit 8: RFU (0) bit 7: 1 = Status Check supported bit 6: 1 = Offline Data Authentication supported bit 5: 1 = Exception File Check required bit 4: 1 = Random Transaction Selection supported bit 3: RFU(0) bit 2: 1 = EMV Mode Supported bit 1: 1 = Legacy Mode supported Byte 2 : RFU
10	MaxTargetPercentage	M	N…2	TagFF8131 Maximum Target Percentage
11	TACDefault	M	Hex 10	TagDF8120 TAC Default
12	TACDenial	M	Hex 10	TagDF8121 TAC Denial
13	TACOnline	M	Hex 10	TagDF8122 TAC Online
14	TargetPercentage	M	N…2	TagFF8132 Target Percentage
15	TerminalCapabilityIndicator	M	Hex 2	Tag9F52 : Terminal Compatibility Indicator Byte 1: bit8 – bit3 : RFU(0) bit 2: 1 = EMV Mode Supported bit 1: 1 = Magstripe Mode Supported
16	TerminalInterchangeProfile	M	Hex 6	Tag9F53: Terminal Interchange Profile (static) Byte 1: bit 8: 1 = CVM required by reader / N/A bit 7: 1 = Signature supported bit 6: 1 = Online PIN supported bit 5: 1 = On-Device CVM supported bit 4: RFU(0) bit 3: 1 = Reader is a Transit Reader

No.	Field Name	Required	Attribute	Description
				bit 2: 1 = EMV contact chip supported bit 1: 1 = (Contact Chip) Offline PIN supported Byte 2: bit 8: 1 = Issuer Update supported bit 7- bit 1: RFU(0) Byte 3 : RFU(0)
17	TerminalType	M	Hex 2	Tag9F35 Terminal type. Below is the supported terminal type according to PX EMV L2 certificates: 22: attended, online with offline capability terminal 25: unattended, online with offline capability terminal
18	TerminalCapability	M	Hex6	Tag 9F33 Terminal Capability
19	Threshold	M	N…12	TagFF8133 Threshold

4.10.7 QPBOC parameter configuration

4.10.7.1 QPBOC AID configuration

No.	Field Name	Required	Attribute	Description
1	PartialAIDSelection	M	N 1	Select Flag The value is as below: 0: Partial Match 1: Full Match
2	ApplicationID	M	Hex…34	Application Identifier
3	IfUseLocalAIDName	M	N 1	The value is as below: 0: Use Application Name From Card 1: Use Application Local Name
4	LocalAIDName	M	Char…16	Local Application Name
5	TerminalAIDVersion	M	Hex 4	Application Version
6	AdditionalTerminalCapability	M	Hex10	Tag 9F40 Additional Terminal Capbilities
7	ContactlessCVMLimit	M	N…12	Contactless CVM Floor Limit
8	ContactlessFloorLimit	M	N…12	Contactless Floor Limit
9	ContactlessQPSLimit	M	N…12	Contactless Quick Pass Limit
10	ContactlessTransactionLimit	M	N…12	Contactless Transaction Limit

No.	Field Name	Required	Attribute	Description
11	IsSupportQPS	M	N 1	If support Quick Pass 0: false 1: true
12	TerminalType	M	Hex 2	Terminal type. Below is the supported terminal type according to PX EMV L2 certificates: 22: attended, online with offline capability terminal 25: unattended, online with offline capability terminal
13	TerminalCapability	M	Hex6	Tag 9F33 Terminal Capability
14	TornLogMaxNum	M	Hex 2	Torn log Max number
15	TornMaxLifeTime	M	Hex4	Torn Max life Time(unit: Second)
16	TornSupport	M	N1	Torn log support flag 1: Support 0: Not support
17	ReaderTTQ	M	Hex…8	Indicates reader capabilities, requirements, and preferences to the card. Byte 1 bit 8: 1 = MSD supported bit 7: RFU (0) bit 6: 1 = qVSDC supported bit 5: 1 = EMV contact chip supported bit 4: 1 = Offline-only reader bit 3: 1 = Online PIN supported bit 2: 1 = Signature supported bit 1: 1 = Offline Data Authentication (ODA) for Online Authorizations supported. Note: Readers compliant to this specification set TTQ byte 1 bit 1 to 0b Byte 2 bit 8: 1 = Online cryptogram required bit 7: 1 = CVM required

No.	Field Name	Required	Attribute	Description
				bit 6: 1 = (Contact Chip) Offline PIN supported bits 5-1: RFU (00000) Byte 3 bit 8: 1 = Issuer Update Processing supported bit 7: 1 = Mobile functionality supported (Consumer Device CVM) bits 6-1: RFU (000000) Byte 4 RFU‘ (’00')

4.11 FileDownload

Smart phone can call this command to download file into the PAX terminal. Terminal support file with following suffix:

1) .ui : for UI layout file (XML format);
2) .emv: for EMV contact parameter file (XML format);
3) .clss: for EMV contactless parameter file (XML format);
4) .font: for font file (binary format);
5) .so: only for library file to the terminal with Prolin platform, like d220;
   Note: D180S is not support this kind of file;
6) .bmp, .png, .gif: for picture file;
   Note: D180 only support .bmp format file;
7) .bin: for application or monitor file to the terminal with Monitor platform;
8) .aip: for application file to the terminal with Prolin platform;
   Note: D180S is not support this kind of file;
9) .lng: for translation file to the terminal with Prolin or Monitor platform;
10) .os: for Prolin OS file; (Note: D180S is not support this kind of file)
11) .monitor: for monitor OS file;
12) .puk: for monitor USPUK; (Note: support from EasyLink Lite v1.03.00, the D180S MONITOR V3.06, D180C MONITOR V4.06)

Attention:

1. If the name and suffix of the file to be downloaded is already exists in the terminal, then the new file shall overwrite the file in the terminal;
2. If the file to be downloaded is application, monitor, Prolin OS, or font file, then the terminal shall reboot automatically after download successful;

- **Sample code in Android:**

```
class myFileDownloadListener implements FileDownloadListener
```

```
{public void onDownloadProgress(int current, int total){
    progressDialog.dismiss();
```

```
            updateResult("process:" + current + "/" + total );
        }
}

private void fileDownloadFun(String fileName)
    {Context context = EasyLikActivity.this;
    String filePath = context.getFilesDir().getPath() + "/app/"+fileName;
    mkDir(context.getFilesDir().getPath() + "/app/");
    copyFileFromAssert(context, fileName, filePath);
    int ret = easyLink.fileDownLoad(fileName, filePath, new
myFileDownloadListener());
}
```

- **Sample code in IOS:**

```
void(^downloadProcessCB)(NSInteger current, NSInteger total) = ^(NSInteger
  current,  NSInteger  total){
                dispatch_async(dispatch_get_main_queue(),  ^{
                        [self.textInfo setText:[NSString stringWithFormat:@"p
rocess:[%ld/%ld]",current,total]];
                });
        };

dispatch_async(dispatch_get_global_queue(0,
        0),^{PaxEasyLinkRetCode ret;
    ret = [_easyController fileDownLoad:filename filePath:filePath
processBlock:downloadProcessCB];
        dispatch_async(dispatch_get_main_queue(), ^{
            [self.textInfo setText:[NSString stringWithFormat:@"ret=%d",ret]];
        });
    });
```

5 RKI Feature

EasyLink solution support to inject keys by PAX RKI.

5.1 Remote Download RKI Key

PAX EasyLink supports online download key from PAX RKI server, such as Master key, DUKPT key etc., user call < **rkiRemoteKeyDownLoad** > command to inject key.

Attention:
Before download key, user should register the POS terminal in RKI server.
When using the iOS SDK, make sure that the path provided to this API is the location of "rkiCert.bundle" , old version of the API without this parameter has been deprecated.

- **Sample code in Android:**

```
private void testRKIRemoteDownloadKey() {
    new Thread(()->{
        easyLink.rkiRemoteKeyDownLoad(etHostIP.getText().toString(),
                etHostPort.getText().toString(),  RKIFuncActivity.this);
    }).start();
}

public class RKIFuncActivity extends AppCompatActivity implements
IRKIStatusListener, View.OnClickListener {
     …
     @Override
     public void onDisplayStatus(String displayMessage) {
         Bundle bundle = new Bundle();
         bundle.putString("statusMsg", displayMessage);
         Message msg= new Message();
         msg.what = NORMALSTATUS;
         msg.setData(bundle);
         mayHandler.sendMessage(msg);
     }

     @Override
     public boolean isNetworkEnable() {
         return true;
     }
}
```

- **Sample code in IOS:**

```
WS(weakSelf);
void(^showStatusCB)(NSString *message) = ^(NSString *message){
  dispatch_sync(dispatch_get_main_queue(), ^{
     __strong __typeof(weakSelf)strongSelf = weakSelf;
     strongSelf.statusMessage.textAlignment = NSTextAlignmentCenter;
     NSError *error;
     NSDate *jsonData =[message dataUsingEncoding:NSUTF8StringEncoding];
     NSDictionary *dic = [NSJSONSerialization JSONObjectWithData:jsonData
                          options:NSJSONReadingMutableContainers error:&error];
     if(!dic || error){
        strongSelf.statusMessage.text = @"Json parse faile!";
```

```
        }else{
            strongSelf.statusMessage.text = dic[@"statusMessage"];
        }
        [strongSelf updateUI];

    });

};

dispatch_async(dispatch_get_global_queue(0, 0), ^{
    PaxEasyLinkRetCode ret = 0;
     ret = [_easyLinkController rkiRemoteKeyDownload:self.hostUrl.text
hostPort:self.hostPort.text rkiBundlePath:path statusMessageBlock:showStatusCB];
    if(EL_RET_OK == ret){
        dispatch_sync(dispatch_get_main_queue(), ^{
            [self.statusMessage setText:@"download key success!"];
        });
    }
});
```

5.2 Inject key by offline RKI key file

PAX EasyLink supports offline inject key from key file which exported from PAX RKI server, user call < **rkiInjectOfflineKey**> command to inject key.

Attention:

1. Before download key, user should register the POS terminal in RKI server.
2. Key file name extension should be ".rki"

➢ **Sample code in Android:**

```
private void downloadRKIOfflineKeyFile() {

    String filePath = getFilesDir().getPath() + "/app/" +
"PAX_D135_1890000712_0.rki";
    Tools.mkDir(getFilesDir().getPath() + "/app/");
    Tools.copyFileFromAssert(this, "PAX_D135_1890000712_0.rki", filePath);

    new Thread(()-> {
        easyLink.rkiInjectOfflineKey(filePath, RKIFuncActivity.this);
    }).start();
}

public class RKIFuncActivity extends AppCompatActivity implements
IRKIStatusListener, View.OnClickListener {
     ...
     @Override
     public void onDisplayStatus(String displayMessage) {
         Bundle bundle = new Bundle();
         bundle.putString("statusMsg", displayMessage);
         Message msg= new Message();
         msg.what = NORMALSTATUS;
         msg.setData(bundle);
```

```
            mayHandler.sendMessage(msg);
        }

        @Override
        public boolean isNetworkEnable() {
            return true;
        }
}
```

- **Sample code in IOS:**

```
void(^showStatusCB)(NSString *message) = ^(NSString *message){
    dispatch_sync(dispatch_get_main_queue(), ^{
        __strong __typeof(weakSelf)strongSelf = weakSelf;
        strongSelf.statusMessage.textAlignment = NSTextAlignmentCenter;
        NSError *error;
        NSDate *jsonData =[message dataUsingEncoding:NSUTF8StringEncoding];
        NSDictionary *dic = [NSJSONSerialization JSONObjectWithData:jsonData
                                options:NSJSONReadingMutableContainers error:&error];
        if(!dic || error){
            strongSelf.statusMessage.text = @"Json parse faile!";
        }else{
            strongSelf.statusMessage.text = dic[@"statusMessage"];
        }
        [strongSelf updateUI];
    });

};

    dispatch_async(dispatch_get_global_queue(0, 0), ^{
        PaxEasyLinkRetCode ret = 0;
        NSArray<NSString *> *splits = [self.fileName.text componentsSeparatedByString:@"."];

        NSString *name = [splits objectAtIndex:0];
        NSString *extension = [splits objectAtIndex:1];

        NSString *keyPath = [[NSBundle mainBundle] pathForResource:name ofType: extension];
        ret = [_easyLinkController rkiInjectOfflineKey:keyPath
statusMessageBlock:showStatusCB];
        if(EL_RET_OK == ret){
            dispatch_sync(dispatch_get_main_queue(), ^{
                [self.statusMessage setText:@"download key compelete!"];
            });
        }
    });
```

5.3 Get RKI certificate information

PAX EasyLink supports read the list of RKI certificates installed in PAX terminal, and contents

format of certificate is Json script. User call < rkiGetCertList > command to get RKI certificate list from PAX terminal.

Note the keys of Json script as below:

```
"statusMessage"
"certType"
"Version"
"SN"
"startDate"
"expiry"
"issueName"
"SubjectName"
"certList"
```

- **Sample code in Android:**

```
public void testGetRKICert(){
    new Thread(()->{

        ByteArrayOutputStream keyInfo = new ByteArrayOutputStream();
        byte [] bKeyType = new byte[1];
        bKeyType[0] = KeyType.PED_TIK;
        int Ret = easyLink.getKeyInfo(bKeyType[0], (byte) 0x01, keyInfo);
        if ((Ret == ResponseCode.EL_RET_OK) && (keyInfo != null) &&
(keyInfo.size()>0)) {
            byte[] keyInfoData = keyInfo.toByteArray();

        }

        StringBuffer certBuffer = new StringBuffer();
        int ret = easyLink.rkiGetCertList( certBuffer, RKIFuncActivity.this);
        if(ret == 0){
            Bundle bundle = new Bundle();
            bundle.putString("certList", certBuffer.toString());
            Message msg= new Message();
            msg.what = NORMALSTATUS;
            msg.setData(bundle);
            mayHandler.sendMessage(msg);
        }
    }).start();
}
```

- **Sample code in IOS:**

```
dispatch_async(dispatch_get_global_queue(0, 0), ^{
    PaxEasyLinkRetCode ret = 0;
     NSString *certList = [[NSString alloc] init];
     ret = [_easyLinkController getRKICertList:&certList statusMessageBlock:showStatusCB];
     if(EL_RET_OK == ret && certList != nil){
        showStatusCB(certList);
     }
  });
```

5.4 Unbind device

PAX EasyLink supports online unbind PAX termina from PAX RKI server, if the terminal recalled.

user call < **rkiRemoteKeyDownLoad** > command to inject key.

Note: this function is limited by RKI server, and contact PAX RKI support team before use this function

Attention:

Only Pax terminal had registered and injected RKI key from RKI server, then if the terminal will be recalled, the terminal can be unbonded with RKI server.

When using the iOS SDK, make sure that the path provided to this API is the location of "rkiCert.bundle", old version of the API without this parameter has been deprecated.

- **Sample code in Android:**

```
private void testRKIUnbid(){
     new Thread(()->{
         easyLink.rkiUnBindDevice(etHostIP.getText().toString(),
                 etHostPort.getText().toString(), RKIFuncActivity.this);
      }).start();
}
```

- **Sample code in IOS:**

```
dispatch_async(dispatch_get_global_queue(0, 0), ^{
    PaxEasyLinkRetCode ret = 0;
     ret = [_easyLinkController rkiUnBindDevice:self.hostUrl.text hostPort:self.hostPort.text
rkiBundlePath:path statusMessageBlock:showStatusCB];
     if(EL_RET_OK == ret){
        NSLog(@"unbind success");
        dispatch_sync(dispatch_get_main_queue(), ^{
           [self.statusMessage setText:@"Unbind device seccess!"];
        });
     }

  });
```

6 Other features of terminal

6.1 Function key

➢ EasyLink Lite

Press [**ENTER** + **CANCEL**] KEY to show the application menu,

- **SETCOMM**
- **2-TERMINFO**

  1-SETCOMM

  Set communication type, including: USB, Bluetooth, RS232

  2-TERMINFO

  Show terminal information, including: application version, release date, firmware version.
- **3-KEYINJECT**

  Key injection by Master POS.

6.2 M30/M50/M8

6.2.1 Connection

M8 and M30, M50 and R20 connects and communicate based on serial port **"/dev/mux1"** through glComm library. Thus, the Android payment application needs to import PAX *GLComm library*.

6.2.2 Sleep – Wake

- Sleep

  Due to power saving, the Mxx/R20 will go to sleep in a certain time (indicated by TAG 0101)if EasyLink App does not receive any request or no key detected.
- Wake

  The Android payment application based on EasyLink Android SDK in M30/M50/M8 needs to invoke *wakeup()* API whose package name is *com.pax.dal* (as described by the Diagrambelow).

Power/Sleep Control

M50 provides APIs to control the power of the payment module and obtain the related status. The package name of the API is *com.pax.dal*, and the APIs are as follows. For more detailed description, please refer to the *Neptune LiteApi* and *IPaymentDevice*.

Modifier and Type	Method and Description
void	controlPower(boolean isOn) power on/off the R20
int	getStatus() obtain the R20 status
void	wakeup() wake up the R20

For more details, please refer to the *Neptune LiteApi* and *IpaymentDevice*, and the M30/M50 programming guide which released on PPN.

6.3 Sync And Process Task

PAX EasyLink supports sync and process tasks pushed from PAXSTORE, such as RKI Download and APP Install/Update. User implement Listener < **ISyncAndProcessListener**>, and call < **syncAndProcessTask** > command to sync and process tasks from PAXSTORE.

Attention:
Before syncing and processing task, user should register the terminal (D135) in PAXSTORE server.
To successfully sync and process task, the terminal should work under good network environment.

- **Sample code in Android:**

```
private Boolean cancelInstall = false;
ISyncAndProcessListener syncAndProcessListener = new ISyncAndProcessListener()
{
    @Override
    public void onDisplayStatus(String message) {
            updateResult("Message:" + current + "/" + total );

    }
    @Override
    public void onDownloadStatus(int current, int total){
        updateResult("Download process:" + current + "/" + total );
    }
    @Override
    public void onInstallStatus(int current, int total) {
        updateResult("Install process:" + current + "/" + total );
    }

    @Override
    public boolean abort() {
        return cancelInstall;
```

```
    }
};
easyLink.syncAndProcessTask(inputHost, inputPort, syncAndProcessListener);
```

➢ **Sample code in IOS:**

```
@implementation SyncAndProcessListener {
    UILabel *message;
    Boolean abort;
}

-(id) init:(UILabel*)message {
    if (self = [super init]){
        self->message = message;
    }
    return self;
}

-(void)setAbort:(Boolean)val {
    abort = val;
}

- (BOOL)abort {
    return abort;
}

- (void)onDisplayStatus:(NSString *)message {
    dispatch_async(dispatch_get_main_queue(), ^{
        [self->message setText:message];
    });
    NSLog(@"onDieplayStatus: %@", message);
}

- (void)onDownloadStatus:(NSInteger)downloadedLength
totalLength:(NSInteger)totalLength {
    dispatch_async(dispatch_get_main_queue(), ^{
        [self->message setText:@"start download"];
        [self->message setText:[[NSString alloc]initWithFormat:@"download
process: %ld / %ld", (long)downloadedLength, (long)totalLength]];
    });
    NSLog(@"download process: %ld / %ld", (long)downloadedLength,
(long)totalLength);
}

- (void)onInstallStatus:(NSInteger)current total:(NSInteger)total {
    dispatch_async(dispatch_get_main_queue(), ^{
        [self->message setText:@"start install"];
        [self->message setText:[[NSString alloc]initWithFormat:@"install
process: %ld / %ld", (long)current, (long)total]];
    });
    NSLog(@"install process: %ld / %ld", (long)current, (long)total);
```

```
}

@end

- (IBAction)onClick_SyncAndProcess:(id)sender {
    [syncAndProcess setAbort:false];
    [_easyLinkController syncAndProcessTask:[_hostUrl text] port:[[_hostPort text]
intValue] listener:syncAndProcess];
}
```

Note the keys of Json script is the same: "statusMessage";

onDisplayMessage would return message including error message (see SyncAndProcessTask return code) and prompt information message listed below.

Sync and Process task:

```
"Sync Task"
"Process Finish"
"Sync&Process Complete"
```

onDownload (from server)

```
"Download " + filename
"Download success"
```

App Install/Update task:

```
"Install " + filePath
"No need to update " + originalName
```

RKI Download task:

```
"ProcessRKI"
```

（For RKI, other messages refer to RKI feature)

Appendix

Appendix 1 – Terminal information list

TAG	Element	Attribute	Read Write	Description
0101	TermSN	ans…32	R	Terminal SN. The returned serial number is a string which ending with '\0'. The maximum length is 32 bytes. If SerialNo[0] = '\0', then it means there is no serial number. Only 8 bytes are used currently, all are digits.
0102	TermModelCode	ans…6	R	Indicates terminal model code, such as: "S80", "S900" etc.
0103	TermPrinterInfo	a 1	R	Indicates the printer type, '–' - Stylus printer. 'T'- Thermal printer.
0104	TermModemExist	an 1	R	Return the Modem module exist or not, '–' - not exist. '–' - exist.
0105	TermUSBExist	an 1	R	Return the USB module exist or not, '–' - not exist. '–' - exist.
0106	TermLANExist	an 1	R	Return the LAN module exist or not, '–' - not exist. '–' - exist.
0107	TermGPRSExist	an 1	R	Return the GPRS module exist or not, '–' - not exist. '–' - exist.
0108	TermCDMAExist	an 1	R	Return the CDMA module exist or not, '–' - not exist. '–' - exist.
0109	TermWIFIExist	an 1	R	Return the WIFI module exist or not,

				'–' - not exist. '–' - exist.
0110	TermRFExist	an 1	R	Return the RF module exist or not, '–' - not exist. '–' - exist.
0111	TermICExist	an 1	R	Return the IC module exist or not, '–' - not exist. '–' - exist.
0112	TermMAGExist	an 1	R	Return the MAG module exist or not, '–' - not exist. '–' - exist.
0113	TermTILTExist	an 1	R	Return the TILT module exist or not, '–' - not exist. '–' - exist.
0114	TermWCDMAExist	an 1	R	Return the WCDMA module exist or not, '–' - not exist. '1' - exist.
0115	TermTOUCHSCRExist	an 1	R	Return the touch screen module exist or not, '0' - not exist. '1' - exist.
0116	TermCOLORSCRExist	an 1	R	Return the color screen module exist or not, '0' - not exist. '1' - exist.
0117	TermScrSize	n 8	R	The size of the screen, first two bytes indicate the width, last two bytes indicate the height.
0118	TermAppName	ans…33	R	The name of the application, such as "EasyLink".
0119	TermAppVer	ans…33	R	The version of the application, such as "1.00.00".
0120	TermAppSOName	ans…33	R	The name of the dynamic library of application, such as "libabc".
0121	TermAppSOVer	ans…33	R	The version of the dynamic library of application, such as "1.00.00".
0122	TermPubSOName	ans…33	R	The name of the dynamic library of public, such as

				“libabc”.
0123	TermPubSOVer	ans…33	R	The version of the dynamic library of public, such as “1.00.00”.
0124	TermBatteryInfo	a 1	R	Indicates the battery information, “7” – Battery is absent “6” – external power supply, battery has already completed charging. “5” – external power supply, and battery is charging. “4” – power 80% ~ 100% “3” – power 60% ~ 80% “2” – power 40% ~ 60% “1” – power 20% ~ 40% “0” – power 0% ~ 20%
0125	TermRestFSSize	an…12	R	The rest file system size, such as: “1024567” meams file system still has 1024567bytes.
0126	TermOSName	ans…24	R	The operation system name, such as: “prolin”, “monitor plus”.
0127	TermOSVer	Ans…8	R	The operation system version, such as: “2.4.31”.
0128	TermPCIExist	A 1	R	Return the PCI module exist or not, ‘0’ - not exist. ‘1’ - exist.
0129	TermTime	N 14	WR	Terminal time , BCD format :YYMMDDhhmmss, set time with format: YYMMDDhhmmss, and get time with format: YYMMDDhhmmss.
012B	parFileVerArray	Ans512	R	Parameter file name and version array, format : “FileName1= Ver1; FileName2= Ver2; … FileNameN=VerN” **Note:** 1. FileName and version is getting from the first paramet “ filePath” of API “ fileDownLoad”. 2. The file name in “filePath” should be : ”

				FileName_vVer.xxx” 3. If no parameter file be downloaded, parFileVerArray is null; For example: Download parameter files “clss_param_v1.0.1.clss”, “emv_param_v1.0.0.emv”, then the parFileVerArray will be: “clss_param=1.0.1;emv_param= 1.0.0”
0x12C	exReaderSN	ans…32	R	External Reader SN. The returned serial number is a string which ending with ‘\0‘. The maximum length is 32 bytes. If SerialNo[0] = ‘\0‘, then it means there is no serial number. .
0x12D	exReaderOSVer	Ans…16	R	The operation system version of external reader, such as: “1.0.00.0000”.
0x012E	certVerArray	Ans…100	R	Certification file version array, format: “CertName1= Ver1; CertName2= Ver2” For example: “PCI=1.06;P2PE=2.00.00”

Note:

1. The length of configuration tag shall be fixed to 2 bytes; The L in TLV shall be conformed to the rule of L in EMV BER-TLV definition;

Appendix 2 – Application parameter list

TAG	Element	Attribute	Read Write	Description
0201	SleepModeTimeout	n…4	WR	Timeout in 1s unit for waiting timeout. Valid value should be [30, 120]. The value 0 means the device won't sleep. Such as: 0x01 0x20 means 120 s.
0202	PINEncryptionType	n 2	WR	For PIN encryption, indicates which kind of encryption is goingto adopt, valid value shall be as below: 1 – TDES 2 – DUKPT
0203	PINEncryptionKeyIdx	n 2	WR	For PIN encryption, indicates which key isgoing to be used for data encryption
0204	PINBlockMode	n 2	WR	PIN block mode, 0x00 – ISO9564 format 0.0x01 – ISO9564 format 1. 0x02 – ISO9564 format 3. ~~0x03 – HK EPS format.~~
0205	DataEncryptionType	n 2	WR	For Data encryption, indicates which kind of encryption is going to adopt, valid value shall be as below: 0 – no encryption(reserved) 1 – TDES 2 – RSA 3 – MAC 4– DUKPT 5 – CYBS DUKPT 6 - AES Note: • Special version (Non P2PE) to support 0, default value is 0; • Normal version default value is 1;

0206	DataEncryptionKeyIdx	n 2	WR	For Data encryption, indicates which key is going to be used for data encryption
0207	FallbackAllowFlag	N 2	WR	Indicates whether allow fallback. 0 – Fallback not allowed 1 – Fallback allowed
0208	PANMaskStartPos	n 2	WR	Indicates the first few number of clear digits for the masked PAN, valid value: 0 to 6. Default value: 6
0209	Transaction processing mode	N2	WR	Transaction processing mode: 0 – normal 1 – demo Note: for demo mode, not do bellowing check: a) expiry date
020A	DUKPT DES mode for data encryption	N2	WR	0: ECB decryption 1: ECB encryption 2: CBC decryption 3: CBC encryption
0210	MAC calculation key index	N2	WR	For calculating MAC, indicates which key is going to be used for MAC calculation.
0211	Language	Ans…128	WR	Language for application. “English” “Farsi” “Japanese” …
0212	MAC calculation key type	N2	WR	MAC calculation key type: 1 – TAK 2 – TIK Default: 1
0213	MAC calculation mode	N2	WR	MAC calculation mode: 0x00(default): Doing TDES encryption for BLOCK1

				by using MAC key(TAK). 0x01: Doing bitwise XOR for BLOCK1 and BLOCK2; Do bitwise XOR again by using previous XOR result with BLOCK3 (TAK). 0x02: ANSIX9.19 standard, Do DES encryption for BLOCK1 by using MAC key (only take the first 8 bytes of key) (TAK). 0x20: Doing TDES encryption for BLOCK1 by using MAC request key (TIK). 0x21: Doing bitwise XOR for BLOCK1 and BLOCK2; Do bitwise XOR againby using previous XOR result with BLOCK3. 0x22: ANSIX9.19 standard, Do DES encryption for BLOCK1 by using MAC key (only take the first 8 bytes of key). 0x40: Doing TDES encryption for BLOCK1 by using MAC response key. 0x41: Doing bitwise XOR for BLOCK1 and BLOCK2; Do bitwise XOR againby using previous XOR result with BLOCK3. 0x42: ANSIX9.19 standard, Do DES encryption for BLOCK1 by using MAC key (only take the first 8 bytes of key).
0214	CardEntryMode	N2	WR	Card entry mode while detecting card in StartTransaction

				processing. 0x01 – MSR Swipe 0x02 – ICC insert 0x04 – PICC tap 0x08 – Manual PAN entry Accept the combination of mode, like 0x01 | 0x02: accept MSR swipe and ICC insert 0x02 | 0x04: accept ICC insert and PICC tap 0x01 | 0x02 | 0x04: accept MSR swipe, ICC insert, and PICC tap. … Default value: 0x01 | 0x02 | 0x04
0215	MsrRequestPINAuto	N2	WR	Request PIN automatically or not for MSR processing flow in StartTransaction API. 1 –request PIN automatically 2 – not request PIN automatically Default value: 1
0216	SupportReport	N2	WR	If support POS terminal send status to host, like searching card, enter PIN, try againand so on. 0(default): do not support, POS terminal will not send status to host. 1: support report. POS terminal will send status to host
0217	EmvRefundNeedAuthorize	N2	WR	For refund transaction with EMV chip card, POS terminal need to do EMV authorization (ODA, CVM, 1stGAC) or not.

				0 (default): need to do authorization; 1: no need to do EMV authorization;
0218	contactlessLightStandard	N2	WR	To indicate the light processing standard of EMV contactless : 0 (default): standard light processing, see Book_A of contactless EMV specification (Option 2); 1: Europe standard;
0219	configTags	N2	WR	Configuration TAGs set by smart device, after read card data, the EasyLink App in PinPad will get the value of TAGs indicated in 0219, then send the TLVs data to the smart device.
021A	emvTags	N2	WR	EMV/CTLS TAGs set by smart device, after read card data, the EasyLink App in PinPad will get the value of TAGs indicated in 021A, then send the TLVs data to the smart device.
021B	notificationCallbackTimeout	N4	WR	Indicate the timeout of the callback or receiving response for the notification or report. Value value: [30, 180]; Unit: second; Default: 30s;
021C	cvmLimitIndicator	N2	WR	CVM limit indicator: 0x00 – use CVM limit from clss_param.clss; 0x01 – use CVM limit from TAG 021C

				Default: 0x00
021D	cvmLimit	N12	WR	Contactless CVM limit.
021E	paddingRuleKey	Ans…128	WR	Used to choose or match padding rule in <paddingrule.paddingrule> parameter file.
021E	LuhnCheck	N2	WR	Enable LUHN check for pan 0 – don't check; 1 – check Default: 0x00
021F	AidFilterType	An1	WR	For contactless aid select according below type: "N" - all aids "0" - credit aids "1" - common debit aids "2" - global debit aids "3" - commonn debit first, then global debit, dual entrance Debit EDC "4" - global debit first, then common debit, dual entrance Debit EDC "5" - commonn debit first, then credit aid, dual entrance Credit EDC
0220	ExpiryDateCheck	n2	WR	Expiry Date Check indicator: 0 – don't check; 1 – check Default: 0x00
0221	AutoTestFlag	n2	WR	Auto Test indicator: 0 – Normal, detect remove card; 1 – Auto Test, don't detect remove card Default: 0x00
0222	EmvMode	n2	WR	EMV Mode indicator: 0 –Full EMV; 1 –Partial EMV Default: 0x00
0223	PowerOffTimeout	n2	WR	Timeout in 1 minute unit for shutdown after sleep. Valid value should be [0, 99]. The value 0 means the device won't shutdown. Such as: 0x10 means 10 minutes. If Tag0201 SleepModeTimeout = 0, this Tag doesn't effects.

0224	TrackPriority	n2	WR	High-priority track number: If read pan failed from track 2 under swipe mode, which prior track be used to get pan. The value should be 0x01 or 0x03. Default value is 0x01;
0225	detectCardTimeout (lite)	N4	WR	Indicate the timeout of detecting card during startTransaction and completeTransaction processing. Valid value: (0, 60] Unit: second
0226	selectAppOnScreen (lite)	N2	WR	Indicate whether need select aid list on mPOS terminal. 0x01 – yes, on mPOS terminal 0x00 – no, on master device default:0x00
0227	notShowIdleUI(lite)	N2	WR	Indicate whether POS terminal will show Idle UI automatically after finish StartTransaction\CompleteTransaction\GetPinBlock command. 0x01 – yes, don't show idle UI; 0x00 – no, show idle UI; default:0x00 **Note:** If this tag be set 0x01, after POS terminal finish StartTransaction\CompleteTransaction\GetPinBlock command, master device should send ShowPage\ShowBarcode command to show UI on POS terminal.
0228	tapDelayTime	N4	WR	Indicate the delay time for checking card swipe or insert while detect tap card. Valid value: (0, 9999] If value is 0, then POS terminal doesn't check card swipe or insert while detecting card.

				Unit: millisecond
0229	reportCandListData	N2	WR	Indicate whether reports all candidate app list data to master device. 0x01 – yes, report all data 0x00 – no, only app Label and aid default:0x00
022A	fallbackMode	N2	W	Indicates current swipe card mode is fallback mode or not. 0 – Normal mode while swipe a card 1 – Fallback mode while swipe a card **Notes:** 1. Default value : 0x00 This tag only takes effect if the Tag0214 is 0x01.
022B	manualReqExpDateAuto	N2	WR	Request expiry date automatically or not for input account processing flow in StartTransaction API. 1 –request expiry date automatically 2 – not request expiry date automatically Default value: 1
022C	manualReqVCodeAuto	N2	WR	Request V-Code automatically or not for input account processing flow in StartTransaction API. 1 –request V-Code automatically 2 – not request V-Code automatically Default value: 1
022E	paddingRuleKey (full)	Ans…128	WR	Used to choose or match padding rule in <paddingrule.paddingrule> parameter file.
022F	track2SentinelFlag (full)	N2	WR	Indicate whether need to add the start and end sentinel for track2 data (TAG 57, TAG 0305): 0x00 – Keep the data format with the track 2 data Appendix 14 – Track 2 data format ;

				0x01 – 1) add start (;) and end(?) sentinel for track 2 data (TAG 0305); 2) add start (;) and end(?) sentinel for strack2 equivalent data (TAG 57) if the data format is ASCII as <paddingrule.paddingrule > indicated;
0230	panEncFormat	Ans…255	WR	Indicate the pan, expiry date, cvv,… combination data's encryption format; **Refer to the Note 2.**
0231	trackEncFormat	Ans…255	WR	Indicate the combination track data's encryption format; **Refer to the Note 2.**
0232	emvEncFormat	Ans…255	WR	Indicate the emv sensitive data encryption format, data content have 2 parts and split by ";". Part 1: Tag name = special labels split by "+" ; a) Tag name: EMV tag or "ALL"; "All"- all tags have same filling format. b) Special label: **"ASCII, TLV"**; "ASCII"-all data should be expanded to ASCII code; "TLV"-data should be pack as TLV format. Part 2: padding=padding value(0-no padding; 1-padding with zero; 2-padding with PKCS7) Example: ALL=TLV+ASCII;padding=1 All sensitive emv tag data should be packed into TLV data, then expands into ASCII code; 56= TLV;57=ASCII;padding=1 56 should be packed into TLV data, and 57 should be expanded into ASCII code.
0233	SupportExternalReader	N2	WR	Indicate whether support external magnetic reader;

				1 –yes, support 0 –no, doesn't support Default value: 0 Note: this tag only used for Mxx+R20 device
0234	OtherInterfaceAllow	N2	WR	Indicate whether support convert to use other interface to continue transaction while contactless transaction prompt use contact card. 1 –yes, allow 0 –no, doesn't allow Default value: 1
0235	SupportPINBypass	N2	WR	Indicate whether support PIN bypass for enter pin. 2 – auto bypass 1 –yes, support 0 –no, doesn't support Default value: 0
0236	SupportCheckIccInit	N2	WR	Indicate whether support check "Iccinit" interface to determine the current card is an EMV chip card or not. 1 –yes, support 0 –no, doesn't support Default value: 0 Note: this tag only uses for Octobox. Not supported by common version.
0237	ReplacementEncryptionType	n 2	WR	For replacement data encryption, indicates which kind of encryption is going to adopt, valid value shall be as below: 0 – no encryption(reserved special version support) 1 – TDES 2 – RSA 3 – MAC 4– DUKPT 5 – CYBS DUKPT 6 - AES Note: Data will be decrypted by tag 0205, 0206 and re-encrypted by tag 0237, 0238

				through ReplacementEncryption command
0238	ReplacementEncryptionKeyIdx	n 2	WR	For replacement data encryption, indicates which key is going to be used for data encryption
0239	AES DES mode for data encryption	N2	WR	0: ECB decryption 1: ECB encryption 2: CBC decryption 3: CBC encryption 4: OFB decryption 5: OFB encryption Default value:1
023A	autoGenerateKey	N2	WR	Indicate whether generate the key if key is not existed: 0x00 – NO 0x01 – YES Default value: 0 Note: only used for AES key.
023B	SupportCompleteTap	N2	WR	Indicate whether support prompt tap card in CompleteTransaction command. 0x00 – NO 0x01 – YES Default value: 0x01
023C	SupportNonFinancialCard	N2	WR	Indicate whether support non financial card while swipe a card. 0x00 – NO 0x01 – YES Default value: 0x00
023D	EnablePlaintextForNonPaymentCard	N2	WR	Indicate enable get plaintext of non-card association issued card (card association: Visa, Master, Diners, Discover, JCB, Amex), such as merchant's gift card etc. 0x00 – NO 0x01 – YES Default value: 0x01
023E	EnableAddFlagForTrack	N2	WR	Indicate whether add start/stop sentinel for track data 0x00 – NO 0x01 – YES Default value: 0x01
023F	ManualIncreaseKsn	N2	WR	Indicate whether increase

				KSN by call command IncreaseKSN; 0x00 – NO (KSN will be increased automatically in command StartTransaction) 0x01 – YES Default value: 0x00
0240	BeepForDetectCard	N2	WR	Indicate whether beep while detect card, Only use for IccDetect/PiccDetect command 0x00 – NO 0x01 – YES Default value:0x00;
0241	EnableFuncMenu	N2	WR	Indicate whether enable function menu while press some special keys(press cancel key then press enter key) ; 0x00 – NO 0x01 – YES Default value:0x00; Note: only support by the device has screen.
0242	ClearSensitiveDateTime	N4	WR	Indicate the timeout to clear the sensitive data after reading card success. Valid value: (1, 1200] Default value: 300 Unit: second **(Reserved)**
0243	GetExpiryDateFromTrack	N2	WR	Indicate whether enable get expiry date from track data; 0x00 – NO 0x01 – YES Default value:0x00; Note: only support by the device has screen.

Note:

1. The length of configuration tag shall be fixed to 2 bytes; The L in TLV shall be conformed to the rule of L in EMV BER-TLV definition;
2. Encryption data content have 2 parts and split by ";".
   a. Part 1: format=content(special label + filling content)
      i. Special label: "**PAN, EXPDATE,VCODE,TRACK1, TRACK2, TRACK3**"
      ii. Filling content: should be quote by "()", and the () is option, if () exist, that means what's in () should be present, if any data in parentheses is null, then all data in parentheses and parentheses should be discard.
   b. Part 2: padding=padding value(0-no padding; 1-padding with zero; 2-padding with PKCS7)

c. Example :
“format= PAN+”|”+EXPDATE+”|”+VCODE;padding=1”
That means encryption data should be like “1234567890123456|2309|345” and padding with ‘0x00”.
“format= “(%”+TRACK1+”?)”+”(;”+TRACK2+”?)”;padding=1”
That means encryption data should be like “%track1 data?;track2 data?” and padding with ‘0x00”; if track 1 is not exist, then only “;track2 data?”.

Appendix 3 – Transaction parameter list

Tag	Element	Attribute	Read Write	Description
0301	CurrentTxnType	n 2	R	Indicates current transaction type: 0 – NONE 1 – Magnetic Swipe Card 2 – Fallback Swipe Card 3 – EMV Contact Card 4 – Contactless Card 5 – Manual PAN entry 6 – Fallback with no app 0xFF – Unknown the card type

0302	CurrentCLSSType	n 2	R	Indicates current contactless transaction type: 0 – KERNTYPE_DEF 1 – KERNTYPE_JCB 2 – KERNTYPE_MC 3 – KERNTYPE_VIS 4 – KERNTYPE_PBOC 5 – KERNTYPE_AE 6 – KERNTYPE_ZIP 0xFF – Unknown the contactless transaction type

0303	CurrentPathType	n 2	R	Indicates current contactless path type: 0 – CLSS_PATH_NORMAL 1 – CLSS_VISA_MSD 2 – CLSS_VISA_QVSDC 3 – CLSS_VISA_VSDC 4 – CLSS_VISA_CONTACT 5 – CLSS_MC_MAG 6 – CLSS_MC_MCHIP 7 – CLSS_VISA_WAVE2 8 – CLSS_JCB_WAVE2 9 – CLSS_VISA_MSD_CVN17 10 – CLSS_VISA_MSD_LEGACY 11 – CLSS_JCB_MAG 12 – CLSS_JCB_EMV 13 – CLSS_LEGACY 14 – CLSS_DPAS_MAG 15 – CLSS_DPAS_EMV 16 – CLSS_DPAS_ZIP 17 – CLSS_AE_MAG 18 – CLSS_AE_EMV 0xFF – Unknown the contactless path type
0304	Track1 data	ans…79	R	Track1 data, Track1 needs 79 bytes.
0305	Track2 data	ans…37	R	Track2 data, Track2 needs 37 bytes.
0306	Track3 data	ans…107	R	Track3 data, Track3 needs 107 bytes.
0307	Expire date	ans 4	R	Expire date of the card.
0308	OnlineAuthorization Result	n 2	WR	Only use for EMV contact and EMV contactless. Indicate the result of online authorization 0: transaction approved online 1: transaction declined online 2: connect host failed
0309	Response code	An 2	WR	Authorisation Response Code. Code that defines the disposition of a message
030A	promptFallbackFlag	N2	W	Prompt fallback indicator: 0- Don't prompt fallback 1- Prompt fallback Defalult value 0x00

				If Tag0207 FallbackAllowFlag = 0, this Tag doesn't effects.
0310	Auth code	An 6	WR	Authorisation Code. Value generated by the authorisation authority for an approved transaction
0311	Auth data	Ans…16	WR	Issuer Authentication Data. Data sent to the ICC for online issuer Authentication.
0312	Auth data length	N 4	WR	Issuer Authentication Data length.
0313	Issuer script	Ans…300	WR	Issuer script, tag 71 + tag 72 data.
0314	Issuer script length	N 4	WR	Issuer script length
0315	Online pin input	An 1	WR	Online pin input, Online PIN input: 0 – no 1 – yes 2 – PIN bypass
0316	Pin block	Ans 8	WR	Pin block
0317	ksn	Ans 10	WR	ksn
0318	ICS	Ans…128	WR	Used to choose ICS. Fill in the ICS type.
0319	MaskedPAN	Ans…32	R	Mask PAN.
031A	Holder Verification Method	B1	R	Holder Verification Method 0x00 – NO CVM 0x01 – SIGNATURE 0x02 – ONLINE PIN 0x03 – OFFLINE PIN 0x04 – REFERENCE THE CUSTOMER DEVICE 0xFF – Unrecognized CVM
031B	Card processing result	B1	R	Card processing result: 0x00 – Offline declined 0x01 – Offline approved 0x02 – Online request 0x03 – Online approved 0x04 – Online declined 0x05 – Terminated Note: only for EMV, EMV contactless card

031C	PAN data	Ans…19	R	Primary account number
031D	Currency symbolor acronym	Ans…3	WR	Symbol or acronym of currency, eg, if currency is US dollars, then the value could be $ or “USD”
031E	55th field of 8583 returned by the host	Ans…256	WR	55th field of 8583 returned by the host
031D	Currency symbol or acronym	Ans…3	WR	Symbol or acronym of currency, eg, if currency is US dollars, then the value could be $ or “USD”
031E	55th field of 8583 returned by the host	Ans…256	WR	55th field of 8583 returned by the host
031F	issuerScriptProc Result	Ans…5	R	Issuer script processing result, byte1: Most significant nibble: Result of the Issuer Script processing performed by the terminal: '0' = Script not performed '1' = Script processing failed '2' = Script processing successful Least significant nibble: Sequence number of the Script Command '0' = Not specified '1' to 'E' = Sequence number from 1 to 14 'F' = Sequence number of 15 or above byte 2-5: Script Identifier of the Issuer Script received by the terminal, if available, zero filled if not. Mandatory if more than one Issuer Script was received by the terminal. Bytes 1–5 are repeated for each Issuer Script processed by the

				terminal. For details, please refer to EMV book4 A5 Issuer Script Results (Table 34: Issuer Script Results)
0320	cardHolderName	Ans26	R	Card holder name: This Tag only for MSR card.
0321	pinKsn	Ans10	R	Ksn of Dukpt get pin
0322	macKsn	Ans10	R	Ksn of Dukpt MAC
0323	AidFilterType	An1	W	For contactless aid select according below type: "N" – Not filter, according card filter rules; "0" – credit aids "1" – common debit aids "2" – global debit aids "3" – common debit first, then global debit, dual entrance Debit EDC "4" – global debit first, then common debit, dual entrance Debit EDC "5" – common debit first, then credit aid, dual entrance Credit EDC **Note:** **This tag replaces the tag021F**
0324	supportAidType	n2	W	which aid type will be filtered with Tag0325 0x01 – credit aids 0x02 – common debit aids 0x04 – global debit aids Accept the combination of mode, like 0x01 | 0x02: accept credit and common debit 0x02 | 0x04: accept common debit and global debit aids 0x01 | 0x02 | 0x04: accept credit, common debit and global debit aids. … If Tag0325 is not "6", this tag must be present; otherwise, this tag doesn't be used.

0325	vmAidFilterType	Ans1	W	"N" – Not filter, according card priority; "0" – credit aids "1" – common debit aids "2" – global debit aids "6" – According the rules of Tag0323
0326	V-Code	Ans 4	R	V-Code of card
0327	panCombinData	Ans… 255	R	The combination encryption data follow tag0230
0328	tracCombinData	Ans… 255	R	The combination encryption data follow tag0231
0329	emvCombinData	Ans… 255	R	The combination encryption data follow tag0232 Reserved
032A	IsChip	Ans1	R	If magnetic card is chip or normal card, used for fallback swipe 1 – Chip 0 – Not Chip
032B	MaskData	Ans…32	R	Get masked data for special input data UI "EnterAllDigits.xml"
032C	CheckWhitelist	N2	W	support check card number in whitelist: 0 - don't support 1 - support Defalult value 0x00 If don't support, all card number will be returned with encryption data. If support, only card number in whitelist will be returned with plaintext, otherwise will be returned with encryption data.
032D	UnsupportAidList	Ans…255	w	Indicate un-supported aid list for transaction Format: count(1byte) + aid group(total counts) Aid group: aidLen(1byte)+aid For example \x02\x05\xA000000030\x06\xA00000004010
032E	CombinEMVTag	Ans…20	W	The first byte should be tag list length. From 2's byte, it will be the list of combination sensitive EMV tags. All the tags are

				consecutive without any separator. e.g. Request for tag 0x5A 0x57 0x56, then this field would be: "\x03\x5A\x57\x56"
032F	IsPlainText	N2	R	Current sensitive data is plaintext. 0x00 – No 0x01 - Yes Note: this value is only be used for nonpayment card.
0330 **(Reserved)**	TransLimitExceed	N2	R	Transaction amount is over contactless transaction limit. No value return – unknow 0x01-No 0x02-Yes Default: No value return **Note:** **Current IOS version doesn't support this tag.**

Note:

1. The length of configuration tag shall be fixed to 2 bytes; The 'L' in TLV shall be conformed to the rule of L in EMV BER-TLV definition;
2. The tag 0309 could be replaced with EMV tag 8A;
3. The tag 0310 could be replaced with EMV tag 89;
4. The tag 0311, 0312 could be replaced with EMV tag 91;
5. The tag 0313, 0314 could be replaced with EMV TLV data 71, 72;

Appendix 4 – Data to be set before StartTransaction

TAG	Field Name	Required	Attribute	Description
9F02	TxnAmount	M	N…12	Transaction Amount
9C	TransactionType	M	Hex 2	Transaction type
5F2A	TxnCurcyCode	M	Hex 4	Transaction Currency Code
5F36	TxnCurcyExp	M	Hex 2	Transaction Currency Exponent
9A	TxnDate	M	N 6	Transaction Date (YYMMDD)
9F21	TxnTime	M	N 6	Transaction Time (hhmmss)

Appendix 5 – Data to be set before CompleteTransaction

TAG	Field Name	Required	Attribute	Description
91	Issuer Authentication Data	C	b 8-16	Issuer authentication data which is contained in the host response message. (value of tag 8A)
71	Issuer Script Data 1	C	var…256	Issuer Script Data which is contained in the host response message. (value of tag 71)
72	Issuer Script Data 2	C	Var…256	Issuer Script Data which is contained in the host response message. (value of tag 72)
0308	Online Authorization	M	N 1	Only use for EMV contact. Indicate the result of online

	Result			authorization 0: transaction approved online 1: transaction declined online 2: connect host failed
8A	Response Code	M	ans2	Only use for EMV contact. Host authorization response code
89	Authorization Code	C	ans6	Only use for EMV contact. Host authorization code

Appendix 6 – EMV TAGs can be accessed after StartTransaction

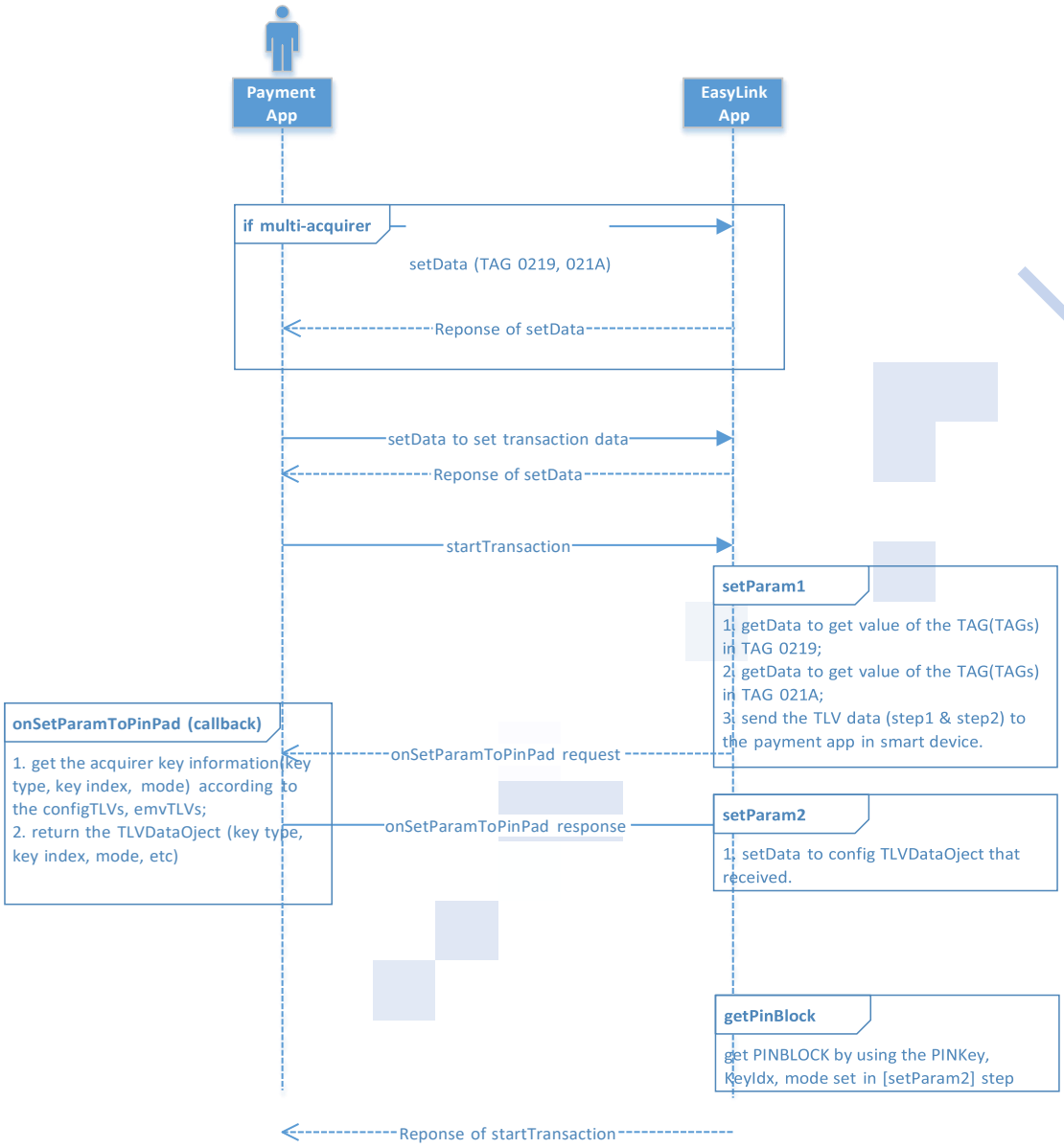
Tag Identifier	Description
4F	Application Identifier
9F12	Application preferred Name
50	Application Label
5F30	Service Code
5F20	Cardholder Name
57	Track 2 Equivalent Primary Account Number Field Separator (Hex 'D') Expiration Date (YYMM) Service Code Discretionary Data (defined by individual payment systems) Pad with one Hex 'F' if needed to ensure whole bytes
5A	Application Primary Account Number
5F24	Application Expiry Date
5F34	Application Primary account Number sequence number
5F25	Application Effective Date
95	Terminal Verification Results
9B	Transaction Status Information
9F27	Cryptogram Information Data
8A	Authorization response Code
9F26	Cryptogram returned by the ICC in response of the GENERATE AC command
9F36	Application Transaction Counter Counter maintained by the application in the ICC (incrementing the ATC is managed by the ICC)
9F10	Issuer Application Data
9F41	Transaction Sequence Counter from terminal
9F02	Amount Authorized
9F03	Amount (other) (TIP)
5F36	Transaction currency component
9F1B	Terminal Floor Limit
9F1C	Terminal Identification
9F35	Terminal Type

Tag Identifier	Description
9F1A	Terminal Country Code
5F2A	Transaction Currency Code
82	Application Interchange Profile Indicates the capabilities of the card to support specific functions in the application
9F37	Unpredictable Number Value to provide variability and uniqueness to the generation of a cryptogram
9C	Transaction Type
84	Dedicated File Name
9F09	Application Version Number
9F34	CVM Cardholder verification Method Result
9F07	Application Usage Control Indicates issuer‘s specified restrictions on the geographic usage and services allowed for the application
9F0D	Issuer Action Code-Default
9F0E	Issuer Action Code – Denial
9F0F	Issuer Action Code – Online
9F33	Terminal Capabilities
5F34	Application PAN sequence Number
9F39	POS Entry Mode
8E	CVM List Identifies a method of verification of the cardholder supported by the application
5A	Application Primary Account Number
57	Track 2 equivalent
5F20 OR 9F0B	Cardholder name or Extended
9F1F	Track 2 discretionary data
9F7A	VLP process indicator
9F74	VLP authorization code

Appendix 7 – Set parameter from smart device while startTransaction

The smart device can set parameters according to the business requirement to the EasyLink App while startTransaction.

For example, if the payment app in smart device need to support multi-acquirer, then the payment app in smart device need to set the TAG 0219, 021A first. The EasyLink app will invoke getData to get the values of TAGs as TAG 0219, 021A indicated. Then pack the TLVs data and report to the smart device. The smart device need to unpack the TLV data, and can get to know the corresponding KEY information (PIN key type, PIN key index, etc) according to the unpacked TLV data, then set the TAG 0202, 0203 to the PinPad (EasyLink App). Below is the transaction flow.



Note:

1) Except the multi-acquirer scenario, the smart device can also set the parameters to support different usage scenario.

Appendix 8 – callBackCfg.report configuration

The file callBackCfg.report is used to indicate the whether the EasyLink APP need to send the [message] to the Payment APP on Master device during the startTransaction processing, and Payment APP on Master device can update some of the [message], then send the updated [message] back to the EasyLink APP.

The [message] can be:

1. Selected AID parameters;
2. ICS parameters to be loaded into the EMV kernel;

3. Application parameters for Easylink APP, like TAG 0202, 0203, etc.
4. Processing status, like “read card OK” (read EMV APP data OK)

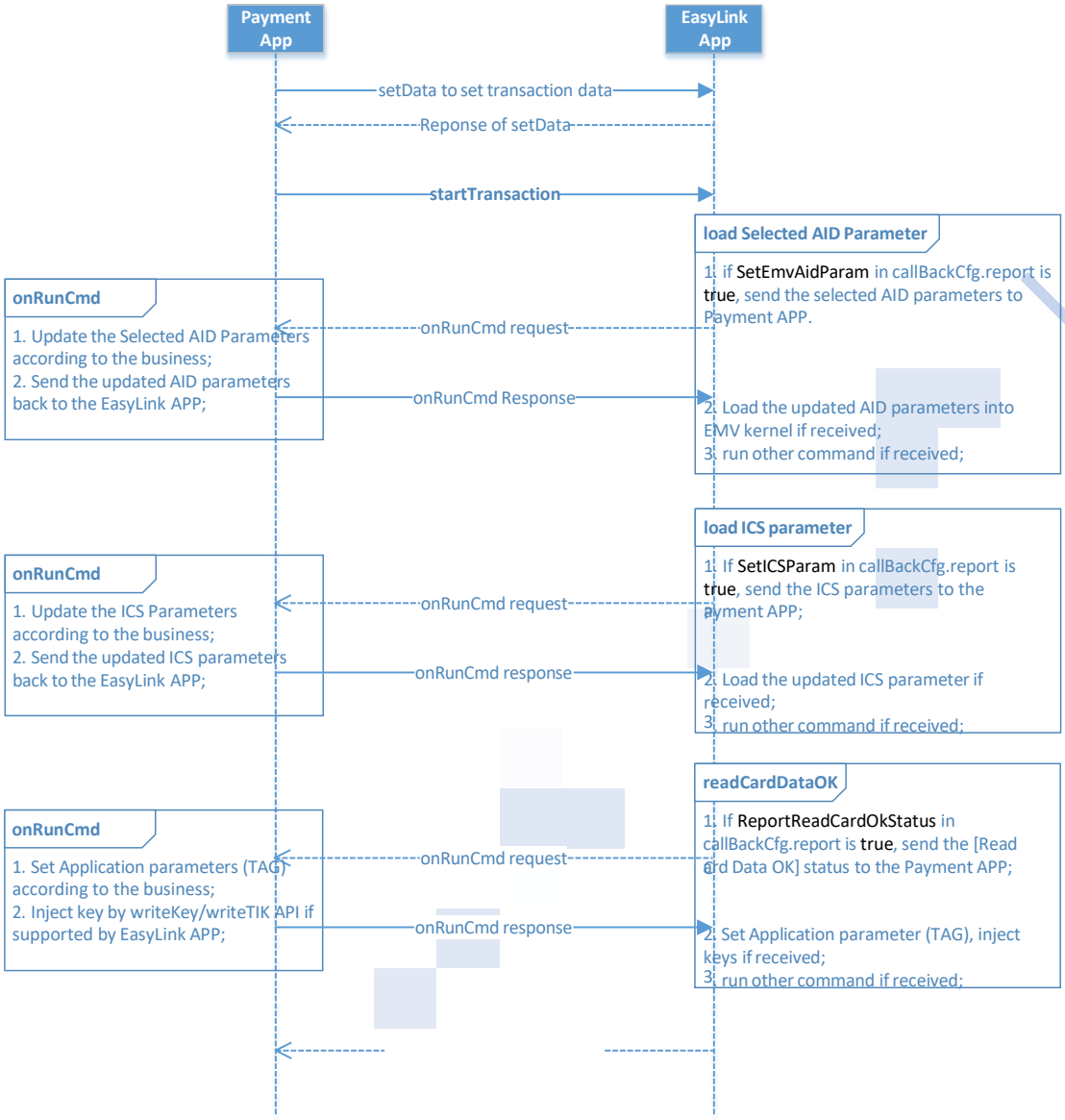
The definition of callBackCfg.report:

```
<Root version="000001" Name="CallBackConfig">
    <Item   CallBackName="SetParamByClient">false</Item>
    <Item   CallBackName="SetEmvAidParam">false</Item>
    <Item   CallBackName="SetICSParam">true</Item>
    <Item   CallBackName="ReportReadCardOkStatus"> true</Item>
</Root>
```

Description:

1. The value of each node can be **true** or **false**;
2. The Node [SetParamByClient] indicates whether need to accept to set parameter (TAG) from Master device during startTransaction;
3. The Node [SetEmvAidParam] indicates whether need to send the selected EMV AID parameters to the Payment APP (Master Device), and the Payment APP need to send the updated EMV AID parameters back to the EasyLink APP (Slave Device);
4. The Node [SetICSParam] indicates whether need to send the ICS parameters (see emv_param.emv) to the Payment APP(Master Device), and the Payment APP need to send the the updated ICS parameters back to the EasyLink APP(Slave Device);
5. The Node [ReportReadCardOkStatus] indicates whether need to send the “read card OK” status to the Payment APP(Master Device), and the Payment APP can set parameters(TAG), writeKey, writeTIK during this period.

Blow is the transaction flow for how does the EasyLink APP support multi-host during startTransaction:


Payment App
EasyLink App
setData to set transaction data
Reponse of setData
startTransaction
load Selected AID Parameter
1. if SetEmvAidParam in callBackCfg.report is true, send the selected AID parameters to Payment APP.
onRunCmd
1. Update the Selected AID Parameters according to the business;
2. Send the updated AID parameters back to the EasyLink APP;
onRunCmd request
onRunCmd Response
2. Load the updated AID parameters into EMV kernel if received;
3. run other command if received;
load ICS parameter
1. If SetICSParam in callBackCfg.report is true, send the ICS parameters to the payment APP;
onRunCmd
1. Update the ICS Parameters according to the business;
2. Send the updated ICS parameters back to the EasyLink APP;
onRunCmd request
onRunCmd response
2. Load the updated ICS parameter if received;
3. run other command if received;
readCardDataOK
1. If ReportReadCardOkStatus in callBackCfg.report is true, send the [Read Card Data OK] status to the Payment APP;
onRunCmd
1. Set Application parameters (TAG) according to the business;
2. Inject key by writeKey/writeTIK API if supported by EasyLink APP;
onRunCmd request
onRunCmd response
2. Set Application parameter (TAG), inject keys if received;
3. run other command if received;

Appendix 9 – Sensitive TAG list

TAG	Element	Attribute	Read Write	Description
0304	Track1 data	ans…79	R	Customized TAG Track1 data, Track1 needs 79 bytes.
0305	Track2 data	ans…37	R	Customized TAG Track2 data, Track2 needs 37 bytes.
0306	Track3 data	ans…107	R	Customized TAG Track3 data, Track3 needs 107 bytes.
031C	PAN data	Ans…19	R	Primary account number
5A	PAN data	Ans…19	R	EMV, CTLS TAG Application Primary Account Number
56	Track1 data	Ans…79	R	EMV, CTLS TAG Track1 data
57	Track2 data	Ans…37	R	EMV, CTLS TAG Track2 data

Reponse of startTransaction

Appendix 10 – Return code

Base return code

Return code	Value	Description
EL_RET_OK	0	success
EL_COMM_ERROR	-1	Communication error
ERR_ARG	-2	Arg err
ERR_PROTO_CONN	-100	Proto conn err
ERR_PROTO_SEND	-101	Proto send err
ERR_PROTO_RECV	-102	Proto recv err
ERR_PROTO_PACKAGE_TOO_LONG	-103	Proto package too long err
ERR_PROTO_NO_ENOUGH_DATA	-104	Proto no enough data err
ERR_PROTO_CHECKSUM	-105	Proto checksum err
ERR_PROTO_DATA_FORMAT	-106	Proto data format err
ERR_PROTO_NAKED	-107	Proto naked err
ERR_PROTO_SYNC	-108	Proto sync err
ERR_PROTO_NOT_CONFIRM	-109	Proto not confirm err
ERR_PROTO_CANCEL	-110	Proto cancel err
EL_COMM_RET_BASE	1000	Base return code for COMM module
EL_RKI_RET_BASE	1400	Base return code for RKI
EL_SML_RET_BASE	1600	Base return code for smartlanding module
EL_UI_RET_BASE	2000	Base return code for UI module

EL_SECURITY_RET_BASE	3000	Base return code for Security module
EL_WRITE_KEY_RET_BASE	3110	Base return code for Write Key module
EL_TRANS_RET_BASE	4000	Base return code for transction module
EL_PARAM_RET_BASE	5000	Base return code for parameter management module
EL_GENERAL_RET_BASE	6000	Base return code for common module
EL_FILEDOWNLOAD_RET_BASE	7000	Base return code for FileDownload module
EL_PICC_BASE	8000	Base return code for Picc module
EL_MSR_BASE	8200	Base return code for MSR module
EL_ICC_BASE	8400	Base return code for ICC module
EL_SDK_RET_BASE	9000	Base return code for SDK status
EL_SDK_PROTO_RET_BASE	9500	Base return code for procotol layer

COMM return code

Return code	Value	Description
EL_COMM_RET_BASE	1000	
EL_COMM_RET_CONNECTED	(EL_COMM_RET_BASE + 1)	Already connected
EL_COMM_RET_DISCONNECT_FAIL	(EL_COMM_RET_BASE + 2)	Disconnect fail
EL_COMM_RET_PARAM_FILE_NOT_EXIST	(EL_COMM_RET_BASE + 4)	Parameter file not exists(ui, emv, clss) **Note: IOS doesn' support this feature**
EL_COMM_RET_NOTCONNECTED	(EL_COMM_RET_BASE + 3)	Not connected
EL_COMM_RET_BUSY	(EL_COMM_RET_BASE + 5)	System is busy
EL_COMM_INPUT_DATA_OVERFLOW	(EL_COMM_RET_BASE + 6)	the size of input data from client side larger than the terminal buffer.
EL_COMM_RET_READER_TAMPER	(EL_COMM_RET_BASE + 7)	Reader tampered

UI return code

Return code	Value	Description
EL_UI_ERT_BASE	2000	
EL_UI_RET_INVALID_WIDGETNAME	(EL_UI_ERT_BASE + 1)	Invalid parameter
EL_UI_RET_TIME_OUT	(EL_UI_RET_BASE + 2)	Time out
EL_UI_RET_INVALID_PAGE	(EL_UI_ERT_BASE + 3)	invalid page
EL_UI_RET_PARSEUI_FAILED	(EL_UI_ERT_BASE + 4)	Parse UI failed
EL_UI_RET_VALUESIZEERROR	(EL_UI_ERT_BASE + 5)	Value size error
EL_UI_RET_INPUT_TYPE_ERRO	(EL_UI_ERT_BASE + 6)	Input type error
EL_UI_RET_INVALID_WIDGETVALUE	(EL_UI_ERT_BASE + 7)	Invalid widget value
EL_UI_RET_USER_CANCEL	(EL_UI_ERT_BASE + 8)	User canceled
EL_UI_RET_MENUITEMNUM_ERROR	(EL_UI_ERT_BASE + 9)	Menu item num error
EL_UI_RET_UNKOWN_ERROR	(EL_UI_ERT_BASE + 10)	Unknown error
EL_UI_RET_GETSIGNDATA_FAIL	(EL_UI_ERT_BASE + 11)	Get signature data filed
EL_UI_RET_CARD_NUMBER_NOT_MATCH	(EL_UI_ERT_BASE + 12)	Card number doesn't match

Security return code

Return code	Value	Description
EL_SECURITY_RET_BASE	3000	
EL_SECURITY_RET_NO_KEY	(EL_SECURITY_RET_BASE + 1)	Key does not exist
EL_SECURITY_RET_PARAM_INVALID	(EL_SECURITY_RET_BASE + 2)	Parameter error or invalid.
EL_SECURITY_RET_ENCRYPT_DATA_ERR	(EL_SECURITY_RET_BASE +3)	Encrypt data error
EL_SECURITY_RET_GET_PIN_BLOCK_ERR	(EL_SECURITY_RET_BASE + 4)	Get pin block error
EL_SECURITY_RET_NO_PIN_INPUT	(EL_SECURITY_RET_BASE + 5)	No input pin
EL_SECURITY_RET_INPUT_CANCEL	(EL_SECURITY_RET_BASE + 6)	User cancel
EL_SECURITY_RET_INPUT_TIMEOUT	(EL_SECURITY_RET_BASE + 7)	Input timeout
EL_SECURITY_RET_KEY_TYPE_ERR	(EL_SECURITY_RET_BASE + 8)	Key type error
EL_SECURITY_ERR_PED_WAIT_INTERVAL	(EL_SECURITY_RET_BASE + 9)	ped wait interval
EL_SECURITY_RET_SENSITIVE_DATA_CLEARD	(EL_SECURITY_RET_BASE + 10)	Sensitive data has been cleared **(Reserved)**

Transaction flow return code

Return code	Value	Description
EL_TRANS_RET_BASE	4000	
EL_TRANS_RET_ICC_RESET_ERR	(EL_TRANS_RET_BASE + 1)	ICC reset error
EL_TRANS_RET_READ_CARD_FAIL	(EL_TRANS_RET_BASE + 2)	Read card failed, this error code returns when: 1) read magnetic card failed; 2) if IC card init error; 3) IC card reset error; 4) IC card command error;
EL_TRANS_RET_CARD_BLOCKED	(EL_TRANS_RET_BASE + 3)	Card is blocked, this error code returns when: 1) IC card blocked; 2) EMV, Contactless EMV APP blocked;
EL_TRANS_RET_EMV_RSP_ERR	(EL_TRANS_RET_BASE + 4)	Status Code returned by IC card is not 9000
EL_TRANS_RET_EMV_APP_BLOCK	(EL_TRANS_RET_BASE + 5)	The application in IC card selected is blocked.
EL_TRANS_RET_EMV_NO_APP	(EL_TRANS_RET_BASE + 6)	There is no AID matched between IC card and terminal
EL_TRANS_RET_USER_CANCELED	(EL_TRANS_RET_BASE + 7)	User canceled.
EL_TRANS_RET_TIME_OUT	(EL_TRANS_RET_BASE + 8)	Timeout
EL_TRANS_RET_CARD_DATA_ERROR	(EL_TRANS_RET_BASE + 9)	Card data error.
EL_TRANS_RET_TRANS_NOT_ACCEPT	(EL_TRANS_RET_BASE + 10)	Transaction is not accepted, this code returns when:
EL_TRANS_RET_EMV_DENIAL	(EL_TRANS_RET_BASE + 11)	Transaction is denied

Return code	Value	Description
EL_TRANS_RET_EMV_KEY_EXP	(EL_TRANS_RET_BASE + 12)	Certification Authority Public Key is Expired
EL_TRANS_RET_EMV_NO_PINPAD	(EL_TRANS_RET_BASE + 13)	PIN enter is required, but PIN pad is not present or not working
EL_TRANS_RET_EMV_NO_PASSWORD	(EL_TRANS_RET_BASE + 14)	PIN enter is required, PIN pad is present, but there is no PIN entered
EL_TRANS_RET_EMV_SUM_ERR	(EL_TRANS_RET_BASE + 15)	Checksum of CAPK is err
EL_TRANS_RET_EMV_NOT_FOUND	(EL_TRANS_RET_BASE + 16)	Appointed Data Element can't be found
EL_TRANS_RET_EMV_NO_DATA	(EL_TRANS_RET_BASE + 17)	The length of the appointed Data Element is 0
EL_TRANS_RET_EMV_OVERFLOW	(EL_TRANS_RET_BASE + 18)	Memory is overflow
EL_TRANS_RET_NO_TRANS_LOG	(EL_TRANS_RET_BASE + 19)	There is no Transaction log
EL_TRANS_RET_RECORD_NOTEXIST	(EL_TRANS_RET_BASE + 20)	Appointed log is not existed
EL_TRANS_RET_LOGITEM_NOTEXIST	(EL_TRANS_RET_BASE + 21)	Appointed Label is not existed in current log record
EL_TRANS_RET_ICC_RSP_6985	(EL_TRANS_RET_BASE + 22)	Status Code returned by IC card for GPO is 6985
EL_TRANS_RET_CLSS_USE_CONTACT	(EL_TRANS_RET_BASE + 23)	Must use other interface for the transaction
EL_TRANS_RET_EMV_FILE_ERR	(EL_TRANS_RET_BASE + 24)	There is file err found
EL_TRANS_RET_CLSS_TERMINATE	(EL_TRANS_RET_BASE + 25)	Must terminate the transaction
EL_TRANS_RET_CLSS_FAILED	(EL_TRANS_RET_BASE + 26)	Contactless transaction is failed
EL_TRANS_RET_CLSS_DECLINE	(EL_TRANS_RET_BASE + 27)	Transaction should be declined
EL_TRANS_RET_CLSS_TRY_ANOTHER_CARD	(EL_TRANS_RET_BASE + 28)	Try another card (DPAS Only)
EL_TRANS_RET_PARAM_ERR	(EL_TRANS_RET_BASE + 30)	Parameter is err = EMV_PARAM_ERR
EL_TRANS_RET_CLSS_WAVE2_OVERSEA	(EL_TRANS_RET_BASE +31)	International transaction (for VISA AP PayWave Level2 IC card use)
EL_TRANS_RET_CLSS_WAVE2_TERMINATED	(EL_TRANS_RET_BASE + 32)	Wave2 DDA response TLV format err
EL_TRANS_RET_CLSS_WAVE2_US_CARD	(EL_TRANS_RET_BASE + 33)	US card (for VISA AP PayWave L2 IC card use)
EL_TRANS_RET_CLSS_WAVE3_INS_CARD	(EL_TRANS_RET_BASE + 34)	Need to use IC card for the transaction (for VISA PayWave IC card use)
EL_TRANS_RET_CLSS_RESELECT_APP	(EL_TRANS_RET_BASE + 35)	Select the next AID in candidate list
EL_TRANS_RET_CLSS_CARD_EXPIRED	(EL_TRANS_RET_BASE + 36)	IC card is expired
EL_TRANS_RET_EMV_NO_APP_PPSE_ERR	(EL_TRANS_RET_BASE + 37)	No application is supported (Select PPSE is err)
EL_TRANS_RET_CLSS_USE_VSDC	(EL_TRANS_RET_BASE + 38)	Switch to contactless PBOC

Return code	Value	Description
EL_TRANS_RET_CLSS_CVMDECLINE	(EL_TRANS_RET_BASE + 39)	CVM result in decline for AE
EL_TRANS_RET_CLSS_REFER_CONSUMER_DEVICE	(EL_TRANS_RET_BASE + 40)	Status Code returned by IC card is 6986, please see phone
EL_TRANS_RET_CLSS_LAST_CMD_ERR	(EL_TRANS_RET_BASE + 41)	The last read record command is err (qPBOC Only)
EL_TRANS_RET_CLSS_API_ORDER_ERR	(EL_TRANS_RET_BASE + 42)	APIs are called in wrong order. Please call Clss_GetDebugInfo_xxx to get err codes.
EL_TRANS_RET_TRANS_FAILED	(EL_TRANS_RET_BASE +43)	Transaction failed
EL_TRANS_RET_TRASN_DECLINED	(EL_TRANS_RET_BASE +44)	Transaction is declined
EL_TRANS_RET_NOT_SUPPORT	(EL_TRANS_RET_BASE +45)	Transaction is not supported
EL_TRANS_RET_EXPIRED	(EL_TRANS_RET_BASE +46)	Card expired
EL_TRANS_RET_LUHN	(EL_TRANS_RET_BASE + 47)	Card Luhn Error
EL_TRANS_RET_LUHN_NO_CARD_ERR	(EL_TRANS_RET_BASE + 48)	No card number for Luhn check

Parameter management return code

Return code	Value	Description
EL_PARAM_RET_BASE	5000	
EL_PARAM_RET_ERR_DATA	(EL_PARAM_RET_BASE + 1)	Input data error.
EL_PARAM_RET_INVALID_PARAM	(EL_PARAM_RET_BASE + 2)	Invalid parameter.
EL_PARAM_RET_PARTIAL_FAILED	(EL_PARAM_RET_BASE + 3)	Partial operation failed.
EL_PARAM_RET_ALL_FAILED	(EL_PARAM_RET_BASE + 4)	All operation failed.
EL_PARAM_RET_BUFFER_TOO_SMALL	(EL_PARAM_RET_BASE + 5)	The output buffer size is not enough.
EL_PARAM_RET_API_ORDER_ERR	(EL_PARAM_RET_BASE + 6)	Must call this function after StartTransaction step.
EL_PARAM_RET_ENCRYPT_SENSITIVE_DATA_FAILED	(EL_PARAM_RET_BASE + 7)	Encrypt sensitive data failed
EL_PARAM_RET_SET_CONFIG_TLV_CMD_ERROR	(EL_PARAM_RET_BASE + 8)	execute setDataCmd(config tlv data) failed during startTransaction
EL_PARAM_RET_SET_EMV_TLV_CMD_ERROR	(EL_PARAM_RET_BASE + 9)	execute setDataCmd(emv tlv data) failed during startTransaction
EL_PARAM_RET_UNSUPPORTED_CARD	(EL_PARAM_RET_BASE + 10)	Can not find current card bin in card bin range list

TMS Proxy return code

Return code	Value	Description
MPOS_STATUS_OK	0	Success
MPOS_STATUS_CMD_NOTSPT	-0xffff	Command not supported

Return code	Value	Description
MPOS_STATUS_NOTASKLIST	1	No task list
MPOS_STATUS_MONITOR_SAVE_ERR	5	Save monitor error
MPOS_STATUS_FONT_SAVE_ERR	6	Save font error
MPOS_STATUS_APP_SAVE_ERR	7	Save application error
MPOS_STATUS_PARA_SAVE_ERR	8	Save parameter file error

FileDownload return code

Return code	Value	Description
EL_FILEDOWNLOAD_RET_BASE	7000	
EL_FILEDOWNLOAD_RET_INVALID_PARAM	(EL_FILEDOWNLOAD_RET_BASE + 1)	Invalid parameter
EL_FILEDOWNLOAD_RET_PARAM_FILE_FAIL	(EL_FILEDOWNLOAD_RET_BASE + 2)	Download parameter file failed
EL_FILEDOWNLOAD_RET_FIRMWARE_FAIL	(EL_FILEDOWNLOAD_RET_BASE + 3)	Download firmware/app/font failed
EL_FILEDOWNLOAD_RET_FILE_OVERSIZE	(EL_FILEDOWNLOAD_RET_BASE + 4)	File oversize: Parameter file size: from 0K to 100K; Firmware/app file size: from 0K to 400K; Font file size: from 0K to 2048K;
EL_FILEDOWNLOAD_RET_PARSE_ERR	(EL_FILEDOWNLOAD_RET_BASE + 5)	Parse file failed

General return code

Return code	Value	Description
EL_RET_OK	0	
EL_GENERAL_RET_BASE	6000	Base general return code
EL_GENERAL_RET_TIMEOUT	(EL_GENERAL_RET_BASE + 1)	timeout
EL_GENERAL_RET_INVALID_PARAM	(EL_GENERAL_RET_BASE + 2)	invalid parameter
EL_GENERAL_RET_DEVICE_NOT_EXIST	(EL_GENERAL_RET_BASE + 3)	device does not exist
EL_GENERAL_RET_DEVICE_BUSY	(EL_GENERAL_RET_BASE + 4)	Device is busy
EL_GENERAL_RET_DEVICE_NOT_OPEN	(EL_GENERAL_RET_BASE + 5)	Device not open
EL_GENERAL_RET_USER_CANCELED	(EL_GENERAL_RET_BASE + 6)	User canceled
EL_GENERAL_RET_CMD_NOT_SUPPORT	(EL_GENERAL_RET_BASE + 7)	Command is not supported

WriteKey return code

Return code	Value	Description
EL_RET_OK	0	
EL_WRITE_KEY_BASE	3110	
EL_WRITE_KEY_NO_KEY	(EL_WRITE_KEY_BASE E + 1)	Key does not exist.
EL_WRITE_KEY_KEYIDX_ERR	(EL_WRITE_KEY_BASE E + 2)	Key index error, parameter index is not in the range.
EL_WRITE_KEY_DERIVE_ERR	(EL_WRITE_KEY_BASE E + 3)	When key is written, the source key level is lower than the destination level.
EL_WRITE_KEY_CHECK_KEY_FAIL	(EL_WRITE_KEY_BASE E + 4)	Key verification failed.
EL_WRITE_KEY_NO_PIN_INPUT	(EL_WRITE_KEY_BASE E + 5)	No PIN input
EL_WRITE_KEY_INPUT_CANCEL	(EL_WRITE_KEY_BASE E + 6)	Cancel to enter PIN.
EL_WRITE_KEY_WAIT_INTERVAL	(EL_WRITE_KEY_BASE E + 7)	Calling function interval is less than minimum interval time.
EL_WRITE_KEY_CHECK_MODE_ERR	(EL_WRITE_KEY_BASE E + 8)	KCV mode error, do not support.
EL_WRITE_KEY_NO_RIGHT_USE	(EL_WRITE_KEY_BASE E + 9)	Not allowed to use the key. When key label is not correct or source key type is bigger than destination key type, PED will return this code
EL_WRITE_KEY_KEY_TYPE_ERR	(EL_WRITE_KEY_BASE E + 10)	Key type error
EL_WRITE_KEY_EXPLEN_ERR	(EL_WRITE_KEY_BASE E + 11)	Expected PIN length string error
EL_WRITE_KEY_DSTKEY_IDX_ERR	(EL_WRITE_KEY_BASE E + 12)	Destination key index error
EL_WRITE_KEY_SRCKEY_IDX_ERR	(EL_WRITE_KEY_BASE E + 13)	Source key index error
EL_WRITE_KEY_LEN_ERR	(EL_WRITE_KEY_BASE E + 14)	Key length error
EL_WRITE_KEY_INPUT_TIMEOUT	(EL_WRITE_KEY_BASE E + 15)	PIN input timeout
EL_WRITE_KEY_NO_ICC	(EL_WRITE_KEY_BASE E + 16)	IC card does not exist
EL_WRITE_KEY_CC_NO_INIT	(EL_WRITE_KEY_BASE E + 17)	IC card is not initialized
EL_WRITE_KEY_GROUP_IDX_ERR	(EL_WRITE_KEY_BASE E + 18)	DUKPT index error
EL_WRITE_KEY_PARAM_PTR_NULL	(EL_WRITE_KEY_BASE E + 19)	Pointer parameter error
EL_WRITE_KEY_LOCKED	(EL_WRITE_KEY_BASE E + 20)	PED locked
EL_WRITE_KEY_ERROR	(EL_WRITE_KEY_BASE E + 21)	PED general error
EL_WRITE_KEY_NOMORE_BUF	(EL_WRITE_KEY_BASE E + 22)	No free buffer

Return code	Value	Description
EL_WRITE_KEY_NEED_ADMIN	(EL_WRITE_KEY_BASE E + 23)	Not administration
EL_WRITE_KEY_DUKPT_OVERFLOW	(EL_WRITE_KEY_BASE E + 24)	DUKPT overflow
EL_WRITE_KEY_KCV_CHECK_FAIL	(EL_WRITE_KEY_BASE E + 25)	KCV check error
EL_WRITE_KEY_SRCKEY_TYPE_ERR	(EL_WRITE_KEY_BASE E + 26)	Source key type error.
EL_WRITE_KEY_UNSPT_CMD	(EL_WRITE_KEY_BASE E + 27)	Command not support
EL_WRITE_KEY_COMM_ERR	(EL_WRITE_KEY_BASE E + 28)	Communication error
EL_WRITE_KEY_NO_UAPUK	(EL_WRITE_KEY_BASE E + 29)	No user authenticatio n public key
EL_WRITE_KEY_ADMIN_ERR	(EL_WRITE_KEY_BASE E + 30)	Administration error
EL_WRITE_KEY_DOWNLOAD_INACTIVE	(EL_WRITE_KEY_BASE E + 31)	PED download inactive
EL_WRITE_KEY_KCV_ODD_CHECK_FAIL	(EL_WRITE_KEY_BASE E + 32)	KCV parity check fail
EL_WRITE_KEY_PED_DATA_RW_FAIL	(EL_WRITE_KEY_BASE E + 33)	Read PED data fail
EL_WRITE_KEY_ICC_CMD_ERR	(EL_WRITE_KEY_BASE E +34)	ICC operation fail
EL_WRITE_KEY_INPUT_CLEAR	(EL_WRITE_KEY_BASE E + 39)	Pressing CLEAR to exit input
EL_WRITE_KEY_NO_FREE_FLASH	(EL_WRITE_KEY_BASE E + 43)	PED has no enough
EL_WRITE_KEY_DUKPT_NEED_INC_KSN	(EL_WRITE_KEY_BASE E + 44)	DUKPT KSN need to plus 1 first
EL_WRITE_KEY_KCV_MODE_ERR	(EL_WRITE_KEY_BASE E + 45)	KCV MODE error.
EL_WRITE_KEY_DUKPT_NO_KCV	(EL_WRITE_KEY_BASE E + 46)	NO KCV
EL_WRITE_KEY_PIN_BYPASS_BYFUNKEY	(EL_WRITE_KEY_BASE E + 47)	Press FN/ATM4 to cancel PIN inputting.
EL_WRITE_KEY_MAC_ERR	(EL_WRITE_KEY_BASE E + 48)	Data MAC check error.
EL_WRITE_KEY_CRC_ERR	(EL_WRITE_KEY_BASE E +49)	Data CRC check error.
EL_WRITE_KEY_PARAM_INVALID	(EL_WRITE_KEY_BASE E + 50)	Parameter error or invalid.

Picc return code

Return code	Value	Description
EL_RET_OK	0	Success
EL_PICC_BASE	8000	Base Picc return code
EL_PICC_PARAM_ERROR	(EL_PICC_BASE + 1)	Parameter error
EL_PICC_RF_MODULE_CLOSE	(EL_PICC_BASE + 2)	RF module close
EL_PICC_NO_SPECIFIC_CARD_IN_AREA	(EL_PICC_BASE + 3)	No specific card in sensing area
EL_PICC_TOO_MUCH_CARD_IN_AREA	(EL_PICC_BASE + 4)	Too much card in sensing area(communication conflict)
EL_PICC_RESP_PROTOCOL_ERROR	(EL_PICC_BASE + 5)	Protocol error(The data response from card breaches the agreement)

Return code	Value	Description
EL_PICC_CARD_NOT_ACTIVATED	(EL_PICC_BASE + 6)	Card not activated
EL_PICC_MULTI_CARD_CONFLICT	(EL_PICC_BASE + 7)	Multi-card conflict
EL_PICC_NO_RESPONSE_TIMEOUT	(EL_PICC_BASE + 8)	No response timeout
EL_PICC_PROTOCOL_ERROR	(EL_PICC_BASE + 9)	Protocol error
EL_PICC_COMM_TRANSMIT_ERROR	(EL_PICC_BASE + 10)	Communication transmission error
EL_PICC_M1_CARD_AUTH_FAIL	(EL_PICC_BASE + 11)	M1 Card authentication failure.
EL_PICC_SECTOR_NOT_CERTIFIED	(EL_PICC_BASE + 12)	Sector is not certified
EL_PICC_DATA_FORMAT_INCORRECT	(EL_PICC_BASE + 13)	The data format of value block is incorrect.
EL_PICC_CARD_STILL_IN_AREA	(EL_PICC_BASE + 14)	Card is still in sensing area.
EL_PICC_CARD_STATUS_ERROR	(EL_PICC_BASE + 15)	Card status error(If A/B card call M1 card interface, or M1 card call PiccIsoCommand interface)
EL_PICC_INTERFACE_CHIP_ERROR	(EL_PICC_BASE + 16)	Interface chip does not exist or abnormal.
EL_PICC_PAR_FLAG	(EL_PICC_BASE + 17)	Parity error.
EL_PICC_CRC_FLAG	(EL_PICC_BASE + 18)	CRC error.
EL_PICC_ECD_FLAG	(EL_PICC_BASE + 19)	Frame format error.
EL_PICC_EMD_FLAG	(EL_PICC_BASE + 20)	Interference
EL_PICC_PARAMFILE_FLAG	(EL_PICC_BASE + 21)	Configuration file does not exist.
EL_PICC_USER_CANCEL	(EL_PICC_BASE + 22)	Transaction is cancelled.
EL_PICC_CARRIER_OBTAIN_FLAG	(EL_PICC_BASE + 23)	No obtained carrier
EL_PICC_CONFIG_FLAG	(EL_PICC_BASE + 24)	Fail to configure register.
EL_PICC_RXLEN_EXCEED_FLAG	(EL_PICC_BASE + 25)	The returned data length exceeds the set receiving length
EL_PICC_NOT_IDLE_FLAG	(EL_PICC_BASE + 26)	Card is not in idle state.
EL_PICC__NOT_POLLING_FLAG	(EL_PICC_BASE + 27)	No polling
EL_PICC_NOT_WAKEUP_FLAG	(EL_PICC_BASE + 28)	Card does not wakeup.
EL_PICC_NOT_SUPPORT	(EL_PICC_BASE + 29)	Chip is unsupported.
EL_PICC_BATTERY_VOLTAGE_TOO_LOW	(EL_PICC_BASE + 30)	Battery voltage is too low.
EL_PICC_CHIP_ERROR	(EL_PICC_BASE + 31)	PICC card error.

Mag return code

Return code	Value	Description
EL_RET_OK	0	Success
EL_MSR_BASE	8200	Base MSR return code
EL_MSR_RET_FAILED	(EL_MSR_BASE + 1)	Fail to operate MSR
EL_MSR_RET_HEADE	(EL_MSR_BASE + 2)	Head mark is not found.
EL_MSR_RET_END	(EL_MSR_BASE + 3)	End mark is not found.
EL_MSR_RET_LRC	(EL_MSR_BASE + 4)	LRC check error.

Return code	Value	Description
EL_MSR_RET_PAR	(EL_MSR_BASE + 5)	Check error of a certain bit of MSR.
EL_MSR_RET_NOT_SWIPED	(EL_MSR_BASE + 6)	No card is swiped.
EL_MSR_RET_NO_DATA	(EL_MSR_BASE + 7)	No data in Mag card.
EL_MSR_END_RET_ZERO	(EL_MSR_BASE + 8)	Magnetic stripe card format error.
EL_MSR_PED_RET_DECRYPT	(EL_MSR_BASE + 9)	PED decryption failed.
EL_MSR_RET_NO_TRACK	(EL_MSR_BASE + 10)	The corresponding magnetic track in

Icc return code

Return code	Value	Description
EL_RET_OK	0	Success
EL_ICC_BASE	8400	Base icc return code
EL_ICC_RET_SCI_HW_NOCARD	(EL_ICC_BASE + 1)	No card.
EL_ICC_RET_SCI_HW_STEP	(EL_ICC_BASE + 2)	No initialization when exchanging data; no power when warm reseting.
EL_ICC_RET_SCI_HW_PARITY	(EL_ICC_BASE + 3)	Parity check error.
EL_ICC_RET_SCI_HW_TIMEOUT	(EL_ICC_BASE + 4)	Timeout.
EL_ICC_RET_SCI_TCK	(EL_ICC_BASE + 5)	TCK error.
EL_ICC_RET_SCI_ATR_TS	(EL_ICC_BASE + 6)	ATR TS error.
EL_ICC_RET_SCI_ATR_TA1	(EL_ICC_BASE + 7)	ATR TA1 error.
EL_ICC_RET_SCI_ATR_TD1	(EL_ICC_BASE + 8)	ATR TD1 error.
EL_ICC_RET_SCI_ATR_TA2	(EL_ICC_BASE + 9)	ATR TA2 error.
EL_ICC_RET_SCI_ATR_TB1	(EL_ICC_BASE + 10)	ATR TB1 error.
EL_ICC_RET_SCI_ATR_TB2	(EL_ICC_BASE + 11)	ATR TB2 error.
EL_ICC_RET_SCI_ATR_TC2	(EL_ICC_BASE + 12)	ATR TC2 error.
EL_ICC_RET_SCI_ATR_TD2	(EL_ICC_BASE + 13)	ATR TD2 error.
EL_ICC_RET_SCI_ATR_TA3	(EL_ICC_BASE + 14)	ATR TA3 error.
EL_ICC_RET_SCI_ATR_TB3	(EL_ICC_BASE + 15)	ATR TB3 error.
EL_ICC_RET_SCI_ATR_TC3	(EL_ICC_BASE + 16)	ATR TC3 error.
EL_ICC_RET_SCI_T_ORDER	(EL_ICC_BASE + 17)	Protocol is not T0 or T1.
EL_ICC_RET_SCI_PPS_PPSS	(EL_ICC_BASE + 18)	PPSS error in PPS
EL_ICC_RET_SCI_PPS_PPS0	(EL_ICC_BASE + 19)	PPS0 error in PPS.
EL_ICC_RET_SCI_PPS_PCK	(EL_ICC_BASE + 20)	ATRPCK error in PPS.
EL_ICC_RET_SCI_T0_PARAM	(EL_ICC_BASE + 21)	Transmitted data is too long in T0.
EL_ICC_RET_SCI_T0_REPEAT	(EL_ICC_BASE + 22)	Too many character repetitions in T0.
EL_ICC_RET_SCI_T0_PROB	(EL_ICC_BASE + 23)	Procedure byte error in T0.
EL_ICC_RET_SCI_T1_PARAM	(EL_ICC_BASE + 24)	Transmitteddata is too long in

Return code	Value	Description
		T1. ERR_SCI_T1_BWT -2851 BWT
EL_ICC_RET_SCI_T1_BWT	(EL_ICC_BASE + 25)	BWT is oversize in T1.
EL_ICC_RET_SCI_T1_CWT	(EL_ICC_BASE +26)	CWT is oversize in T1.
EL_ICC_RET_SCI_T1_BREP	(EL_ICC_BASE + 27)	Too many block repetitions in T1.
EL_ICC_RET_SCI_T1_LRC	(EL_ICC_BASE + 28)	LRC error in T1.
EL_ICC_RET_SCI_T1_NAD	(EL_ICC_BASE + 29)	NAD error in T1.
EL_ICC_RET_SCI_T1_LEN	(EL_ICC_BASE + 30)	LEN error in T1.
EL_ICC_RET_SCI_T1_PCB	(EL_ICC_BASE + 31)	PCB error in T1.
EL_ICC_RET_SCI_T1_SRC	(EL_ICC_BASE + 32)	SRC error in T1.
EL_ICC_RET_SCI_T1_SRL	(EL_ICC_BASE + 33)	SRL error in T1.
EL_ICC_RET_SCI_T1_SRA	(EL_ICC_BASE + 34)	SRA error in T1.
EL_ICC_RET_SCI_PARAM	(EL_ICC_BASE +35)	Invalid parameter.
EL_ICC_RET_SCI_OTHER_ERROR	(EL_ICC_BASE +36)	Other error
EL_ICC_RET_SCI_CHANNEL	(EL_ICC_BASE +37)	Card channel error.
EL_ICC_RET_SCI_DATA_LEN	(EL_ICC_BASE +38)	Data length error.
EL_ICC_RET_SCI_T1_ABORT	(EL_ICC_BASE +39)	Abort comm error in T1.
EL_ICC_RET_SCI_T1_EDC	(EL_ICC_BASE +40)	EDC error in T1.
EL_ICC_RET_SCI_T1_SYNC	(EL_ICC_BASE +41)	SYNC error in T1.
EL_ICC_RET_SCI_T1_BG	(EL_ICC_BASE +42)	EG error in T1.
EL_ICC_RET_SCI_T1_EG	(EL_ICC_BASE +43)	BG error in T1.
EL_ICC_RET_SCI_T1_IFSC	(EL_ICC_BASE +44)	IFSC error in T1.
EL_ICC_RET_SCI_T1_IFSD	(EL_ICC_BASE +45)	IFSD error in T1.
EL_ICC_RET_SCI_CHAR_GROUP_INVALID	(EL_ICC_BASE +46)	Character group invalid.
EL_ICC_RET_SCI_ATR_TC1	(EL_ICC_BASE +47)	ATR TC1 error.
EL_ICC_RET_SCI_ATR_DATA_LEN	(EL_ICC_BASE +48)	ATR data length error.
EL_ICC_RET_SCI_T0_TIMEOUT	(EL_ICC_BASE +49)	Timeout in T0.
EL_ICC_RET_SCI_T0_RESENT	(EL_ICC_BASE +50)	Re-send error in T0.
EL_ICC_RET_SCI_T0_RECEIVE	(EL_ICC_BASE +51)	Re-receive error in T0.
EL_ICC_RET_SCI_T0_STATUS_BYTE_INVALID	(EL_ICC_BASE +52)	Status byte error.
EL_ICC_RET_SCI_RE_CHAR_GROUP	(EL_ICC_BASE +53)	Re-send/receive character group error.
EL_ICC_RET_SCI_COMM_ERROR	(EL_ICC_BASE +54)	Card communication error.

SDK return code

Return code	Value	Description
EL_SDK_RET_BASE	9000	Base return code for SDK status
EL_SDK_RET_PARAM_INVALID	(EL_SDK_RET_BASE) + 1	invalid param
EL_SDK_RET_COMM_CONNECT_ERR	(EL_SDK_RET_BASE) + 2	comm connect err

Return code	Value	Description
EL_SDK_RET_COMM_DISCONNECT_ERR	(EL_SDK_RET_BASE) + 3	comm disconnect err
EL_SDK_RET_COMM_DISCONNECTED	(EL_SDK_RET_BASE) + 4	comm disconnected
EL_SDK_RET_COMM_SEND_ERR	(EL_SDK_RET_BASE) + 5	comm sent err
EL_SDK_RET_COMM_RECV_ERR	(EL_SDK_RET_BASE) + 6	comm recv err
EL_SDK_RET_FILE_NOT_EXIST	(EL_SDK_RET_BASE) + 7	file or file node not exist
EL_SDK_RET_GET_DATA_FAIL	(EL_SDK_RET_BASE) + 8	Get data failed
EL_SDK_RET_BT_NOT_ENABLED	(EL_SDK_RET_BASE) + 9	Bluetooth is not enable
EL_SDK_RET_ENC_SESSION_KEY_DEC_ERR	(EL_SDK_RET_BASE) + 10	Encrypted Session Key dec err

SDK PROTO return code

Return code	Value	Description
EL_SDK_PROTO_RET_BASE	9500	Base return code for SDK protocol
EL_SDK_RET_PROTO_GENERAL_ERR	(EL_SDK_PROTO_RET_BASE) + 1	general error
EL_SDK_RET_PROTO_ARG_ERR	(EL_SDK_PROTO_RET_BASE) + 2	argument error
EL_SDK_RET_PROTO_PACKET_TOO_LONG	(EL_SDK_PROTO_RET_BASE) + 3	packet too long
EL_SDK_RET_PROTO_NO_ENOUGH_DATA	(EL_SDK_PROTO_RET_BASE) + 4	receive data not enough
EL_SDK_RET_PROTO_DATA_FORMAT	(EL_SDK_PROTO_RET_BASE) + 5	data format error
EL_SDK_RET_PROTO_TIMEOUT	(EL_SDK_PROTO_RET_BASE) + 6	timeout

RKI return code

Return code	Value	Description
EL_RKI_RET_BASE	1400	Base return code for RKI
EL_RKI_RET_BIND_PHASE_ONE	(EL_RKI_RET_BASE) + 1	RKI binding phase one
EL_RKI_RET_BIND_PHASE_TWO	(EL_RKI_RET_BASE) + 2	RKI binding phase two
EL_RKI_RET_REBIND_PHASE_ONE	(EL_RKI_RET_BASE) + 3	RKI rebinding phase one
EL_RKI_RET_REBIND_PHASE_TWO	(EL_RKI_RET_BASE) + 4	RKI rebinding phase two
EL_RKI_RET_DOWNLOAD_PHASE_ONE	(EL_RKI_RET_BASE) + 5	RKI download phase one
EL_RKI_RET_DOWNLOAD_PHASE_TWO	(EL_RKI_RET_BASE) + 6	RKI download phase two
EL_RKI_RET_PARAM_NULL_ERR	(EL_RKI_RET_BASE) + 7	RKI parameter null error
EL_RKI_RET_INIT_ERR	(EL_RKI_RET_BASE) + 8	RKI init error
EL_RKI_RET_COMM_ERR	(EL_RKI_RET_BASE) + 9	RKI communication error
EL_RKI_RET_CONSTRUCT_TLV_ERR	(EL_RKI_RET_BASE) + 10	Construct tlv error
EL_RKI_RET_GET_DEVICE_INFO_ERR	(EL_RKI_RET_BASE) + 11	Get device info error
EL_RKI_RET_GET_DA_CERT_ERR	(EL_RKI_RET_BASE) + 12	Read PUK error
EL_RKI_RET_GET_DE_CERT_ERR	(EL_RKI_RET_BASE) + 13	Read PUK error
EL_RKI_RET_GET_RKI_INFO_ERR	(EL_RKI_RET_BASE) + 14	Get RKI information error

Return code	Value	Description
EL_RKI_RET_PARSE_TLV_ERR	(EL_RKI_RET_BASE) + 15	Communication error
EL_RKI_RET_SERVICE_REJECT	(EL_RKI_RET_BASE) + 16	System reject
EL_RKI_RET_SERVER_CERT_EXPIRE	(EL_RKI_RET_BASE) + 17	Server certifications expire
EL_RKI_RET_SERVER_CLOSE_ERR	(EL_RKI_RET_BASE) + 18	Close Server Err
EL_RKI_RET_GEN_DE_AUTH_DATA_ERR	(EL_RKI_RET_BASE) + 19	Term Auth Err
EL_RKI_RET_GET_AUTH_DATA_ERR	(EL_RKI_RET_BASE) + 20	Term Auth Err
EL_RKI_RET_CERT_INFO_ERR	(EL_RKI_RET_BASE) + 21	Read Cert Err
EL_RKI_RET_WRITE_AUTH_KEY_ERR	(EL_RKI_RET_BASE) + 22	Server Cert Err
EL_RKI_RET_DE_AUTH_TOKEN_ERR	(EL_RKI_RET_BASE) + 23	Device Token Err
EL_RKI_RET_WRITE_KMS_ID_ERR	(EL_RKI_RET_BASE) + 24	Term Status Err
EL_RKI_RET_VERIFY_PUK_ERR	(EL_RKI_RET_BASE) + 25	Server Cert Err
EL_RKI_RET_INJECT_KEY_ERR	(EL_RKI_RET_BASE) + 26	Inject Key Err
EL_RKI_RET_PINPAD_PROTOCOL_ERR	(EL_RKI_RET_BASE) + 27	PED protocol Err
EL_RKI_RET_PINPAD_INIT_ERR	(EL_RKI_RET_BASE) + 28	PED init Err
EL_RKI_RET_UNBIND_WRITE_KEY_LIST_ERR	(EL_RKI_RET_BASE) + 29	Key List Err
EL_RKI_RET_UNBIND_PHASE_ONE	(EL_RKI_RET_BASE) + 30	RKI unbind phase one
EL_RKI_RET_UNBIND_PHASE_TWO	(EL_RKI_RET_BASE) + 31	RKI unbind phase two
EL_RKI_RET_INJECT_OFFLINE_KEY	(EL_RKI_RET_BASE) + 32	RKI inject offline key
EL_RKI_RET_OFFLINE_KEY_ERR	(EL_RKI_RET_BASE) + 33	Offline key error
EL_RKI_RET_OFFLINE_KEY_TOO_LONG	(EL_RKI_RET_BASE) + 34	Offline key too long
EL_RKI_RET_END	(EL_RKI_RET_BASE) + 99	System Err
EL_RKI_SERVER_RET_COMM_TIMEOUT	303	Comm connect error
EL_RKI_SERVER_RET_COMM_NET_ERR	305	Comm net error
EL_RKI_SERVER_RET_COMM_DISCONNECTED	307	Comm disconnected
EL_RKI_SERVER_RET_COMM_NOT_CONNECT	308	Comm net not connect
EL_RKI_SERVER_RET_COMM_SEND_ERR	309	Comm sent error
EL_RKI_SERVER_RET_COMM_RECV_ERR	310	Comm receive error
EL_RKI_SERVER_RET_PARAM_INVALID	602	Invalid param
EL_RKI_SERVER_RET_COMM_CONNECT_ERR	1201	Comm connect error
EL_RKI_SERVER_RET_SSL_CERT_ERR	1202	Invalid certifications

Smartlanding return code

Return code	Value	Description
EL_SML_RET_BASE	1600	Base return code for smartlanding module

Return code	Value	Description
EL_SML_DB_SAVE_ERROR	(EL_SML_RET_BASE) + 1	Database save error
EL_SML_DB_OPEN_ERROR	(EL_SML_RET_BASE) + 2	Database open error
EL_SML_COMM_NO_CERT	(EL_SML_RET_BASE) + 3	Comm no cert.
EL_SML_COMM_DNS_ERROR	(EL_SML_RET_BASE) + 4	Comm DNS error.
EL_SML_COMM_SEND_ERR	(EL_SML_RET_BASE) + 5	Comm send error.
EL_SML_COMM_RECV_ERR	(EL_SML_RET_BASE) + 6	Comm receive error.
EL_SML_COMM_NEW_SSL_CTX_ERR	(EL_SML_RET_BASE) + 7	Comm error in creating new ssl context.
EL_SML_COMM_CREATE_SOCKET_ERR	(EL_SML_RET_BASE) + 8	Comm error in creating socket
EL_SML_COMM_SET_SOCKET_OPT_ERR	(EL_SML_RET_BASE) + 9	Comm error in setting socket opt
EL_SML_COMM_CONNECT_ERR	(EL_SML_RET_BASE) + 10	Comm connect error.
EL_SML_PACK_PARAM_ERR	(EL_SML_RET_BASE) + 11	Pack parameters error
EL_SML_PACK_TYPE_NOT_SUPPORT	(EL_SML_RET_BASE) + 12	Pack type does not support
EL_SML_UNPACK_FORMAT	(EL_SML_RET_BASE) + 13	Format error after unpack.
EL_SML_UNPACK_LRC	(EL_SML_RET_BASE) + 14	Unpack LRC error.
EL_SML_DEVICE_OPEN_ERR	(EL_SML_RET_BASE) + 15	Device open error
EL_SML_DEVICE_CONFIG_ERR	(EL_SML_RET_BASE) + 16	Device config error
EL_SML_COMM_PARAM_ERR	(EL_SML_RET_BASE) + 17	Communication parameters error
EL_SML_NOT_SUPPORT_RKI_COM	(EL_SML_RET_BASE) + 18	RKI comm param not support.
EL_SML_EVN_CHECK_ERR	(EL_SML_RET_BASE) + 19	Environment check error
EL_SML_EVN_CLEAR_TASK_ERR	(EL_SML_RET_BASE) + 20	Clear task error
EL_SML_SERVER_ERR	(EL_SML_RET_BASE) + 21	Server error
EL_SML_DOWNLOAD_ERR	(EL_SML_RET_BASE) + 22	Download error
EL_SML_DOWNLOAD_MD5_ERR	(EL_SML_RET_BASE) + 23	Download MD5 error
EL_SML_RET_COMM_HOST_ERROR	(EL_SML_RET_BASE) + 51	Fail to connect with Host URL.
EL_SML_RET_SYNC_ERROR	(EL_SML_RET_BASE) + 52	Sync task error
EL_SML_RET_PROCESS_ERROR	(EL_SML_RET_BASE) + 53	Process task error
EL_SML_RET_APP_INSTALL_ERROR	(EL_SML_RET_BASE) + 54	App install error
EL_SML_RET_RKI_DOWNLOAD_ERROR	(EL_SML_RET_BASE) + 55	RKI download error
EL_SML_RET_INVALID_DOWNLOAD_FILE	(EL_SML_RET_BASE) + 56	Invalid download file
EL_SML_RET_TERMINAL_NOT_REGISTER	(EL_SML_RET_BASE) + 57	Terminal not register, need TID.
EL_SML_USER_CANCEL	(EL_SML_RET_BASE) + 99	Users cancel

Appendix 11 – Track 2 data format

Card Type	Track 2 (or Track2 equivalent data) format	Format	TAG
Magnetic Card	Contains the data elements of track2 according to ISO/IEC 7813, excluding start sentinel, end sentinel, and Longitudinal Redundancy Check (LRC), as follows: PAN - Primary Account Number, up to 19 digits FS - Field Separator ('=') Expiration Date - YYMM Service Code – 3 digits Discretionary data – defined by individual payment systems	ASCII	0305
EMV Chip Card	Contains the data elements of track2 according to ISO/IEC 7813, excluding start sentinel, end sentinel, and Longitudinal Redundancy Check (LRC), as follows: PAN - Primary Account Number, n, var. up to 19 FS - Field Separator (Hex 'D'), b Expiration Date – YYMM, n4 Service Code – n3 Discretionary data - defined by individual payment systems, n, var. Pad with one Hex 'F' if needed to ensure whole bytes	BCD	57
EMV Chip Card	Contains the data elements of track2 according to ISO/IEC 7813, excluding start sentinel, end sentinel, and Longitudinal Redundancy Check (LRC), as follows: PAN - Primary Account Number, up to 19 digits FS - Field Separator ('=') Expiration Date - YYMM Service Code – 3 digits Discretionary data - defined by individual payment systems	ASCII	0305

EMV Contactless Card: Visa PayWave; MasterCard Contactless; ExpressPay; DPAS; JSpeedy; Rupay Contactless; qPBOC;	Contains the data elements of track2 according to ISO/IEC 7813, excluding start sentinel, end sentinel, and Longitudinal Redundancy Check (LRC), as follows: PAN - Primary Account Number, n, var. up to 19 FS - Field Separator (Hex 'D'), b Expiration Date – YYMM, n4 Service Code – n3 Discretionary data - defined by individual payment systems, n, var. Pad with one Hex 'F' if needed to ensure whole bytes	BCD	57
EMV Contactless Card: Visa PayWave (MSD mode)	Contains the data elements of track2 according to ISO/IEC 7813, excluding start sentinel, end sentinel, and Longitudinal Redundancy Check (LRC), as follows: SS – Start sentinel, 1 character: ';' PAN - Primary Account Number, up to 19 digits FS - Field Separator ('=') Expiration Date - YYMM Service Code – 3 digits PIN Verification Field (optional): 5 digits Discretionary data - defined by individual payment systems Pad with one Hex 'F' if needed to ensure whole bytes ES – End sentinel, 1 character: '?' LRC	ASCII	0305
EMV Contactless Card: MasterCard Contactless(MSD mode); JSpeedy;	Contains the data elements of track2 according to ISO/IEC 7813, excluding start sentinel, end sentinel, and Longitudinal Redundancy Check (LRC), as follows: PAN - Primary Account Number, up to 19 digits FS - Field Separator ('=')	ASCII	0305

Rupay Contactless; qPBOC;	Expiration Date - YYMM Service Code – 3 digits Discretionary data - defined by individual payment systems Pad with one Hex 'F' if needed to ensure whole bytes		
EMV Contactless Card: ExpressPay(MSD mode)	Contains the data elements of track2 according to ISO/IEC 7813, excluding start sentinel, end sentinel, and Longitudinal Redundancy Check (LRC), as follows: SS – Start sentinel, 1 character: ';' PAN - Primary Account Number, up to 19 digits FS - Field Separator ('=')s's Expiration Date - YYMM Service Code – 3 digits Discretionary data - defined by individual payment systems ES – End sentinel, 1 character: '?'	ASCII	0305
EMV Contactless Card: DPAS(MSD mode)	Contains the data elements of track2 according to ISO/IEC 7813, excluding start sentinel, end sentinel, and Longitudinal Redundancy Check (LRC), as follows: SS – Start sentinel, 1 character: ';' PAN - Primary Account Number, up to 19 digits FS - Field Separator ('=') Expiration Date - YYMM Service Code – 3 digits Discretionary data - defined by individual payment systems ES – End sentinel, 1 character: '?' LRC	ASCII	0305

Appendix 12 – Reported data format

The interfaces that need to display the message or have interaction with cardholder may need to report the data to the Android/iOS/Windows Payment Application.
The interface of EasyLink report data according to its processing status.

All the reported data are in JSON format.

report data to smart device	response data from smart device	description	related API

{
"currentStatus":"searchMode",
"prompts":"PLS swipe card",
"jsonData":{
"curSearchMode":7
}
}
	N/A	1) The status “curSearchMode” is the current mode for searching card, valid value:
		0x01-MSR: swipe card
		0x02-ICC: insert card
		0x04-PICC: tap card
		0x08-Manual: Input card by manual
		other<=0x0F: combination of mode, like
		0x01 | 0x02: MSR swipe and ICC insert
		0x02 | 0x04: ICC insert and PICC tap
		0x01 | 0x02 | 0x04: MSR swipe, ICC insert, and PICC tap.
		16-Fallback: fallback to swipe card
		
		2) The “prompts” is the prompt message to displayed on the screen;
			startTransaction
			completeTransaction

{
"currentStatus":"readCardStatus",
prompts":"Processing...,PLS Not Remove Card",
"jsonData":{
"status":0
	N/A	1) The status “readCardStatus” indicates that EasyLink Application detect card success and trying to read card.	startTransaction
			completeTransaction

"detectType":0 } }		2) The “prompts” is the prompt message to displayed on the screen; 3) The “status” indicates the status of card processing, valid value: 0-Not Ready 1-Idle 2-Ready to Read 3-Processing 4-Card Read Successfully 5-Processing Error(Contactless error) 6-Processing Error(Conditions for use of contactless not satisfied) 7-Processing Error(Contactless collision detected) 8-Processing Error(Card not removed from reader) 9-Detected Card Successfully 10-Card Removed 11-Processing after GPO. 4) The “detectType” indicates the current card detecting type, valid value: 0-contact 1-contactless 2-Magnetic 3-Manual	

{
"currentStatus":"processingStatus",
"prompts":"PLS Remove Card",
"jsonData":{
 	
}
	N/A	1) The “processingStatus” indicate card processing status.
		
		2) The “prompts” is the prompt message to displayed on the screen, like, “PLS Remove Card”,
			startTransaction
			completeTransaction

}			

EMV Application Selection:
(1)
{
"currentStatus":"selectApp",
"prompts":"PLS Select App",
"jsonData":{
"timeout":30,//unit: second
"appList":[
{ "appLabel":"App1"
"aid":"AID1"},
{ "appLabel":"App2"
"aid":"AID2" },
{ "appLabel":"App3"
"aid":"AID3" }
]
}
}
 	
(2)
{
"currNum":1 // current report number, from 0 to total-1
"totalNum":5 // total report numbers
"currentStatus":"selectApp",
"prompts":"PLS Select App",
"jsonData":{
"timeout":30,//unit: second
"appList":{
appPreName:"name1",
appLabel:"app1",
issDiscrData:"issDiscrData1",
aid:"AID1",
aidLen:4,
priority:1,
appName:"appName1"
}
}
	(1) Return selected app list:
	{
	"currentStatus":"selectApp", "jsonData":{
	"status":"SUCCESS"
	"appList":[
	{ "appLabel":"App2"
	" appIndex ":1},
	{ "appLabel":"App3"
	" appIndex ":2 }
	]
	}
	}
	Note:
	appIndex valid value:
	0 - (appList.size-1)
	
	(2) User cancel:
	{
	"currentStatus":"selectApp",
	"jsonData":{
	"status":"CANCEL"
	}
	}
	
	(3) Select timeout:
	{
	"currentStatus":"selectApp",
	"jsonData":{
	"status":"TIMEOUT"
	}
	}
	
	(4) No app list:
	""
	Note: return "" to the easylink App;
		Reported data:
		1) The "selectApp" indicate the EMV application selection.
		
		2) The "prompts" is the prompt message to display on the screen.
		
		3) The "timeout" is the timeout for application selection, unit: second.
		
		4) The "appList" is the candidate application list for EMV application selection.
		
		Reponse data:
		1) The "selectApp" indicate the EMV application selection.
		
		2) The "status" indicate the processing result of application selection, valid value:
		"NO_APP"
		"SUCCESS"
		"USER CANCEL"
		"TIMEOUT"
		"NOT_ACCEPT"
		Note:
		a. If the POS terminal doesn't have screen, then the number of "appList" in response shall be only 1, which is selected appLabel and appIndex.
		b. if the POS terminal has screen, then the number of "appList" in response can be
			startTransaction

}		more than 1, and terminal provide those appList for customer to select. c. if number is 1, then terminal don't need display select " appLabel" menu, ans use this as selected appLable and appIndex ; 3) The "appLabel" indicate the selected application.	

Language Selection:
{
"currentStatus":"selectLanguage",
"prompts":"SELECT LANGUAGE"
"jsonData":{
"timeout":30,//unit: second
"languages":"enfr"
` "termLanguages":"en"
}
}
	(1) Return selected app list:
	{
	"currentStatus":" selectLanguage",
	"jsonData":{
	"status":"SUCCESS"
	"language":"en"
	}
	}
	Note:
	language valid value:
	should be the 2 characters in" termLanguages", such as "en", "fr" or current support language abbreviation (ISO 639 - 1).
	
	(2) User cancel:
	{
	"currentStatus":" selectLanguage",
	"jsonData":{
	"status":"CANCEL"
	}
	}
	
	(3) Select timeout:
	{
	"currentStatus":" selectLanguage",
	"jsonData":{
		Reported data:
		1) The "selectLanguage" indicate the Language selection.
		
		2) The "prompts" is the prompt message to display on the screen.
		
		3) The "timeout" is the timeout for language selection, unit: second
		
		4) The "languages" is the language list from card.
		
		5) The "termLanguages" is the language list supported by slaver terminal.
		
		Reponse data:
		The "language" is selected language which want slaver terminal displayed.
		2) The "status" indicates the processing result of language selection, valid value:
		"SUCCESS"
		"USER CANCEL"	

	"status":"TIMEOUT" } }	"TIMEOUT" **Note：** The language abbreviation in "languages", "termLanguage" should follow the standard of ISO 963-1.	

ENTER PIN:
(1)
{
"currentStatus":"enterPin",
"prompts":"PLS Enter PIN"
"jsonData":{
"pinStatus":"INIT"
"pinLen":0
}
}
 	
(2)
{
"currentStatus":"enterPin",
"prompts":" PLS Enter PIN"
"jsonData":{
"pinStatus":"INPUT"
"pinLen":3
}
}
(3)
{
"currentStatus":"enterPin",
"prompts":"PIN Error, Retry"
"jsonData":{
"pinStatus":"RETRY"
"pinLen":0
}
}
	1. Success:
	{
	"currentStatus":" enterPin ",
	"jsonData":{
	"status ":"Success"
	}
	}
	
	2. User cancel
	{
	"currentStatus":" enterPin ",
	"jsonData":{
	"status ":"CANCEL"
	}
	}
		1) The status "enterPin" indicate that the POS terminal request to ENTER PIN;
		
		2) The "prompts" is the prompt message to display on the screen.
		
		3) The "pinStatus" indicate the status of ENTER PIN, valid value:
		"INIT"
		"INPUT"
		"CANCEL"
		"CLEAR"
		"ENTER"
		"RETRY"
		"LAST"
		Note: "ENTER" means user press ENTER key to bypass the PIN entry;
		
		4) The "pinLen" indicate current PIN length that user input.
		
		Reponse data:
		Enable cancel Pin input from Master device.
			getPinBlock
			startTransaction

(4)
{
"currentStatus":"enterPin",
"prompts":"Last Enter PIN"
"jsonData":{
"pinStatus":"LAST"
"pinLen":0
}
}			

Set parameter (TLV) from smart device:
(1)
{
"currentStatus": "setParamByClient",
"prompts": "Please Set Param",
"jsonData": {
"configTlv":
"031B0100031C10343136363636363636363636363636360340003052334313636363636363636363636363636363D32303131323031303133233435363738393003060003 1A0003070432303131",
"emvTlv":
"4F07A00000000310105A084166666666666666",
"timeout": 30
}
}
 	
(2)
{
"currentStatus": "setParamByClient",
"prompts": "Please Set Param",
"jsonData": {
"configTlv": "",
"emvTlv":
"4F07A00000000310105A084166666666666666",
"timeout": 30
}
}
	Set parameter (TLV) from smart device:
	{
	"currentStatus": "setParamByClient",
	"jsonData": {
	"configTlv":
	"0202010102030101",
	}
	}
		For detail description, please see Appendix 7 – Set parameter from smart device while startTransaction	startTransaction

(3)
{
"currentStatus": "setParamByClient",
"prompts": "Please Set Param",
"jsonData": {
"configTlv":
"031B0100031C10343136363636363636363636363636360340003052334313636363636363636363636363636363D32303131323031303132333435363738393003060003 1A0003070432303131",
"emvTlv": "",
"timeout": 30
}
}
 	
Note:
1) “configTlv” is the TLV String converted from TLV(bcd) that the EasyLink App get these TLVs (bcd) data according to the TAGs as TAG 0219 indicated;
2) “emvTlv” is the TLV String converted from TLV(bcd) that the EasyLink App get the these TLVs (bcd) data according to the TAGs as TAG 021A indicated;
3) If TAG 0205 and TAG 0206 are set, then the value of the sensitive data (see Appendix 11 – Sensitive TAG list) shall be encrypted.			